

CHIẾN LƯỢC THIẾT KẾ



TÓM TẮT

Phần C là phần dày nhất và cũng là phần sinh ra nhiều lời giải nhất trong thực tế. Tám chương ứng với tám khuôn tư duy: vết cặn có cất tĩa, chia để trị, tham lam, quy hoạch động, đồ thị, luồng và ghép đôi, tìm kiếm trên đáp án, và ngẫu nhiên hoá. Điểm chung của cả tám: mỗi khuôn khai thác một loại cấu trúc riêng — tính con-bài-toán-lập-lại, tính chọn-được-cục-bộ, tính đơn điệu, tính đối ngẫu — và câu hỏi cốt lõi của phần này là “bài toán trước mặt để lộ ra cấu trúc nào”. Nếu chỉ nhớ một điều: mọi lời giải nhanh đều bắt đầu từ một lời giải vết cặn đúng, rồi tìm cách bỏ đi phần công việc lặp lại hoặc chắt chắt vô ích.

Trình tự tám chương phản ánh một mạch phát triển ý tưởng chứ không phải một danh sách. Chương 13 bắt đầu từ vết cặn, cách giải luôn tồn tại và luôn đúng — và chỉ ra rằng phần lớn thuật toán hay chỉ là vết cặn cộng thêm một quan sát giúp bỏ bớt nhánh. Chương 14 lấy ý tưởng “bỏ bớt” đó theo hướng cấu trúc đệ quy: nếu bài chia được thành các phần độc lập thì chi phí sụp xuống theo hệ thức truy hồi. Chương 15 và 16 là hai cách khai thác con bài toán chồng lặp: tham lam khi lựa chọn cục bộ chứng minh được là an toàn, quy hoạch động khi không.

Chương 17 và 18 chuyển sang một ngôn ngữ khác: khi bạn phát biểu lại bài toán thành đồ thị, rất nhiều bài tưởng khác nhau hoá ra là cùng một bài. Đây là kỹ năng *mô hình hoá*, và nó đáng giá hơn việc thuộc thêm mười thuật toán. Chương 19 và 20 khép lại bằng hai công cụ có tỉ lệ hiệu quả trên độ khó cao bất thường: nhị phân trên đáp án biến bài tối ưu thành bài kiểm tra, và ngẫu nhiên hoá cho lời giải đơn giản đến bất ngờ cho những bài mà lời giải tất định rất rắc rối.

Bản đồ phần C — Chiến lược thiết kế

TÓM TẮT

Tám chương, tám khuôn tư duy, và mỗi khuôn có một *dấu hiệu nhận biết* trên đề bài. Bản đồ này liệt kê tám dấu hiệu đó thành bảng để dùng lúc đang bí. Chuỗi phụ thuộc: 13 → 14 → 16 là mạch chính; 15 nên đọc ngay trước 16 để thấy ranh giới giữa tham lam và quy hoạch động; 17 là điều kiện tiên quyết của 18; 19 và 20 độc lập, đọc sớm được và nên đọc sớm.

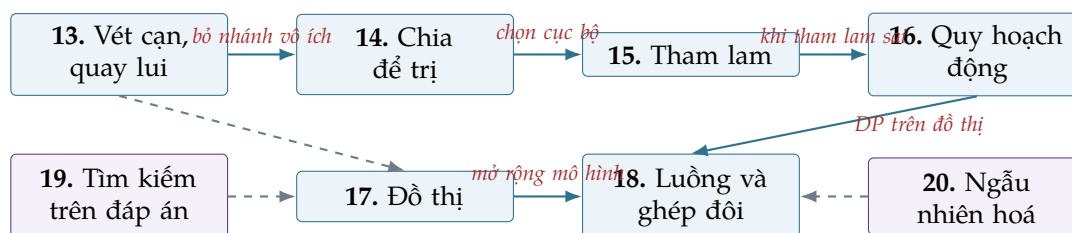
A. Bảng dấu hiệu: đề bài nói gì → thử khuôn nào

Dấu hiệu trên đề	Chương	Vì sao
$n \leq 20-25$, hỏi “có tồn tại cách nào”	13	không gian 2^n còn duyệt nổi
Bài chia đôi mà hai nửa không ảnh hưởng nhau	14	truy hồi $T(n) = 2T(n/2) + f(n)$
Xếp lịch, chọn nhiều nhất, luôn có “cái tốt nhất hiện tại”	15	thử đổi chỗ để chứng minh an toàn
Đếm số cách, giá trị tối ưu, có trạng thái nhỏ	16	con bài toán lập lại
Quan hệ đôi một, đường đi, phụ thuộc	17	phát biểu lại thành đồ thị
Phân công, sức chứa, “mỗi bên dùng một lần”	18	luồng cực đại hoặc ghép đôi
“Nhỏ nhất của giá trị lớn nhất”, đáp án đơn điệu	19	nhị phân trên đáp án
Bài tất định rắc rối, chỉ cần đúng với xác suất cao	20	ngẫu nhiên hoá

B. Vì sao chia như vậy

Tám khuôn này không phải tám chủ đề rời rạc mà là một cây quyết định. Gốc cây luôn là vết cạn: viết ra lời giải chậm nhưng chắc chắn đúng, rồi hỏi *tại sao nó chậm*. Nếu chậm vì tính lại cùng một thứ nhiều lần, đi hướng quy hoạch động. Nếu chậm vì duyệt những nhánh chắc chắn không tối ưu, đi hướng tham lam hoặc cắt tỉa. Nếu chậm vì bài toán bị nhìn sai dạng, đi hướng mô hình hoá thành đồ thị hoặc luồng. Cách chia này giữ nguyên mạch suy nghĩ thật khi giải bài, thay vì trình bày tám thuật toán như tám hòn đảo.

C. Phụ thuộc và thứ tự đọc



Hình 1. Phụ thuộc giữa tám chương của phần C. Mạch trên là mạch “khai thác cấu trúc đệ quy”; mạch dưới là mạch “mô hình hoá”. Hai chương tô tím là công cụ cắt ngang, dùng được cho cả hai mạch.

D. Cốt lõi hay tham khảo

Cốt lõi: toàn bộ chương 15, 16, 17 và 19 — bốn chương này chiếm tỉ lệ áp đảo trong mọi kỳ thi, phỏng vấn và cả công việc thật. Trong chương 13, phần cắt tỉa và gặp nhau ở giữa. Trong chương 14, phần đọc hệ thức truy hồi. **Tham khảo:** luồng chi phí nhỏ nhất và Hungarian ở chương 18; các tối ưu quy hoạch động nâng cao (divide and conquer optimization, Knuth optimization) ở chương 16; các thuật toán ngẫu nhiên chuyên sâu ở chương 20.

Để đối chiếu cách chọn, xem hình đối chỗ ở [C/15](#), chiều duyệt ba lô ở [C/16](#), phản ví dụ BFS/Dijkstra ở [C/17](#), cạnh dư ngược ở [C/18](#) và vết chạy tìm kiếm đáp án ở [C/19](#).

Vết cạn, quay lui và cắt tỉa (Brute Force, Backtracking, Pruning)

PHỤ THUỘC A/01 cho vai trò của vết cạn trong bước 2; A/03 cho việc ước lượng kích thước không gian tìm kiếm; B/07 cho ngăn xếp và đệ quy.

TÓM TẮT

Vết cạn là lời giải luôn tồn tại và luôn đúng, nên nó là điểm xuất phát của mọi chương trong phần C. Chương này dựng nó thành một kỹ thuật đẳng hoàng: khuôn quay lui chuẩn, cách ước lượng kích thước không gian trước khi cài, và bốn nhóm cắt tỉa — cắt theo cận, cắt theo đối xứng, cắt theo ràng buộc lan truyền, và sắp thứ tự biến để thất bại sớm. Chương cũng nói về gặp nhau ở giữa, kỹ thuật biến 2^n thành $2^{n/2}$, và về ranh giới quan trọng: khi nào vết cạn có cắt tỉa *chính là* lời giải đúng chứ không phải giải pháp tạm. Nếu chỉ nhớ một điều: đừng hỏi “làm sao tránh vết cạn”, hãy hỏi “vết cạn của tôi đang lãng phí ở đâu” — câu thứ hai dẫn thẳng tới mọi kỹ thuật của bảy chương còn lại.

KIẾN THỨC CẦN CÓ TRƯỚC

- **không gian tìm kiếm** — tập mọi cấu hình mà thuật toán có thể phải xét; kích thước của nó quyết định vết cạn có khả thi hay không.
- **quay lui (backtracking)** — xây lời giải từng bước; khi một bước dẫn tới ngõ cụt thì huỷ bước đó và thử lựa chọn khác. Về bản chất là duyệt sâu trên cây các lời giải bộ phận.
- **cắt tỉa (pruning)** — loại bỏ cả một nhánh của cây tìm kiếm khi chứng minh được nhánh đó không chứa lời giải tốt hơn cái đã có.
- **nhánh cận (branch and bound)** — quay lui có kèm chặn: nếu cận tốt nhất có thể của một nhánh còn tệ hơn lời giải đã tìm được thì bỏ nhánh.
- **gặp nhau ở giữa (meet in the middle)** — chia bài toán thành hai nửa độc lập, liệt kê đủ mỗi nửa ($2^{n/2}$), rồi ghép hai danh sách bằng sắp xếp hoặc bảng băm.
- **phá đối xứng** — loại bỏ những cấu hình chỉ khác nhau bởi một phép hoán vị hay phản chiếu vô nghĩa, để không đếm và không duyệt trùng.

CÂU HỎI MỞ ĐẦU

- Q1. Vết cạn luôn đúng và luôn chậm. Vậy vì sao chương đầu tiên của phần thiết kế lại dành cho nó?
- Q2. Bài toán tám quân hậu có $64^8 \approx 2,8 \cdot 10^{14}$ cách đặt thô, nhưng chương trình quay lui giải trong tích tắc. Ba tầng cắt giảm nào đã xảy ra?
- Q3. Cắt tỉa không đối độ phức tạp trường hợp xấu nhất. Vậy tại sao nó lại tạo khác biệt lớn tới vậy trong thực tế?
- Q4. Kỹ thuật gặp nhau ở giữa đưa 2^n xuống $2^{n/2}$ — vẫn là hàm mũ. Vì sao điều đó vẫn thay đổi được kết cục của bài toán?
- Q5. Khi nào một lời giải vết cạn có cắt tỉa là câu trả lời *đúng* cho bài toán, chứ không phải là giải pháp tạm bợ?

Phần A kết thúc bằng một lời khuyên lặp lại nhiều lần: khi bí, hãy viết ra lời giải vết cặn trước. Chương này giữ đúng lời khuyên đó nhưng đi xa hơn — nó chỉ ra rằng vết cặn không chỉ là bước đệm mà là một kỹ thuật có kỹ thuật riêng, và trong một số bài, vết cặn được cắt tia tốt *chính là* lời giải tốt nhất mà loài người biết.

Điều này đáng nhấn mạnh vì người mới thường coi vết cặn là thứ đáng xấu hổ. Thực tế thì các bộ giải SAT hiện đại — công cụ giải những bài logic công nghiệp với hàng triệu biến — về bản chất là quay lui có học từ thất bại. Các máy chơi cờ mạnh nhất trong nhiều thập kỷ là tìm kiếm có cắt tia. Đó không phải giải pháp tạm; đó là trạng thái nghệ thuật cho những bài toán mà không ai biết lời giải đa thức.

1.1 Ước lượng trước khi cài

Việc đầu tiên, và nó tốn ba mươi giây: *ước lượng kích thước không gian tìm kiếm*. Ba dạng thường gặp và cách nhận ra chúng trên đề bài:

Tập con — 2^n cấu hình. Dấu hiệu: mỗi phần tử chỉ có hai lựa chọn, chọn hoặc không. Khả thi tới $n \approx 25$.

Hoán vị — $n!$ cấu hình. Dấu hiệu: sắp thứ tự, gán một-một. Khả thi tới $n \approx 11$.

Gán k giá trị cho n biến — k^n . Dấu hiệu: tô màu, điền số. Khả thi khi k^n dưới cỡ 10^8 .

Bảng ở chương A/03 biến các con số này thành quyết định. Nhưng có một điều quan trọng hơn con số thô: ước lượng đó là *ước lượng trước khi cắt tia*. Với cắt tia tốt, số nhánh thực sự được duyệt có thể nhỏ hơn hàng chục bậc — và đó chính là lý do bài tám quân hậu giải được. Vì vậy quy trình đúng là: ước lượng thô để biết mình đang ở đâu, rồi hỏi “ràng buộc nào của đề cho phép tôi cắt sớm”.

1.2 Khuôn quay lui và bốn tầng cắt

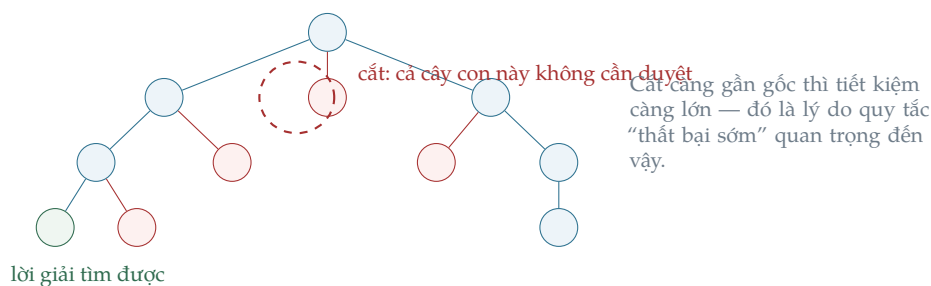
Mọi thuật toán quay lui đều có cùng khuôn: chọn biến tiếp theo, thử từng giá trị khả dĩ, kiểm tra tính hợp lệ, đệ quy, rồi huỷ lựa chọn. Phần khác nhau — và phần tạo ra toàn bộ hiệu năng — nằm ở bốn chỗ cắt.

Tầng 1 — cắt theo ràng buộc ngay khi đặt. Đừng đặt xong cả cấu hình rồi mới kiểm tra. Với bài tám quân hậu, ngay khi đặt quân thứ hai đã kiểm tra xung đột với quân thứ nhất; nhánh nào vi phạm thì cắt luôn. Chỉ tầng này thôi đã đưa 64^8 xuống cỡ vài nghìn nút.

Tầng 2 — phá đối xứng. Nhiều cấu hình chỉ khác nhau bởi một phép hoán vị vô nghĩa. Nếu bài không phân biệt thứ tự các phần tử được chọn, hãy ép chỉ số tăng dần; nếu bàn cờ đối xứng, hãy cố định quân đầu tiên ở nửa trái. Cắt này thường cho hệ số 2 tới $n!$ tùy bài, và nó cũng chữa luôn lỗi đếm trùng — một lỗi rất hay gặp trong các bài đếm.

Tầng 3 — cắt theo cận (nhánh cận). Nếu đang tìm cực tiểu và đã có lời giải giá B , thì với một nhánh mà ngay cả trong trường hợp tốt nhất vẫn không thể dưới B , hãy bỏ nhánh. Chất lượng của cắt này phụ thuộc vào chất lượng của hàm cận: cận càng chặt thì cắt càng nhiều, nhưng tính cận cũng tốn. Đây là một sự đánh đổi phải cân bằng bằng thực nghiệm.

Tầng 4 — thứ tự thử. Hai quy tắc kinh nghiệm mạnh. Thứ nhất, *chọn biến bị ràng buộc nhiều nhất trước* — ô sudoku còn ít khả năng nhất, quân cờ có ít nước đi nhất — vì như vậy thất bại xảy ra sớm, gần gốc, cắt được cây to. Thứ hai, *thử giá trị hứa hẹn nhất trước* khi tìm cực trị, vì tìm được lời giải tốt sớm làm cận ở tầng 3 chặt hơn cho toàn bộ phần còn lại.



Hình 1.1. Cây quay lui với các nhánh bị cắt (đỏ). Điều đáng nhớ: giá trị của một lần cắt tỉ lệ với kích thước cây con bị loại, mà kích thước đó giảm theo hàm mũ theo độ sâu. Vì vậy một cắt ở tầng 2 đáng giá hơn hàng nghìn cắt ở tầng 10, và đó là toàn bộ lý do của quy tắc “chọn biến bị ràng buộc nhiều nhất trước”.

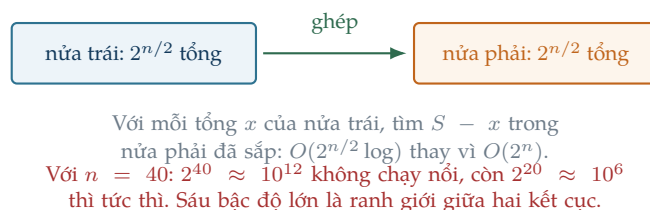
CẦN NHỚ

Cắt tia *không* cải thiện độ phức tạp trường hợp xấu nhất — bài toán vẫn là hàm mũ. Nó chỉ thay đổi trường hợp trung bình trên dữ liệu thực tế, nhưng thay đổi đó thường là vài bậc độ lớn, đủ để biến “không chạy nổi” thành “xong trong một giây”. Đây là một trong số ít chỗ trong quyển sách này mà lý thuyết và thực hành nói hai câu chuyện khác nhau, và cả hai đều đúng.

1.3 Gặp nhau ở giữa: đổi bộ nhớ lấy thời gian

Khi n khoảng 40, $2^n \approx 10^{12}$ là quá lớn, nhưng $2^{n/2} \approx 10^6$ thì rất thoải mái. Kỹ thuật gặp nhau ở giữa khai thác đúng khe đó: chia tập thành hai nửa, liệt kê đủ mọi tập con của mỗi nửa, rồi ghép hai danh sách.

Ví dụ chuẩn: cho $n \leq 40$ số, hỏi có tập con nào có tổng bằng S không. Liệt kê mọi tổng của nửa đầu (2^{20} giá trị) và mọi tổng của nửa sau; sắp xếp danh sách thứ hai, rồi với mỗi tổng x của nửa đầu, tìm nhị phân $S - x$. Chi phí $O(2^{n/2} \cdot n)$ thời gian và $O(2^{n/2})$ bộ nhớ.



Hình 1.2. Gặp nhau ở giữa. Điều cần nhận ra trên đề bài chính là giới hạn: n trong khoảng 30–45 nằm đúng giữa hai vùng — quá lớn cho 2^n , quá nhỏ để đòi hỏi lời giải đa thức — nên nó gần như luôn là lời mời dùng kỹ thuật này.

Điều đáng nhớ về mặt tư duy không phải bản thân kỹ thuật mà là *dấu hiệu* nhận biết: giới hạn n khoảng 30–45 gần như luôn là lời mời gặp nhau ở giữa, vì nó nằm đúng giữa hai vùng — quá lớn cho 2^n , quá nhỏ để đòi hỏi lời giải đa thức. Đây là ví dụ rõ nhất cho luận điểm ở chương A/03: giới hạn trên đề là một nửa gợi ý.

Điều kiện áp dụng cũng cần nói rõ: hai nửa phải *độc lập* và phải *ghép được bằng một phép tra cứu*. Nếu quyết định ở nửa sau phụ thuộc chi tiết vào cấu hình cụ thể của nửa đầu chứ không chỉ vào một giá trị tổng hợp, thì kỹ thuật này không áp dụng — và lúc đó bạn đang ở địa hạt của quy hoạch động (C/16).

1.4 Khi vết cặn là câu trả lời đúng

Có ba tình huống mà vết cặn có cắt tia không phải giải pháp tạm mà là lời giải tốt nhất đã biết.

Thứ nhất: bài toán NP-khó với dữ liệu nhỏ. Không ai biết thuật toán đa thức, và n đủ nhỏ. Lúc đó câu hỏi không còn là “có tránh được hàm mũ không” mà là “hằng số của hàm mũ nhỏ tới đâu”. Chương D/23 bàn kỹ.

Thứ hai: bài toán công nghiệp có cấu trúc. Các bài logic, lập lịch và kiểm chứng phần cứng trong thực tế có rất nhiều cấu trúc ẩn, và các bộ giải hiện đại khai thác được. Một bộ giải SAT hiện nay có thể xử lý những bài hàng triệu biến — không phải vì nó tránh được hàm mũ mà vì nó *học từ mỗi lần thất bại*: khi một nhánh mâu thuẫn, nó suy ra một mệnh đề mới ngăn cả một lớp cấu hình tương tự. Đây là bước tiến khái niệm quan trọng nhất của quay lui trong ba chục năm qua.

Thứ ba: khi bạn cần một lời giải chắc chắn đúng để đối chiếu. Trong quy trình stress test ở chương E/25, lời giải vết cặn đóng vai trò trọng tài. Nó chậm nhưng chạy trên dữ liệu nhỏ, và nó là thứ duy nhất bạn tin tuyệt đối.

1.5 Gốc rễ: từ trò chơi ô chữ tới bộ giải công nghiệp

Quay lui là kỹ thuật cổ, và nó có tên trước khi có máy tính hiện đại: nhà toán học D. H. Lehmer đặt ra thuật ngữ *backtrack* trong thập niên 1950 khi mô tả cách giải các bài toán tổ hợp bằng máy. Đến năm 1965, Golomb và Baumert viết một khảo sát hệ thống hoá kỹ thuật này, và nhiều mẹo trong mục 2 — phá đối xứng, chọn biên ràng buộc nhiều nhất — đã có mặt ở đó.

Có một đóng góp của Knuth đáng nhắc riêng vì nó rất thực dụng và ít người biết. Năm 1975, ông công bố một cách *ước lượng* kích thước cây quay lui mà không phải duyệt hết: chạy thử một đường ngẫu nhiên từ gốc xuống lá, mỗi bước nhân số nhánh khả dĩ vào một tích, lặp lại vài trăm lần rồi lấy trung bình. Kết quả là ước lượng không chệch cho tổng số nút. Ý nghĩa thực hành rất lớn: trước khi bỏ hai giờ chờ một chương trình quay lui, bạn tốn vài giây để biết nó sẽ chạy vài giây hay vài năm. Đây là một ví dụ đẹp cho tinh thần “đo trước khi làm” ở chương A/02.

Nhánh nhánh cặn đến từ nghiên cứu vận trù học: Land và Doig đề xuất nó năm 1960 cho quy hoạch nguyên, tức là để giải các bài tối ưu công nghiệp có biến nguyên. Nhánh SAT bắt đầu năm 1962 với thuật toán DPLL — vẫn là quay lui, cộng thêm hai quy tắc lan truyền ràng buộc. Bước nhảy vọt thật sự đến vào cuối thập niên 1990 với ý tưởng *học mệnh đề từ xung đột*: mỗi lần bế tắc, bộ giải phân tích nguyên nhân và ghi lại một ràng buộc mới. Chính bước này biến SAT từ bài toán học thuật thành công cụ công nghiệp, dùng để kiểm chứng thiết kế vi mạch và phân tích chương trình.

Cuối cùng, một liên hệ đáng nhớ với miền khác: trong thị giác máy tính và robot, khi bài toán ước lượng có quá nhiều tổ hợp dữ liệu ngoại lai, người ta không vết cặn mà *lấy mẫu ngẫu nhiên* — kỹ thuật RANSAC. Đó là một câu trả lời khác cho cùng câu hỏi “không gian quá lớn thì làm gì”, và chương C/20 sẽ khai thác chính hướng đó.

Gốc rễ: ý tưởng này ra đời từ bài toán nào

Mốc	Bài toán thúc ép lúc đó	Điều thay đổi sau đó
Thập niên 1950 — Lehmer	Giải các bài toán tổ hợp bằng máy tính đời đầu.	Thuật ngữ <i>backtrack</i> ; ý tưởng huỷ lựa chọn và thử nhánh khác được đặt tên.
1960 — Land & Doig	Bài tối ưu công nghiệp có biến nguyên, không giải được bằng quy hoạch tuyến tính.	Nhánh cặn: dùng cặn nói lòng để cắt nhánh; nền của các bộ giải quy hoạch nguyên ngày nay.

Mốc	Bài toán thúc ép lúc đó	Điều thay đổi sau đó (tiếp)
1962 — Davis, Putnam, Logemann, Loveland	Chứng minh tự động các mệnh đề logic.	DPLL: quay lui + lan truyền ràng buộc; khung sườn vẫn dùng tới nay.
1965 — Golomb & Baumert	Nhiều người tự phát minh lại cùng các mẹo quay lui.	Khảo sát hệ thống; phá đối xứng, chọn biến ràng buộc nhất, kiểm tra sớm.
1975 — Knuth	Không biết một chương trình quay lui sẽ chạy vài giây hay vài năm.	Ước lượng kích thước cây bằng cách lấy mẫu đường ngẫu nhiên — vài giây để biết có nên chạy hay không.
Cuối thập niên 1990 — học mệnh đề từ xung đột	Bộ giải SAT bế tắc rồi lặp lại đúng sai lầm ở nhánh khác.	Học từ mỗi thất bại thành một ràng buộc mới; biến SAT thành công cụ công nghiệp cho kiểm chứng vi mạch và phân tích chương trình.

1.6 Liên hệ ngang

Liên hệ ngang: ý này còn sống ở đâu nữa

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu
Cờ vua và trò chơi	Tìm kiếm cây trò chơi với cắt tỉa alpha-beta	Alpha-beta <i>chính là</i> nhánh cận cho trò chơi hai người: cận trên và cận dưới thay cho một cận. Cùng quy tắc thứ tự: thử nước hứa hẹn trước để cắt được nhiều hơn.
Kiểm chứng phần cứng, phân tích chương trình	Bộ giải SAT/SMT	Cùng khung quay lui, cộng học từ xung đột. Bài học chuyển được: khi bế tắc, đừng chỉ lùi một bước — hãy hỏi <i>nguyên nhân</i> bế tắc và ghi lại nó để không lặp lại.
Vận trù học	Quy hoạch nguyên, nhánh cận với nới lỏng tuyến tính	Cùng cấu trúc; cận đến từ việc bỏ ràng buộc nguyên và giải bài liên tục. Ý tưởng chung rất mạnh: cận tốt thường đến từ một <i>phiên bản dễ hơn</i> của chính bài toán — đúng heuristic “bỏ bớt ràng buộc” ở A/01.
Trí tuệ nhân tạo cổ điển	Thoả mãn ràng buộc, lan truyền ràng buộc, cung nhất quán	Tầng cắt thứ nhất và thứ tư của mục 2 chính là các kỹ thuật ở đó dưới tên khác. Cùng phát hiện: chi phí lan truyền ràng buộc thường rẻ hơn nhiều so với chi phí duyệt nhánh chết.
Thị giác máy tính, SLAM	RANSAC: lấy mẫu ngẫu nhiên thay vì duyệt mọi tổ hợp	Câu trả lời khác cho cùng vấn đề “không gian quá lớn”. Khác biệt cốt lõi: RANSAC từ bỏ tính đầy đủ để đổi lấy tốc độ, còn quay lui giữ tính đầy đủ và cắt những nhánh <i>chứng minh được</i> là vô ích.
Hoá học, sinh học tính toán	Tìm cấu hình phân tử, gấp protein	Không gian cấu hình khổng lồ, cắt theo năng lượng chính là nhánh cận. Khác: hàm mục tiêu ở đó chỉ là mô hình xấp xỉ, nên “lời giải tối ưu” của bài toán không chắc là lời giải đúng của thực tại.
Mật mã	Tấn công vết cặn khoá, đánh đổi thời gian-bộ nhớ	Gặp nhau ở giữa là một kỹ thuật thám mã kinh điển; nó chính là lý do một số sơ đồ mã hoá kép không an toàn gấp đôi như trực giác gợi ý.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì nó là điểm xuất phát chung của cả bảy chương còn lại. Mọi kỹ thuật sau này đều được sinh ra bằng cách hỏi “vết cặn lãng phí ở đâu”: tính lại thứ đã tính dẫn tới quy hoạch động, duyệt nhánh chắc chắn tệ dẫn tới tham lam và cắt tỉa, nhìn sai dạng bài dẫn tới mô hình hoá lại. Ngoài

ra vết cạn còn là trọng tài cho stress test, và trong nhiều bài NP-khó nó là lời giải tốt nhất đã biết.

- A2. Tầng một: mỗi cột chỉ đặt một quân, đưa 64^8 xuống $8^8 \approx 1,7 \cdot 10^7$. Tầng hai: kiểm tra xung đột hàng và đường chéo *ngay khi đặt* thay vì đặt xong mới kiểm, cắt phần lớn nhánh ở gần gốc và đưa số nút xuống cỡ vài nghìn. Tầng ba: phá đối xứng, ví dụ chỉ đặt quân đầu ở nửa trái rồi nhân đôi kết quả. Điều đáng rút ra là tầng hai — kiểm tra sớm — đóng góp lớn nhất, đúng như hình 1.1 chỉ ra.
- A3. Vì trường hợp xấu nhất hiếm khi là trường hợp bạn gặp. Dữ liệu thực tế có cấu trúc: các ràng buộc mâu thuẫn nhau sớm, các cặn chặn chặt, và phần lớn cây tìm kiếm chết ở gần gốc. Cắt tỉa khai thác đúng cấu trúc đó. Nói theo ngôn ngữ chương A/03: độ phức tạp tiệm cận mô tả trường hợp xấu nhất của *mọi* dữ liệu, còn cắt tỉa cải thiện phân phối thực tế — hai câu chuyện khác nhau và cả hai đều đúng.
- A4. Vì hàm mũ với số mũ bằng nửa là một hàm hoàn toàn khác về mặt thực tiễn: 2^{40} là 10^{12} , còn 2^{20} là 10^6 . Sáu bậc độ lớn là ranh giới giữa “không chạy nổi” và “xong tức thì”. Cái giá là bộ nhớ $O(2^{n/2})$, tức là ta đổi thời gian lấy bộ nhớ — một đánh đổi rất đáng khi bộ nhớ còn dư. Dấu hiệu nhận biết trên đề: n khoảng 30–45.
- A5. Ba tình huống. Khi bài toán NP-khó và n nhỏ — không tồn tại lời giải đa thức đã biết, nên câu hỏi chỉ còn là hằng số của hàm mũ. Khi bài toán công nghiệp có cấu trúc mà bộ giải khai thác được — SAT và quy hoạch nguyên giải những bài hàng triệu biến bằng đúng cách này. Và khi bạn cần một lời giải chắc chắn đúng làm trọng tài cho stress test (E/25). Trong cả ba, vết cạn không phải bước lùi mà là lựa chọn có cân nhắc.

NÓI LẠI THẬT ĐƠN GIẢN

Vết cạn là cách giải mà ai cũng nghĩ ra: thử hết mọi khả năng. Vấn đề duy nhất là “mọi khả năng” thường nhiều tới mức không sống nổi để thử hết.

Nhưng hãy để ý cách một người thật sự đi tìm chìa khoá trong nhà. Họ không mở lần lượt từng ngăn kéo trong toàn bộ căn nhà. Họ nghĩ: “chìa khoá không thể ở trong tủ lạnh” — và loại cả căn bếp bằng một câu. Đó chính là cắt tỉa: bỏ cả một vùng vì *chắc chắn* không có gì trong đó, chứ không phải vì lười tìm.

Ba mẹo mà người tìm chìa khoá dùng cũng chính là ba mẹo trong chương này. Một, kiểm tra sớm: vừa mở ngăn kéo đã thấy nó rỗng thì đóng lại ngay, đừng lục từng góc. Hai, đừng tìm hai lần cùng một chỗ chỉ vì bạn đi vào phòng theo hướng khác — đó là phá đối xứng. Ba, tìm chỗ khả nghi nhất trước; nếu tìm thấy sớm thì bạn khỏi phải lục phần còn lại.

Có một mẹo thứ tư rất hay dùng khi số khả năng quá lớn: chia đôi bài toán. Nếu bạn cần tìm hai món ghép lại thành một tổng cho trước, đừng thử mọi cặp. Hãy liệt kê mọi khả năng của nửa trái ra một danh sách, rồi với mỗi khả năng của nửa phải, tra xem cái bù của nó có trong danh sách không. Số việc từ “nhân” thành “cộng”, và đó là toàn bộ ý của ghép nhau ở giữa.

Cuối cùng, đừng coi thường vết cạn. Có những bài mà cả thế giới không biết cách nào tốt hơn ngoài thử hết một cách thông minh — và các phần mềm kiểm tra thiết kế vi mạch ngày nay chính là những cỗ máy vết cạn rất giỏi cắt tỉa và biết rút kinh nghiệm sau mỗi lần đâm vào ngõ cụt.

Một câu: Đừng hỏi làm sao tránh vết cạn; hãy hỏi vết cạn của tôi đang phí công ở đâu.

Chỗ không rút gọn được: Không có cách nào biết trước một hàm cặn có đủ chặt để cắt hiệu quả hay không; phải cài và đo.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Vai trò của vết cạn	Điểm xuất phát của mọi kỹ thuật ở phần C; trọng tài cho stress test; và là lời giải tốt nhất đã biết cho nhiều bài NP-khó.
••• Ba dạng không gian và ngưỡng	Tập con 2^n (tới $n \approx 25$); hoán vị $n!$ (tới $n \approx 11$); gán k giá trị k^n . Ước lượng trước khi cài.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●●● Bốn tầng cắt	Kiểm tra ràng buộc ngay khi đặt; phá đối xứng; cắt theo cặn (nhánh cặn); chọn thứ tự biến và giá trị để thất bại sớm.
●●● Vì sao “thất bại sớm” quan trọng	Giá trị của một lần cắt tỉ lệ với kích thước cây con bị loại, mà cây con giảm theo hàm mũ theo độ sâu — cắt gần gốc đáng giá gấp bội.
●●● Cắt tỉa và độ phức tạp	Không cải thiện trường hợp xấu nhất; chỉ cải thiện phân phối thực tế — nhưng thường vài bậc độ lớn.
●●● Gặp nhau ở giữa	Chia đôi, liệt kê $2^{n/2}$ mỗi nửa, ghép bằng sắp xếp hoặc băm. Dấu hiệu: n khoảng 30–45. Đổi bộ nhớ lấy thời gian.
●● Điều kiện của gặp nhau ở giữa	Hai nửa độc lập và ghép được qua một giá trị tổng hợp. Nếu nửa sau cần biết chi tiết cấu hình nửa đầu thì phải chuyển sang quy hoạch động.
●● Hai quy tắc thứ tự	Chọn biến bị ràng buộc nhiều nhất trước; thử giá trị hứa hẹn nhất trước (để cặn sớm chặt hơn).
●● Ước lượng cây quay lui của Knuth	Chạy vài trăm đường ngẫu nhiên từ gốc, nhân số nhánh dọc đường, lấy trung bình — ước lượng không chệch cho tổng số nút.
●● Học từ xung đột	Khi bế tắc, phân tích nguyên nhân và ghi lại thành một ràng buộc mới để không lặp lại ở nhánh khác. Bước tiến lớn nhất của quay lui hiện đại.
●● Alpha-beta là gì	Nhánh cặn cho trò chơi hai người: duy trì đồng thời cặn trên và cặn dưới. Cùng quy tắc thứ tự nước đi.
● Nguồn của một cặn tốt	Thường là lời giải của một <i>phiên bản dễ hơn</i> của chính bài toán (bỏ bớt ràng buộc) — đúng heuristic của A/01.

●●● phải thuộc ●● nên nắm ● biết là có

I Câu hỏi ôn tập có đáp án (che phần đáp án, tự trả lời trước)

1. Làm sao biết chương trình quay lui của tôi sẽ chạy bao lâu trước khi chạy?

| Dùng phép ước lượng của Knuth ở mục 5: chạy vài trăm lần một đường đi ngẫu nhiên từ gốc xuống, tại mỗi nút nhân vào tích số nhánh khả dĩ, và lấy trung bình các tích. Con số đó là ước lượng không chệch cho tổng số nút của cây. Cách này tốn vài giây và cho bạn biết ngay thứ tự độ lớn — rất đáng làm trước khi ngồi chờ một chương trình có thể chạy hàng giờ.

2. Quay lui nên viết đệ quy hay dùng ngăn xếp tường minh?

| Đệ quy trong hầu hết trường hợp: nó ngắn hơn, khớp với cấu trúc bài toán, và việc “huỷ lựa chọn” xảy ra tự nhiên khi hàm trả về. Chuyển sang ngăn xếp tường minh chỉ khi độ sâu quá lớn gây tràn, hoặc khi bạn cần tạm dừng và tiếp tục việc tìm kiếm — ví dụ tìm kiếm có giới hạn thời gian trong một hệ thống thật.

3. Tôi nên phá đối xứng thế nào cho đúng, tránh làm mất nghiệm?

| Nguyên tắc: đặt một *thứ tự chuẩn tắc* và chỉ duyệt các cấu hình chuẩn tắc. Ví dụ với tập con thì ép chỉ số tăng dần; với việc chia n phần tử vào các nhóm không phân biệt thì ép phần tử đầu tiên của mỗi nhóm mới phải tăng dần. Cách kiểm tra an toàn: đếm số nghiệm với và không có phá đối xứng trên n nhỏ, rồi đối chiếu với con số kỳ vọng (có thể phải nhân lại hệ số đối xứng).

4. Nhánh cặn cần hàm cặn, mà hàm cặn lấy ở đâu ra?

| Gần như luôn từ một *phiên bản nói lỏng* của bài toán: bỏ bớt một ràng buộc rồi giải bài dễ hơn. Với bài ba lô, cặn là lời giải của phiên bản cho phép lấy phần lẻ của món đồ — giải bằng tham lam. Với bài người bán hàng, cặn có thể là tổng các cạnh nhẹ nhất tại mỗi đỉnh, hoặc trọng số cây khung nhỏ nhất. Đây là ứng dụng trực tiếp của heuristic “bỏ bớt ràng buộc” ở A/01: bài dễ hơn không chỉ giúp bạn hiểu bài, nó còn cho bạn một cặn.

5. Khi nào quay lui thua quy hoạch động và ngược lại?

| Quy hoạch động thắng khi các nhánh khác nhau dẫn tới *cùng một trạng thái* và trạng thái đó mô tả được gọn — lúc đó lưu lại kết quả tránh tính lại. Quay lui thắng khi không gian trạng thái quá lớn hoặc không nên được thành ít chiều, nhưng ràng buộc lại chặt tới mức phần lớn nhánh chết sớm. Dấu hiệu thực dụng: nếu bạn viết được một khoá trạng thái ngắn thì hãy nghĩ tới quy hoạch động (C/16); nếu khoá đó phải chứa cả cấu hình đã chọn thì quay lui có cắt tỉa thường là hướng đúng.

6. Có nên dùng ghi nhớ cho quay lui không?

| Có, và ranh giới giữa hai kỹ thuật mờ hơn người ta tưởng. Nếu nhiều nhánh dẫn tới cùng trạng thái, hãy lưu kết quả — lúc đó bạn đã đi từ quay lui sang quy hoạch động mà không đổi cấu trúc code. Điều cần cẩn thận là khoá trạng thái: nó phải *đủ* để xác định phần còn lại của bài toán. Khoá thiếu thông tin cho kết quả sai một cách im lặng, và đây là lỗi phổ biến nhất khi thêm ghi nhớ vào một hàm đệ quy đã chạy đúng.

7. Bộ giải SAT thực sự làm gì khác so với quay lui trong sách giáo khoa?

| Ba thứ, và cả ba đều là ý tưởng chuyển được. Một, lan truyền ràng buộc rất mạnh: mỗi lần gán một biến, suy ra ngay mọi hệ quả bắt buộc. Hai, học mệnh đề từ xung đột: khi bế tắc, phân tích *nguyên nhân* và ghi lại một ràng buộc mới, nhờ đó không lặp lại sai lầm ở nhánh khác. Ba, quay lui không tuần tự: thay vì lùi một bước, nhảy thẳng về mức thực sự gây ra mâu thuẫn. Ý tưởng thứ hai là thứ đáng mang theo nhất, kể cả khi bạn tự viết quay lui cho một bài cụ thể.

I Bài tập (mức: ● khởi động ●● chuẩn ●●● khó)

Bài	Nguồn	Điều cần nhận ra
● Sinh mọi tập con và mọi hoán vị của n phần tử	CSES “Creating Strings”, “Apple Division”[1]	Viết cả bản đệ quy lẫn bản dùng mặt nạ bit; hiểu rõ cả hai.
● Bài toán tám quân hậu	CSES “Chessboard and Queens”[1]	Đếm số nút cây với và không có kiểm tra sớm, để thấy hiệu quả bằng số liệu.
●● Tô màu đồ thị bằng k màu	Bài kinh điển	Chọn đỉnh có nhiều màu bị cấm nhất trước — quy tắc “thất bại sớm”.
●● Giải sudoku	Bài kinh điển	Kết hợp lan truyền ràng buộc với chọn ô ít khả năng nhất; so thời gian với bản không có hai mẹo đó.
●● Tập con có tổng bằng S với $n \leq 40$	Bài kinh điển	Gặp nhau ở giữa; dấu hiệu là chính giới hạn n .
●● Chia n món đồ thành hai nhóm sao cho chênh lệch nhỏ nhất	CSES “Apple Division”[1]	$n \leq 20$ nên duyệt tập con là đủ; sau đó thử lại với $n = 40$ bằng gặp nhau ở giữa.
●●● Bài toán người bán hàng với $n \leq 20$	Bài kinh điển	Quy hoạch động bitmask (C/16); rồi thử nhánh cận và so hai cách trên cùng dữ liệu.
●●● Đếm số cách đặt n quân hậu, có phá đối xứng	Bài kinh điển	Kiểm chứng bằng cách so với dãy kết quả đã biết cho n nhỏ.
●●● Ước lượng kích thước cây quay lui bằng phương pháp Knuth	Bài tập thực nghiệm	Áp dụng cho bài n quân hậu; so ước lượng với số nút thật.
●●● Cài một bộ giải SAT đơn giản kiểu DPLL	Bài tập cài đặt	Thêm lan truyền đơn vị, rồi đo lại; đây là cách rẻ nhất để hiểu vì sao lan truyền ràng buộc mạnh đến vậy.
●●● Bài toán phủ chính xác (xếp hình pentomino hoặc tương tự)	Bài kinh điển	Cấu trúc “liên kết nhảy” giúp huỷ và khôi phục lựa chọn trong $O(1)$; một bài tập cài đặt đẹp.

Kết chương và đọc tiếp

Chương này mở phần C bằng thứ mà mọi lời giải đều bắt đầu. Ba điều đáng mang theo: ước lượng không gian trước khi cài; bốn tầng cắt, trong đó “thất bại sớm” đáng giá nhất vì cắt gần gốc; và ý thức rằng vết cạn có cắt tỉa đôi khi *chính là* lời giải đúng, không phải bước lùi.

Bảy chương còn lại của phần C đều trả lời câu hỏi “vết cạn của tôi lãng phí ở đâu”. Câu trả lời đầu tiên: nếu bài toán chia được thành các phần *độc lập* thì chi phí không cộng lại mà sụp xuống theo hệ thức truy hồi — đó là chia để trị (C/14), và nó cũng là nơi ta gặp lại một ý tưởng của chương B/09: ngẫu nhiên hoá để không có trường hợp xấu.

Nguồn

- [1] A. Laaksonen. “CSES Problem Set.” (2026), address: <https://cses.fi/problemset/> (visited on 09/06/2026).
- [2] M. Blum, R. W. Floyd, V. Pratt, et al., “Time Bounds for Selection,” *Journal of Computer and System Sciences*, vol. 7, no. 4, pp. 448–461, 1973.
- [3] V. Strassen, “Gaussian Elimination Is Not Optimal,” *Numerische Mathematik*, vol. 13, pp. 354–356, 1969.
- [4] D. Harvey and J. van der Hoeven, “Integer Multiplication in Time $O(n \log n)$,” *Annals of Mathematics*, vol. 193, no. 2, pp. 563–617, 2021.
- [5] J. Alman and V. Vassilevska Williams, “A Refined Laser Method and Faster Matrix Multiplication,” in *Proceedings of the 32nd ACM-SIAM Symposium on Discrete Algorithms (SODA)*, 2021.
- [6] J. Alman, R. Duan, V. Vassilevska Williams, et al. “Improving the Matrix Multiplication Exponent with Modern Optimization and AlphaEvolve.” bản tiền ấn arXiv:2608.16884, chưa qua phản biện. (2026), address: <https://arxiv.org/abs/2608.16884> (visited on 09/07/2026).
- [7] J. W. Cooley and J. W. Tukey, “An Algorithm for the Machine Calculation of Complex Fourier Series,” *Mathematics of Computation*, vol. 19, no. 90, pp. 297–301, 1965.
- [8] D. R. Musser, “Introspective Sorting and Selection Algorithms,” *Software: Practice and Experience*, vol. 27, no. 8, pp. 983–993, 1997.
- [9] J. L. Bentley and M. D. McIlroy, “Engineering a Sort Function,” *Software: Practice and Experience*, vol. 23, no. 11, pp. 1249–1265, 1993.
- [10] D. E. Knuth, “Optimum Binary Search Trees,” *Acta Informatica*, vol. 1, pp. 14–25, 1971.
- [11] F. F. Yao, “Speed-Up in Dynamic Programming,” *SIAM Journal on Algebraic and Discrete Methods*, vol. 3, no. 4, pp. 532–540, 1982.
- [12] A. Aggarwal, M. M. Klawe, S. Moran, et al., “Geometric Applications of a Matrix-Searching Algorithm,” *Algorithmica*, vol. 2, pp. 195–208, 1987.
- [13] D. S. Hirschberg, “A Linear Space Algorithm for Computing Maximal Common Subsequences,” *Communications of the ACM*, vol. 18, no. 6, pp. 341–343, 1975.
- [14] R. E. Tarjan, “Depth-First Search and Linear Graph Algorithms,” *SIAM Journal on Computing*, vol. 1, no. 2, pp. 146–160, 1972.
- [15] M. L. Fredman and R. E. Tarjan, “Fibonacci Heaps and Their Uses in Improved Network Optimization Algorithms,” *Journal of the ACM*, vol. 34, no. 3, pp. 596–615, 1987.
- [16] M. Thorup, “Undirected Single-Source Shortest Paths with Positive Integer Weights in Linear Time,” *Journal of the ACM*, vol. 46, no. 3, pp. 362–394, 1999.
- [17] R. Duan, J. Mao, X. Mao, et al., “Breaking the Sorting Barrier for Directed Single-Source Shortest Paths,” in *Proceedings of the 57th Annual ACM Symposium on Theory of Computing (STOC)*, Best Paper Award, 2025.
- [18] E. W. Dijkstra, “A Note on Two Problems in Connexion with Graphs,” *Numerische Mathematik*, vol. 1, pp. 269–271, 1959.

- [19] KTH Royal Institute of Technology. “KACTL: KTH Algorithm Competition Template Library.” (2026), address: <https://github.com/kth-competitive-programming/kactl> (visited on 09/06/2026).
- [20] H. W. Kuhn, “The Hungarian Method for the Assignment Problem,” *Naval Research Logistics Quarterly*, vol. 2, no. 1-2, pp. 83–97, 1955.
- [21] J. B. Orlin, “Max Flows in $O(nm)$ Time, or Better,” in *Proceedings of the 45th Annual ACM Symposium on Theory of Computing (STOC)*, 2013, pp. 765–774.
- [22] L. Chen, R. Kyng, Y. P. Liu, et al., “Maximum Flow and Minimum-Cost Flow in Almost-Linear Time,” in *Proceedings of the 63rd IEEE Symposium on Foundations of Computer Science (FOCS)*, Best Paper Award, 2022.
- [23] J. E. Hopcroft and R. M. Karp, “An $n^{5/2}$ Algorithm for Maximum Matchings in Bipartite Graphs,” *SIAM Journal on Computing*, vol. 2, no. 4, pp. 225–231, 1973.
- [24] J. Bentley, *Programming Pearls*, 2nd ed. Addison-Wesley, 2000.
- [25] R. Motwani and P. Raghavan, *Randomized Algorithms*. Cambridge University Press, 1995.

Chia để trị (Divide and Conquer)

PHỤ THUỘC A/03 cho hệ thức truy hồi; A/04 cho quy nạp — chia để trị và quy nạp là hai mặt của một đồng xu; C/13 cho điểm xuất phát vết cạn.

TÓM TẮT

Chia để trị khai thác một tính chất rất cụ thể: nếu bài toán tách được thành các phần *độc lập* thì tổng chi phí không cộng lại mà sụp xuống theo hệ thức truy hồi. Chương đi qua các đại diện — sắp xếp trộn, quicksort, chọn phần tử thứ k , cặp điểm gần nhất, nhân nhanh — nhưng trọng tâm không phải danh sách thuật toán mà là ba câu hỏi thiết kế: chia ở đâu, ghép thế nào, và chi phí ghép có nhỏ hơn lợi ích chia không. Chương cũng là nơi ta gặp lại ngẫu nhiên hoá lần thứ tư, ở dạng thuyết phục nhất: quicksort với chốt ngẫu nhiên. Nếu chỉ nhớ một điều: phần khó của chia để trị nằm ở bước *ghép*, không ở bước chia — và mọi thuật toán hay trong chương này đều là kết quả của việc tìm ra một cách ghép rẻ bất ngờ.

KIẾN THỨC CẦN CÓ TRƯỚC

- **chia để trị** — giải bài lớn bằng cách chia thành các bài con cùng dạng nhưng nhỏ hơn, giải chúng độc lập, rồi ghép kết quả.
- **bước ghép (combine)** — công đoạn tạo lời giải của bài lớn từ lời giải các bài con; đây là nơi chứa gần như toàn bộ độ khó.
- **hệ thức truy hồi** — $T(n) = aT(n/b) + f(n)$; a là số bài con, n/b là kích thước mỗi bài con, $f(n)$ là chi phí chia và ghép. Xem A/03.
- **sắp xếp trộn (merge sort)** — chia đôi, sắp mỗi nửa, rồi trộn hai nửa đã sắp trong $O(n)$. Ổn định, $O(n \log n)$ đảm bảo, nhưng cần bộ nhớ phụ.
- **quicksort** — chọn một chốt, phân hoạch mảng thành nhỏ hơn và lớn hơn, rồi đệ quy hai phía. Không cần bộ nhớ phụ, nhanh trong thực tế, nhưng có trường hợp xấu $O(n^2)$.
- **tính ổn định của sắp xếp** — các phần tử bằng nhau giữ nguyên thứ tự tương đối ban đầu. Quan trọng khi sắp nhiều lần theo nhiều khoá.

CÂU HỎI MỞ ĐẦU

- Q1. Chia đôi rồi giải hai nửa — nghe như vẫn phải làm hết công việc. Vậy tiết kiệm đến từ đâu?
- Q2. Vì sao quicksort thường nhanh hơn sắp xếp trộn trên máy thật, dù nó có trường hợp xấu $O(n^2)$ còn sắp xếp trộn thì không?
- Q3. Bài toán tìm cặp điểm gần nhất trong mặt phẳng: sau khi chia đôi và giải hai nửa, vì sao bước ghép lại *không* tốn $O(n^2)$?
- Q4. Nhân hai số n chữ số theo cách học ở trường tốn $O(n^2)$. Karatsuba xuống $O(n^{1,585})$ chỉ bằng một mẹo đại số. Mẹo đó thay đổi cái gì trong hệ thức truy hồi?
- Q5. Khi nào chia để trị là lựa chọn sai — tức là chia bài ra làm mọi thứ tệ hơn?

Chương trước kết thúc bằng câu hỏi “vết cạn lãng phí ở đâu”. Một câu trả lời rất phổ biến: vết cạn xét mọi cặp phần tử, trong khi phần lớn các cặp đó không liên quan gì tới nhau. Nếu ta chia

dữ liệu thành hai nửa và chứng minh được rằng phần lớn công việc chỉ xảy ra *bên trong* mỗi nửa, thì số cặp phải xét giảm mạnh, và giảm lặp đi lặp lại ở mỗi mức của đệ quy.

Đó là toàn bộ ý tưởng chia để trị, và nó có một đặc điểm dễ bị hiểu sai. Bước “chia” hầu như luôn tầm thường — cắt mảng làm đôi thì có gì khó. Cái khó, và cái tạo ra mọi thuật toán đẹp trong chương này, nằm ở bước *ghép*: làm sao dựng lời giải của bài lớn từ lời giải hai nửa mà không tốn lại $O(n^2)$. Khi bạn đọc mỗi thuật toán dưới đây, hãy đọc nó như câu trả lời cho câu hỏi ghép.

2.1 Ba câu hỏi thiết kế và hệ thức truy hồi

Khuôn chung: chia bài kích thước n thành a bài con kích thước n/b , giải đệ quy, rồi ghép với chi phí $f(n)$. Chi phí tổng là $T(n) = aT(n/b) + f(n)$, và chương A/03 đã cho cách đọc nó bằng hình ảnh “tầng nào nặng”.

Điều đáng rút ra từ hình đó, và nó định hướng toàn bộ việc thiết kế: *nếu chi phí ghép $f(n)$ đủ nhỏ thì tổng chi phí bị chi phối bởi các lá; nếu $f(n)$ lớn thì bị chi phối bởi gốc*. Với sắp xếp trộn, $a = 2$, $b = 2$, $f(n) = n$ — rơi đúng vào trường hợp cân bằng, cho $\Theta(n \log n)$. Thừa số $\log n$ chính là số tầng, và nó là cái giá phải trả cho việc bước ghép tốn tuyến tính.

Từ đó có ba câu hỏi thiết kế cần đặt mỗi khi định dùng chia để trị:

Chia ở đâu? Chia đôi đều là mặc định, nhưng không phải luôn đúng. Quicksort chia theo giá trị chốt nên hai phần có thể lệch; thuật toán chọn phần tử thứ k chỉ đệ quy *một* nhánh; thuật toán cặp điểm gần nhất chia theo toạ độ.

Các bài con có thật sự độc lập không? Nếu lời giải của nửa này ảnh hưởng tới nửa kia thì chia để trị không áp dụng trực tiếp, và bạn đang ở địa hạt quy hoạch động (C/16).

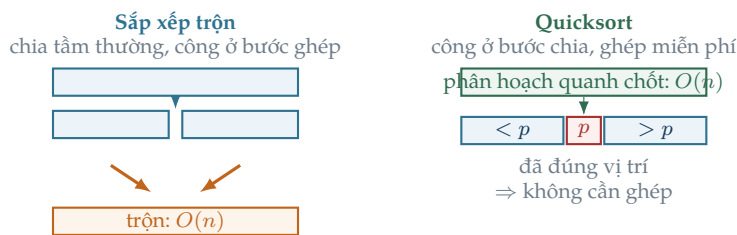
Ghép có rẻ hơn không? Nếu ghép vẫn tốn $O(n^2)$ thì chẳng tiết kiệm gì. Câu hỏi này là chỗ đòi hỏi sáng tạo thật sự, và ba ví dụ dưới đây là ba câu trả lời đẹp.

2.2 Sắp xếp: hai triết lý ngược nhau

Sắp xếp trộn và quicksort đều là chia để trị, nhưng chúng đặt công việc ở hai chỗ ngược nhau — và so sánh đó dạy nhiều hơn bản thân hai thuật toán.

Sắp xếp trộn: chia dễ, ghép khó. Chia là cắt đôi ở giữa, không cần suy nghĩ. Toàn bộ công việc nằm ở bước trộn hai dãy đã sắp, tốn $O(n)$. Kết quả: $O(n \log n)$ *đảm bảo* cho mọi dữ liệu, ổn định, và trộn là thao tác quét tuần tự nên rất hợp cho dữ liệu lớn hơn bộ nhớ (D/24). Cái giá: cần $O(n)$ bộ nhớ phụ.

Quicksort: chia khó, ghép miễn phí. Toàn bộ công việc nằm ở bước phân hoạch — sắp xếp lại mảng sao cho mọi phần tử nhỏ hơn chốt nằm bên trái. Sau khi hai nửa được sắp xong thì không cần ghép gì cả, vì chúng đã ở đúng vị trí. Kết quả: không cần bộ nhớ phụ, truy cập tuần tự nên rất hợp cache, và trong thực tế thường nhanh hơn sắp xếp trộn vài lần. Cái giá: nếu chốt chọn tệ thì hai phần lệch hẳn và chi phí thành $O(n^2)$.



Hình 2.1. Hai cách phân bổ công việc trong chia để trị. Bài học tổng quát: công việc phải nằm ở đâu đó — hoặc lúc chia, hoặc lúc ghép. Khi thiết kế thuật toán chia để trị cho bài mới, hãy thử cả hai hướng: đôi khi chuyển công việc từ ghép sang chia làm mọi thứ đơn giản hẳn.

Chốt ngẫu nhiên: lần thứ tư của cùng một ý tưởng

Trường hợp xấu của quicksort không hiếm: mảng đã sắp sẵn cộng với việc chọn chốt là phần tử đầu tiên cho ngay $O(n^2)$. Và như ở chương B/08 và B/09, cái nguy hiểm là dữ liệu thực tế *rất hay* đã sắp sẵn một phần.

Cách chữa lại là ý tưởng cũ: *chọn chốt ngẫu nhiên*. Khi đó không có dữ liệu vào nào là “xấu”, vì hành vi của thuật toán không còn phụ thuộc thứ tự dữ liệu mà phụ thuộc các lựa chọn ngẫu nhiên của chính nó. Kỳ vọng chi phí là $\Theta(n \log n)$ với *mọi* dữ liệu vào, và xác suất tệ hơn nhiều lần giảm theo hàm mũ.

Đây là lần thứ tư nguyên lý này xuất hiện — bầm phổ dụng, treap, cắt tia qua lấy mẫu, và bây giờ là quicksort. Nếu bạn thấy nó lặp lại thì đó là dấu hiệu nên đọc kỹ chương C/20, nơi nó được phát biểu thành nguyên lý chung.

Chọn phần tử thứ k là biến thể quan trọng của quicksort: sau khi phân hoạch, ta biết chốt đứng ở vị trí nào, nên chỉ cần đệ quy *một* nửa. Chi phí kỳ vọng trở thành $\Theta(n)$ chứ không phải $\Theta(n \log n)$, vì $n + n/2 + n/4 + \dots < 2n$ — lại là chuỗi hình học của chương A/05. Có cả phiên bản tất định đảm bảo $O(n)$ trong trường hợp xấu nhất, dựa trên ý tưởng “trung vị của các trung vị” [2], nhưng hằng số lớn nên trong thực tế bản ngẫu nhiên thường thắng.

2.3 Khi bước ghép đòi hỏi một ý tưởng: cặp điểm gần nhất

Cho n điểm trên mặt phẳng, tìm hai điểm gần nhau nhất. Vết cặn $O(n^2)$.

Chia để trị: sắp các điểm theo hoành độ, chia làm hai nửa bằng một đường thẳng đứng, giải đệ quy mỗi nửa, được khoảng cách nhỏ nhất d trong hai nửa. Bước ghép phải xét các cặp *vắt qua* đường chia — và nếu làm ngây thơ thì đó vẫn là $O(n^2)$.

Ý tưởng cứu bài toán: chỉ những điểm nằm trong dải rộng $2d$ quanh đường chia mới có cơ hội, và trong dải đó, nếu ta sắp theo tung độ thì *mỗi điểm chỉ cần so với một số lượng điểm bị chặn bởi hằng số ở phía sau nó*. Lý do hình học: trong một hình chữ nhật $2d \times d$ không thể có quá một số hằng định điểm mà đôi một cách nhau ít nhất d . Vậy bước ghép là $O(n)$, và tổng chi phí là $O(n \log n)$.

Đây là ví dụ mẫu mực cho luận điểm mở đầu: bước chia tầm thường, còn cái làm nên thuật toán là một quan sát hình học cho phép bước ghép rẻ đi. Khi bạn thiết kế một thuật toán chia để trị mới, đây chính là chỗ phải tìm.

2.4 Nhân nhanh: giảm số bài con thay vì giảm chi phí ghép

Nhân hai số n chữ số theo cách trường học tốn $O(n^2)$. Chia để trị ngây thơ: tách mỗi số thành hai nửa, và tích trở thành bốn phép nhân nửa kích thước cộng vài phép cộng. Truy hồi $T(n) = 4T(n/2) + O(n)$ cho... vẫn $O(n^2)$. Không tiết kiệm gì.

Karatsuba nhận ra rằng *bốn* phép nhân là thừa: ba là đủ, vì tích chéo tính được từ ba tích kia bằng phép cộng trừ. Truy hồi thành $T(n) = 3T(n/2) + O(n)$, cho $O(n^{\log_2 3}) = O(n^{1,585})$.

Ngây thơ a_1b_1 a_1b_2 a_2b_1 a_2b_2 $4T(n/2) \Rightarrow \Theta(n^2)$

Karatsuba a_1b_1 a_2b_2 $(a_1+a_2)(b_1+b_2)$ $3T(n/2) \Rightarrow \Theta(n^{1,585})$

Tích chéo $a_1b_2 + a_2b_1$ suy được từ ba tích kia bằng cộng trừ, nên phép nhân thứ tư là thừa.

Hình 2.2. Hai chiến lược tăng tốc trong chia để trị. Bài cặp điểm gần nhất thắng bằng cách làm *bước ghép* rẻ đi; Karatsuba thắng bằng cách *giảm số bài con*. Vì số bài con nằm ở số mũ của lời giải còn chi phí ghép chỉ cộng thêm, chiến lược thứ hai đổi hẳn bậc.

Điểm đáng học ở đây khác hẳn với mục trước, và đó là lý do tôi đặt hai ví dụ cạnh nhau: ở bài cặp điểm gần nhất, ta thắng bằng cách làm *bước ghép* rẻ đi; ở đây, ta thắng bằng cách *giảm số bài con*. Nhìn vào hệ thức truy hồi thì thấy rõ: một bên giảm $f(n)$, một bên giảm a . Và vì a nằm ở số mũ còn $f(n)$ chỉ là cộng thêm, việc giảm a có tác động sâu hơn — nó đổi *bậc* của lời giải.

Cùng ý tưởng cho phép nhân ma trận: Strassen[3] giảm từ 8 xuống 7 phép nhân ma trận con, cho $O(n^{\log_2 7}) \approx O(n^{2,81})$. Và cùng ý tưởng, đẩy tới cùng, cho biến đổi Fourier nhanh với $O(n \log n)$ — công cụ ở chương D/21.

CẠM BÃY

Các thuật toán nhân nhanh có ngưỡng hoà vốn thật sự. Karatsuba chỉ thắng cách trường học khi số đủ lớn — thường vài trăm tới vài nghìn chữ số, tùy cài đặt — vì nó tốn thêm nhiều phép cộng và nhiều lời gọi đệ quy. Strassen còn tệ hơn về mặt này: hằng số lớn, kém ổn định số học, và không thân thiện cache, nên các thư viện đại số tuyến tính chỉ dùng nó ở kích thước rất lớn nếu có dùng. Đây là ví dụ tiêu biểu cho lời cảnh báo ở chương A/03: bậc tiệm cận nhỏ hơn không tự động nghĩa là nhanh hơn.

2.5 Chia để trị ở những nơi không ngờ

Ba biến thể đáng biết vì chúng mở rộng phạm vi áp dụng.

Đếm trong lúc trộn. Bài đếm số nghịch thế giải được bằng cách sửa sắp xếp trộn: khi trộn, mỗi lần lấy một phần tử từ nửa phải trước một phần tử của nửa trái thì ta vừa phát hiện một loạt nghịch thế và đếm được ngay. Mẫu tổng quát: *nếu bài toán đếm các cặp có quan hệ nào đó, hãy thử đếm chúng trong lúc trộn*. Cùng mẫu giải bài đếm cặp có tổng nhỏ hơn ngưỡng, hay đếm cặp đảo ngược có trọng số.

Chia để trị trên trục thời gian. Nếu các đối tượng chỉ “tồn tại” trong một khoảng thời gian, hãy xây một cây phân đoạn trên trục thời gian, đặt mỗi đối tượng vào $O(\log n)$ nút, rồi duyệt sâu cây đó với một cấu trúc có hoàn tác. Đây chính là kỹ thuật đã nhắc ở B/11 cho bài liên thông động ngoại tuyến.

Tối ưu hoá chia để trị cho quy hoạch động. Khi điểm chuyển tối ưu có tính đơn điệu, ta tính các trạng thái theo kiểu chia để trị và giảm được một thừa số n . Chi tiết ở chương C/16; điều đáng nhớ ở đây là chia để trị không chỉ là cách tổ chức thuật toán mà còn là cách tổ chức việc tính toán bên trong một thuật toán khác.

2.6 Biên giới hiện nay: $O(n \log n)$ cho nhân số, và số mũ của nhân ma trận

Câu chuyện Karatsuba không dừng ở năm 1960, và phần tiếp theo của nó dạy một điều mà chương A/03 mới chỉ nói bằng lời cảnh báo: một cận có thể vừa đúng vừa vô dụng.

Sau Karatsuba, Toom và Cook tổng quát hoá mẹo ấy thành “chia thành k mảnh, dùng $2k - 1$ phép nhân”, đẩy số mũ xuống gần 1 nhưng không bao giờ tới. Bước nhảy thật sự đến khi người ta bỏ hẳn cách nhìn chia để trị thô và chuyển sang miền tần số: nhân hai đa thức là tích chập, mà tích chập trở thành phép nhân từng điểm sau biến đổi Fourier (D/21). Schönhage và Strassen dùng đúng ý đó năm 1971 để đạt $O(n \log n \log \log n)$, và cùng lúc phỏng đoán rằng đáp án cuối cùng phải là $O(n \log n)$.

Phỏng đoán ấy được chứng minh năm 2021 bởi Harvey và van der Hoeven[4], đăng trên *Annals of Mathematics* — một tạp chí toán học chứ không phải kỷ yếu tin học, điều đã nói lên tính chất của kết quả. Nhưng đây mới là phần đáng học: thuật toán của họ chỉ thắng Schönhage–Strassen từ những kích thước lớn hơn mọi thứ có thể xuất hiện trong máy tính thật. Nó là một thuật toán thiên hà: đúng, đẹp, giải quyết một câu hỏi mở nửa thế kỷ, và không bao giờ được cài vào thư viện nào. Giá trị của nó là chốt lại hình dạng của câu trả lời, chứ không phải làm chương trình của ai đó nhanh lên. Và chú ý sự bất đối xứng quen thuộc: chưa ai chứng minh được cận dưới $\Omega(n \log n)$ tương ứng, nên khả năng còn nhanh hơn nữa vẫn chưa bị loại trừ (D/24).

Nhân ma trận có cùng cấu trúc câu chuyện nhưng chưa có hồi kết. Strassen[3] mở màn với $O(n^{2.81})$; nửa thế kỷ tiếp theo là một chuỗi cải tiến bằng những kỹ thuật đại số ngày càng phức tạp, tới $\omega < 2.3729$ của Alman và Vassilevska Williams[5], và các bản tiền ấn tới năm 2026 hạ tiếp xuống quanh 2.3712[6] — những con số này chưa qua phản biện nên hãy đọc chúng như tin tức, không như kiến thức đã ổn định. Cận dưới hiển nhiên là $\omega \geq 2$, vì chỉ riêng việc đọc dữ liệu vào đã tốn n^2 ; không ai biết $\omega = 2$ hay không, và đó là một trong những câu hỏi mở lớn nhất của đại số tính toán.

Phần thực hành thì gọn và ngược hẳn với vẻ hào nhoáng ở trên: mọi thuật toán sau Strassen đều thiên hà, và ngay cả Strassen cũng chỉ được dùng ở kích thước lớn vì hằng số, độ ổn định số học và tính thân thiện cache đều kém hơn thuật toán ba vòng lặp đã tối ưu theo khối. Đây là lời nhắc cụ thể nhất mà tôi biết cho luận điểm của chương A/03: bậc tiệm cận là một trong các thước đo, không phải thước đo duy nhất, và giữa “tồn tại thuật toán với số mũ nhỏ hơn” và “chương trình của tôi chạy nhanh hơn” là cả một khoảng cách có thể đo bằng thập kỷ.

2.7 Gốc rễ: một mẹo đại số phá vỡ một phỏng đoán

Sắp xếp trộn xuất hiện rất sớm — nó được ghi nhận trong một bản mô tả của von Neumann từ năm 1945, thuộc những thuật toán đầu tiên được viết cho máy tính có chương trình lưu trữ. Điều đó không ngẫu nhiên: máy tính thời đó dùng băng từ, và trộn hai dãy đã sắp là thao tác gần như duy nhất mà băng từ làm tốt — đọc tuần tự, không nhảy. Nói cách khác, thuật toán sinh ra từ ràng buộc của phần cứng, đúng như câu chuyện B-cây ở chương B/09.

Quicksort có một nguồn gốc rất cụ thể và rất đời. Tony Hoare nghĩ ra nó khoảng năm 1960 khi đang làm việc về dịch máy Nga–Anh: ông cần sắp xếp các từ của một câu để tra chúng trong

từ điển lưu trên băng từ một cách hiệu quả. Ràng buộc bộ nhớ thời đó rất chặt, và đó là lý do quicksort được thiết kế để sắp xếp *tại chỗ* — một quyết định tình cờ hoá ra lại là lý do nó vẫn thắng trên phần cứng hiện đại, nơi việc tránh cấp phát và truy cập tuần tự quan trọng hơn bao giờ hết.

Câu chuyện đáng nhớ nhất trong chương này thuộc về Karatsuba, và nó là một bài học về giá trị của việc nghi ngờ. Năm 1960, Kolmogorov — một trong những nhà toán học lớn nhất thế kỷ — phát biểu tại một buổi seminar phỏng đoán rằng nhân hai số n chữ số bất buộc cần $\Omega(n^2)$ phép toán. Karatsuba, khi đó là một sinh viên, tìm ra thuật toán ba phép nhân chỉ trong khoảng một tuần, bác bỏ phỏng đoán. Điều đáng học không phải mẹo đại số mà là bối cảnh: một phát biểu “chắc chắn phải cần n^2 ” được đưa ra bởi người có uy quyền lớn nhất, và nó sai — vì trực giác về cận dưới rất khó tin cậy khi chưa có chứng minh. Chương D/24 sẽ nói vì sao chứng minh cận dưới lại khó tới vậy.

Cuối cùng, biến đổi Fourier nhanh có một gốc rễ gây bất ngờ: thuật toán mà Cooley và Tukey công bố năm 1965[7] về sau được phát hiện là đã có mặt trong các ghi chép chưa xuất bản của Gauss từ năm 1805, viết bằng tiếng Latin và bị bỏ quên. Bối cảnh thúc đẩy việc tái phát minh năm 1965 rất cụ thể: nhu cầu phát hiện các vụ thử hạt nhân qua tín hiệu địa chấn, một bài toán xử lý tín hiệu quy mô lớn.

■ Gốc rễ: ý tưởng này ra đời từ bài toán nào

Mốc	Bài toán thúc ép lúc đó	Điều thay đổi sau đó
1805 — Gauss (không xuất bản)	Nội suy quỹ đạo thiên thể từ các quan sát rời rạc.	Ý tưởng của biến đổi Fourier nhanh, bị bỏ quên trong ghi chép suốt 160 năm.
1945 — von Neumann	Sắp xếp dữ liệu trên băng từ, nơi chỉ đọc tuần tự được.	Sắp xếp trộn — một trong những thuật toán đầu tiên viết cho máy có chương trình lưu trữ; ràng buộc phần cứng sinh ra thuật toán.
1960 — Hoare	Sắp xếp các từ của câu để tra từ điển trên băng, với bộ nhớ rất hạn chế.	Quicksort: sắp tại chỗ. Ràng buộc bộ nhớ thời đó tình cờ tạo ra ưu thế cache của ngày nay.
1960 — Karatsuba	Kolmogorov phỏng đoán nhân số cần $\Omega(n^2)$.	Ba phép nhân thay vì bốn, cho $O(n^{1,585})$; phỏng đoán bị bác bỏ trong một tuần. Bài học: trực giác về cận dưới rất dễ sai.
1965 — Cooley & Tukey	Xử lý tín hiệu địa chấn để phát hiện thử hạt nhân.	FFT $O(n \log n)$; sau đó phát hiện ra Gauss đã có ý tưởng từ 1805.
1969 — Strassen	Nhân ma trận có bất buộc $\Omega(n^3)$ không?	Bảy phép nhân thay vì tám, cho $O(n^{2,81})$; mở ra cả một nhánh nghiên cứu về số mũ nhân ma trận vẫn đang tiếp diễn.

■ 2.8 Liên hệ ngang

■ Liên hệ ngang: ý này còn sống ở đâu nữa

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu
Tính toán song song	MapReduce, phép rút gọn theo cây	Chia để trị là cấu trúc song song lý tưởng vì các bài con độc lập nên chạy đồng thời được. Điều kiện cũng giống hệt: phép ghép phải kết hợp — cùng điều kiện với cây phân đoạn ở B/10.

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu (tiếp)
Xử lý ảnh	Kim tự tháp ảnh, biến đổi wavelet, phương pháp đa lưới	Cùng cấu trúc đệ quy chia đôi độ phân giải. Bài học chung: nhiều bài toán dễ hơn hẳn khi giải ở độ phân giải thô rồi tinh chỉnh dần — một chiến lược mà chia để trị thuần túy không có nhưng rất mạnh trong thực tế.
Đại số tuyến tính số	Phân rã khối, nhân ma trận theo khối	Chia theo khối không phải để giảm bậc mà để <i>hợp cache</i> — mỗi khối vừa trong bộ nhớ nhanh. Đây là chia để trị phục vụ mô hình chi phí bộ nhớ chứ không phục vụ hệ thức truy hồi ($D/24$).
Xử lý tín hiệu	FFT, tích chập nhanh	Ứng dụng nổi tiếng nhất của chia để trị. Ý tưởng chuyển miền — việc khó ở miền này thành việc dễ ở miền kia — cũng chính là mẫu “tổng tiền tố và mảng hiệu” ở B/06, phóng to lên.
Ước lượng trạng thái, SLAM	Chia bản đồ thành các bản đồ con rồi ghép	Cùng khuôn: giải cục bộ rồi ghép. Điểm khác quyết định là bước ghép ở đây <i>không</i> chính xác — các bản đồ con có sai số tương quan — nên việc ghép ngay thơ tạo ra sai lệch hệ thống. Đây là ví dụ cho câu hỏi thứ hai ở mục 1: các bài con có thật sự độc lập không.
Quản lý dự án	Chia nhỏ công việc	Cùng trực giác, nhưng đáng nêu vì lý do thất bại giống nhau: nếu các phần việc phụ thuộc lẫn nhau thì chi phí “ghép” (phối hợp) tăng nhanh hơn lợi ích của việc chia — đúng trường hợp $f(n)$ lớn trong hệ thức truy hồi.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Từ việc phân lớn công việc trong lời giải vét cạn là so sánh những cặp *vắt qua* ranh giới, và chia để trị làm cho số cặp như vậy nhỏ đi rất nhiều. Nói bằng hệ thức truy hồi: nếu chi phí ghép $f(n)$ nhỏ hơn hẳn $n^{\log_b a}$ thì tổng bị chi phối bởi lá, còn nếu bằng nhau thì ta chỉ trả thêm một thừa số $\log n$. So với $O(n^2)$ thì cả hai kết cục đều tốt hơn nhiều bậc. Nếu bước ghép vẫn tốn $O(n^2)$ thì chia để trị không tiết kiệm gì — đó là lý do bước ghép mới là phần khó.
- A2. Vì bậc tiệm cận không nói hết. Quicksort sắp tại chỗ nên không có chi phí cấp phát và không chép dữ liệu qua lại giữa hai mảng; phân hoạch là một lượt quét gần như tuần tự nên rất hợp cache; và vòng lặp trong của nó rất ngắn. Sắp xếp trộn phải ghi ra mảng phụ rồi chép về, tức là chạm mỗi phần tử nhiều lần hơn. Trường hợp xấu $O(n^2)$ thì được xử lý bằng chốt ngẫu nhiên, hoặc bằng cách chuyển sang heapsort khi độ sâu vượt ngưỡng[8].
- A3. Vì một quan sát hình học: chỉ những điểm nằm trong dải rộng $2d$ quanh đường chia mới có thể tạo cặp gần hơn d , và trong dải đó, khi các điểm được sắp theo tung độ thì mỗi điểm chỉ cần so với một số hằng định điểm kế tiếp. Lý do là trong một hình chữ nhật $2d \times d$ không thể nhét quá một số hằng định điểm đôi một cách nhau ít nhất d . Nhờ vậy bước ghép là $O(n)$ thay vì $O(n^2)$.
- A4. Nó giảm số *bài con* a từ 4 xuống 3, chứ không giảm chi phí ghép $f(n)$. Điều này quan trọng vì a nằm ở số mũ của lời giải — $T = \Theta(n^{\log_b a})$ — nên giảm a làm đổi *bậc*, trong khi giảm $f(n)$ chỉ ảnh hưởng khi f đang là thành phần chi phối. Đây là hai chiến lược khác nhau của chia để trị, và bài cặp điểm gần nhất minh họa chiến lược còn lại.
- A5. Ba tình huống. Khi các bài con *không độc lập* — lời giải nửa này ảnh hưởng nửa kia — thì phải chuyển sang quy hoạch động. Khi bước ghép đặt ngang bài gốc, ví dụ vẫn phải xét mọi cặp *vắt ngang*. Và khi n nhỏ, vì chi phí gọi đệ quy và các hằng số phụ vượt quá lợi ích — đó là lý do mọi thư viện sắp xếp thật đều chuyển sang sắp xếp chèn khi đoạn còn dưới vài chục phần tử[9].

NÓI LẠI THẬT ĐƠN GIẢN

Chia để trị là cách bạn xử lý một chồng bài kiểm tra cần chấm: thay vì tự chấm hết, bạn chia đôi chồng bài cho hai người, mỗi người lại chia đôi cho hai người nữa, tới khi mỗi người chỉ còn vài bài. Việc chấm xong rất nhanh. Nhưng có một khâu dễ bị quên: sau khi chấm xong phải *gộp kết quả lại*, và nếu khâu gộp cũng mệt như chấm thì chia ra chẳng lợi gì.

Đó là ý chính của cả chương: chia thì dễ, gộp mới khó, và mọi thuật toán hay ở đây đều là kết quả của việc tìm ra một cách gộp rẻ bất ngờ.

Hai cách sắp xếp kinh điển cho thấy hai kiểu phân bổ công việc. Sắp xếp trộn giống việc bạn cứ chia bừa đôi chồng bài, rồi khi gộp thì phải so từng cặp để trộn hai chồng đã sắp thành một — công việc dồn vào lúc gộp. Quicksort thì ngược: bạn chọn một bài làm mốc và ngay từ đầu chia thành “kém hơn mốc” và “hơn mốc”; sau khi hai bên được sắp xong thì chỉ việc đặt cạnh nhau, không phải gộp gì cả. Công việc dồn vào lúc chia.

Còn mẹo của Karatsuba thì thuộc loại khác hẳn và rất đáng nhớ. Khi nhân hai số lớn, cách chia tự nhiên đòi bốn phép nhân nhỏ. Ông nhận ra chỉ cần ba, vì cái thứ tư suy được từ ba cái kia bằng cộng trừ. Nghe như một mẹo vặt, nhưng vì nó lặp lại ở mọi tầng của đệ quy nên tiết kiệm nhân lên theo cấp số nhân. Đó cũng là bài học chung: trong chia để trị, giảm *số bài con* có tác động sâu hơn nhiều so với làm cho mỗi bài con nhẹ đi một chút.

Một câu: Chia thì dễ; cả nghệ thuật nằm ở chỗ ghép hai nửa lại mà không phải làm lại từ đầu.

Chỗ không rút gọn được: Không có quy trình chung để tìm ra một bước ghép rẻ; nó thường đến từ một quan sát riêng của bài toán, như quan sát hình học trong bài cặp điểm gần nhất.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
●●● Ba câu hỏi thiết kế	Chia ở đâu; các bài con có độc lập không; bước ghép có rẻ hơn bài gốc không. Câu thứ ba chứa gần hết độ khó.
●●● Đọc hệ thức truy hồi	$T(n) = aT(n/b) + f(n)$; so $f(n)$ với $n^{\log_b a}$ để biết tầng nào chi phối (Θ).
●●● Sắp xếp trộn so với quicksort	Trộn: chia tầm thường, ghép $O(n)$, đảm bảo $O(n \log n)$, ổn định, cần bộ nhớ phụ. Quicksort: công ở bước phân hoạch, ghép miễn phí, tại chỗ, hợp cache, xấu nhất $O(n^2)$.
●●● Chốt ngẫu nhiên	Làm hành vi phụ thuộc lựa chọn của thuật toán thay vì thứ tự dữ liệu; kỳ vọng $\Theta(n \log n)$ với mọi dữ liệu vào.
●●● Chọn phần tử thứ k	Chỉ đệ quy một nửa sau khi phân hoạch; $n + n/2 + n/4 + \dots < 2n$ cho kỳ vọng $\Theta(n)$.
●●● Hai chiến lược tăng tốc	Giảm chi phí ghép $f(n)$ (cặp điểm gần nhất) hoặc giảm số bài con a (Karatsuba, Strassen). Giảm a đôi <i>bạc</i> vì a nằm ở số mũ.
●● Bước ghép của cặp điểm gần nhất	Chỉ xét dải rộng $2d$ quanh đường chia, sắp theo tung độ, mỗi điểm so với một số hằng định điểm kế tiếp $\Rightarrow O(n)$.
●● Karatsuba	Ba phép nhân thay vì bốn: $T(n) = 3T(n/2) + O(n)$, cho $O(n^{\log_2 3})$. Có ngưỡng hoà vốn thật, chỉ thắng khi số đủ lớn.
●● Đếm trong lúc trộn	Đếm nghịch thế bằng sắp xếp trộn; mẫu tổng quát cho các bài “đếm cặp có quan hệ”.
●● Chia để trị trên trục thời gian	Cây phân đoạn trên trục gian + cấu trúc có hoàn tác; giải bài liên thông động ngoại tuyến ($B/11$).
●● Khi nào chia để trị sai	Bài con phụ thuộc nhau ($\rightarrow$ quy hoạch động); bước ghép đắt ngang bài gốc; hoặc n nhỏ nên chi phí đệ quy vượt lợi ích.
● Bài học Karatsuba–Kolmogorov	Một phỏng đoán cận dưới của nhà toán học lớn bị bác bỏ trong một tuần — trực giác về cận dưới rất dễ sai ($D/24$).
● Biên giới của nhân nhanh	Nhân số nguyên đã có $O(n \log n)$ (2021) nhưng là thuật toán thiên hà; nhân ma trận dừng ở $\omega < 2,3729$ và câu hỏi $\omega = 2$ vẫn mở. Thực hành chỉ dùng tới Karatsuba và Strassen.

Câu hỏi ôn tập có đáp án (che phần đáp án, tự trả lời trước)

1. Vì sao các thư viện sắp xếp thật không dùng thuần một thuật toán nào?

| Vì mỗi thuật toán mạnh ở một vùng khác nhau. Cách làm chuẩn — introsort — là bắt đầu bằng quicksort vì nó nhanh nhất trong trường hợp thường; chuyển sang sắp xếp chèn khi đoạn còn nhỏ vì lúc đó hằng số quan trọng hơn bậc; và chuyển sang heapsort khi độ sâu đệ quy vượt ngưỡng, để chặn trường hợp xấu $O(n^2)$ [8]. Ngoài ra còn phải xử lý dữ liệu có nhiều phần tử bằng nhau bằng phân hoạch ba phần[9]. Đây là một trong những ví dụ rõ nhất cho chương E/26.

2. Sắp xếp ổn định có quan trọng không, hay chỉ là chi tiết lý thuyết?

| Rất quan trọng trong thực tế, vì nó cho phép sắp nhiều lần theo nhiều khoá: sắp theo khoá phụ trước, rồi theo khoá chính, và các phần tử bằng khoá chính vẫn giữ đúng thứ tự theo khoá phụ. Đây là cách các bảng dữ liệu và giao diện người dùng làm việc. Sắp xếp trộn ổn định tự nhiên; quicksort thì không, và làm nó ổn định đòi bộ nhớ phụ — một lý do nữa vì sao cả hai cùng tồn tại.

3. Có phải mọi thuật toán chia để trị đều song song hoá được không?

| Về nguyên tắc thì các bài con độc lập nên chạy song song được, nhưng có ba điều kiện thực tế. Bước ghép phải kết hợp và rẻ, nếu không nó thành nút cổ chai. Kích thước bài con phải đủ lớn để bù chi phí tạo luồng — nên trong thực tế người ta chỉ song song hoá vài mức trên cùng rồi chuyển sang tuần tự. Và dữ liệu phải chia được mà không tranh chấp bộ nhớ. Sắp xếp trộn song song hoá tốt hơn quicksort vì bước phân hoạch của quicksort khó song song.

4. Tôi có nên tự cài quicksort trong thi đấu không?

| Không, trừ khi bài yêu cầu một biến thể đặc biệt như chọn phần tử thứ k hoặc phân hoạch ba phần. Thư viện chuẩn đã được kỹ thuật hoá kỹ hơn nhiều so với những gì bạn viết trong năm phút. Điều cần biết là *cạm bẫy*: trong một số ngôn ngữ, hàm sắp xếp chuẩn có trường hợp xấu bị khai thác được, nên khi nghi ngờ hãy trộn ngẫu nhiên mảng trước khi sắp — một dòng code, và nó áp dụng đúng nguyên lý ngẫu nhiên hoá của chương này.

5. Khi nào nên dùng đếm trong lúc trộn thay vì Fenwick cho bài đếm nghịch thế?

| Cả hai đều $O(n \log n)$. Fenwick ngắn hơn nếu bạn đã có sẵn trong thư viện, nhưng cần nén toạ độ. Đếm trong lúc trộn không cần nén và tổng quát hoá tự nhiên sang các quan hệ phức tạp hơn — ví dụ đếm cặp thoả $a_i > 2a_j$, thứ mà Fenwick phải xử lý khéo hơn. Lời khuyên: biết cả hai, và chọn theo việc quan hệ cần đếm có dễ biểu diễn thành “đếm phần tử đã chèn nhỏ hơn x ” hay không.

6. Chia để trị và quy nạp liên quan với nhau thế nào?

| Chúng là hai mặt của một đồng xu, và nhận ra điều đó làm việc viết code lẫn viết chứng minh dễ hơn hẳn. Thuật toán chia để trị giải bài lớn bằng bài nhỏ; chứng minh quy nạp khẳng định tính đúng cho n dựa vào tính đúng cho các giá trị nhỏ hơn. Chúng khớp nhau từng dòng: trường hợp cơ sở của đệ quy ứng với cơ sở quy nạp, và bước ghép ứng với bước quy nạp. Vì vậy nếu bạn viết được thuật toán mà không viết được chứng minh, thường là do bước ghép có lỗ hổng — xem A/04.

7. Ngưỡng chuyển sang thuật toán đơn giản nên đặt bao nhiêu?

| Không có con số phổ quát — nó phụ thuộc ngôn ngữ, kích thước phần tử và máy. Cách đúng là đo: cài cả hai, chạy trên dữ liệu thật với nhiều kích thước, và tìm điểm giao. Trong thực tế các thư viện thường dùng ngưỡng trong khoảng vài chục phần tử, nhưng đừng chép con số mà hãy chép *phương pháp*: đây là một quyết định thực nghiệm, không phải quyết định lý thuyết (E/26).

Bài tập (mức: ● khởi động ●● chuẩn ●●● khó)

Bài	Nguồn	Điều cần nhận ra
● Cài sắp xếp trộn và kiểm chứng bằng stress test	Bài tập cài đặt	Viết bất biến của bước trộn ra trước; kiểm với mảng rỗng và mảng toàn phần tử bằng nhau.

Bài	Nguồn	Điều cần nhận ra (tiếp)
• Tìm phần tử thứ k bằng phân hoạch ngẫu nhiên	Bài kinh điển	Chỉ đề quy một nửa; giải thích vì sao kỳ vọng là $\Theta(n)$ chứ không $\Theta(n \log n)$.
•• Đếm số nghịch thế bằng sắp xếp trộn	Bài kinh điển	So với cách Fenwick ở B/10; nêu điều kiện chọn mỗi cách.
•• Lũy thừa nhanh và lũy thừa ma trận	CSES “Exponentiation”[1]	Chia để trị trên số mũ; nối với A/05 và D/21.
•• Tìm cặp điểm gần nhất trong mặt phẳng	CSES “Minimum Euclidean Distance”[1]	Bước ghép là toàn bộ bài; hãy tự chứng minh cận hằng số cho số điểm phải so.
•• Tìm phần tử đa số bằng chia để trị	LeetCode “Majority Element”	So với thuật toán bỏ phiếu $O(n)$ ở A/01 — ví dụ cho thấy chia để trị không phải luôn là cách tốt nhất.
••• Cài Karatsuba cho số lớn và tìm ngưỡng hoà vốn trên máy bạn	Bài tập thực nghiệm	Vẽ đường thời gian theo số chữ số cho cả hai cách; điểm giao là câu trả lời.
••• Bài toán “skyline” — gộp đường viền các toà nhà	LeetCode “The Skyline Problem”	Chia để trị với bước ghép giống trộn; bài mẫu cho việc thiết kế phép ghép.
••• Đếm số cặp (i, j) với $i < j$ và $a_i > 2a_j$	Bài kinh điển	Biến thể của đếm trong lúc trộn; chỗ dễ sai là phải đếm trước khi trộn.
••• Nhân đa thức bằng chia để trị, rồi bằng FFT	Chương D/21	So $O(n^{1,585})$ với $O(n \log n)$ bằng số liệu thật; ngưỡng hoà vốn lại là câu hỏi thực nghiệm.
••• Chia để trị trên trục thời gian cho bài liên thông động ngoại tuyến	Bài thi đấu kinh điển	Kết hợp B/10 và B/11; đây là bài tập tổng hợp tốt nhất của phần B và C.

Kết chương và đọc tiếp

Chia để trị là kỹ thuật đầu tiên trong phần C khai thác cấu trúc *đệ quy* của bài toán. Ba điều đáng mang theo: phần khó nằm ở bước ghép; có hai chiến lược tăng tốc rất khác nhau — làm ghép rẻ đi hay giảm số bài con — và chiến lược thứ hai đôi bậc; và điều kiện tiên quyết luôn là các bài con phải *độc lập*.

Chính điều kiện cuối dẫn tới hai chương tiếp theo. Khi các lựa chọn không độc lập mà nối tiếp nhau, có hai kịch bản. Kịch bản may mắn: tồn tại một lựa chọn cục bộ luôn an toàn, và ta có thuật toán tham lam — ngăn tới bất ngờ nhưng cần chứng minh thật sự (C/15). Kịch bản còn lại: không lựa chọn cục bộ nào an toàn, và ta phải xét mọi khả năng nhưng nhớ lại kết quả đã tính — quy hoạch động (C/16).

Đọc thêm: công trình gốc (chỉ liệt kê nguồn đã qua phản biện; bản tiền ấn được ghi rõ)

Mức	Công trình và tác giả	Nơi công bố, năm	Vì sao đọc, đọc phần nào
nền tảng	Harvey & van der Hoeven, “Integer Multiplication in Time $O(n \log n)$ ”	Annals of Mathematics, 2021	Đọc phần mở đầu: nó kể lại toàn bộ lịch sử cận trên của phép nhân và nói rõ vì sao $n \log n$ là ngưỡng được kỳ vọng.
nền tảng	Strassen, “Gaussian Elimination Is Not Optimal”	Numerische Mathematik, 1969	Ba trang, đọc trong hai mươi phút, và là ví dụ hoàn hảo cho chiến lược “giảm số bài con”.
tra khi cần	Alman & Vassilevska Williams, “A Refined Laser Method and Faster Matrix Multiplication”	SODA, 2021	Tra khi muốn biết các cận ω hiện đại được lấy ra bằng kỹ thuật gì; phần mở đầu đủ cho người đọc không chuyên.

Mức	Công trình và tác giả	Nơi công bố, năm	Vì sao đọc (tiếp)
tra khi cần	Alman, Duan, Vassilevska Williams, Xu, Xu, Zhou, "Improving the Matrix Multiplication Exponent..."	bản tiền ấn arXiv, 2026 — <i>chưa qua phản biện</i>	Chỉ đọc để biết con số mới nhất; không trích dẫn như kết quả đã ổn định.
tra khi cần	Cooley & Tukey, "An Algorithm for the Machine Calculation of Complex Fourier Series"	Mathematics of Computation, 1965	Bài gốc của FFT — công cụ đứng sau mọi thuật toán nhân nhanh hiện đại (D/21).

Nguồn

- [1] A. Laaksonen. “CSES Problem Set.” (2026), address: <https://cses.fi/problemset/> (visited on 09/06/2026).
- [2] M. Blum, R. W. Floyd, V. Pratt, et al., “Time Bounds for Selection,” *Journal of Computer and System Sciences*, vol. 7, no. 4, pp. 448–461, 1973.
- [3] V. Strassen, “Gaussian Elimination Is Not Optimal,” *Numerische Mathematik*, vol. 13, pp. 354–356, 1969.
- [4] D. Harvey and J. van der Hoeven, “Integer Multiplication in Time $O(n \log n)$,” *Annals of Mathematics*, vol. 193, no. 2, pp. 563–617, 2021.
- [5] J. Alman and V. Vassilevska Williams, “A Refined Laser Method and Faster Matrix Multiplication,” in *Proceedings of the 32nd ACM-SIAM Symposium on Discrete Algorithms (SODA)*, 2021.
- [6] J. Alman, R. Duan, V. Vassilevska Williams, et al. “Improving the Matrix Multiplication Exponent with Modern Optimization and AlphaEvolve.” bản tiền ấn arXiv:2608.16884, chưa qua phản biện. (2026), address: <https://arxiv.org/abs/2608.16884> (visited on 09/07/2026).
- [7] J. W. Cooley and J. W. Tukey, “An Algorithm for the Machine Calculation of Complex Fourier Series,” *Mathematics of Computation*, vol. 19, no. 90, pp. 297–301, 1965.
- [8] D. R. Musser, “Introspective Sorting and Selection Algorithms,” *Software: Practice and Experience*, vol. 27, no. 8, pp. 983–993, 1997.
- [9] J. L. Bentley and M. D. McIlroy, “Engineering a Sort Function,” *Software: Practice and Experience*, vol. 23, no. 11, pp. 1249–1265, 1993.
- [10] D. E. Knuth, “Optimum Binary Search Trees,” *Acta Informatica*, vol. 1, pp. 14–25, 1971.
- [11] F. F. Yao, “Speed-Up in Dynamic Programming,” *SIAM Journal on Algebraic and Discrete Methods*, vol. 3, no. 4, pp. 532–540, 1982.
- [12] A. Aggarwal, M. M. Klawe, S. Moran, et al., “Geometric Applications of a Matrix-Searching Algorithm,” *Algorithmica*, vol. 2, pp. 195–208, 1987.
- [13] D. S. Hirschberg, “A Linear Space Algorithm for Computing Maximal Common Subsequences,” *Communications of the ACM*, vol. 18, no. 6, pp. 341–343, 1975.
- [14] R. E. Tarjan, “Depth-First Search and Linear Graph Algorithms,” *SIAM Journal on Computing*, vol. 1, no. 2, pp. 146–160, 1972.
- [15] M. L. Fredman and R. E. Tarjan, “Fibonacci Heaps and Their Uses in Improved Network Optimization Algorithms,” *Journal of the ACM*, vol. 34, no. 3, pp. 596–615, 1987.
- [16] M. Thorup, “Undirected Single-Source Shortest Paths with Positive Integer Weights in Linear Time,” *Journal of the ACM*, vol. 46, no. 3, pp. 362–394, 1999.
- [17] R. Duan, J. Mao, X. Mao, et al., “Breaking the Sorting Barrier for Directed Single-Source Shortest Paths,” in *Proceedings of the 57th Annual ACM Symposium on Theory of Computing (STOC)*, Best Paper Award, 2025.
- [18] E. W. Dijkstra, “A Note on Two Problems in Connexion with Graphs,” *Numerische Mathematik*, vol. 1, pp. 269–271, 1959.

- [19] KTH Royal Institute of Technology. “KACTL: KTH Algorithm Competition Template Library.” (2026), address: <https://github.com/kth-competitive-programming/kactl> (visited on 09/06/2026).
- [20] H. W. Kuhn, “The Hungarian Method for the Assignment Problem,” *Naval Research Logistics Quarterly*, vol. 2, no. 1-2, pp. 83–97, 1955.
- [21] J. B. Orlin, “Max Flows in $O(nm)$ Time, or Better,” in *Proceedings of the 45th Annual ACM Symposium on Theory of Computing (STOC)*, 2013, pp. 765–774.
- [22] L. Chen, R. Kyng, Y. P. Liu, et al., “Maximum Flow and Minimum-Cost Flow in Almost-Linear Time,” in *Proceedings of the 63rd IEEE Symposium on Foundations of Computer Science (FOCS)*, Best Paper Award, 2022.
- [23] J. E. Hopcroft and R. M. Karp, “An $n^{5/2}$ Algorithm for Maximum Matchings in Bipartite Graphs,” *SIAM Journal on Computing*, vol. 2, no. 4, pp. 225–231, 1973.
- [24] J. Bentley, *Programming Pearls*, 2nd ed. Addison-Wesley, 2000.
- [25] R. Motwani and P. Raghavan, *Randomized Algorithms*. Cambridge University Press, 1995.

Tham lam (Greedy Algorithms)

PHỤ THUỘC A/04 cho lập luận đối chỗ — chương này không dùng được nếu thiếu nó; B/07 cho heap; B/11 cho DSU trong Kruskal.

TÓM TẮT

Tham lam là chiến lược có khoảng cách lớn nhất giữa độ ngắn của code và độ khó của chứng minh: bốn dòng chương trình, nhưng nếu không chứng minh thì bạn không biết mình đúng hay sai. Chương này tập trung vào phần chứng minh — hai khuôn chuẩn là lập luận đối chỗ và “tham lam luôn đi trước” — và vào câu hỏi sâu hơn: *cấu trúc nào của bài toán làm cho tham lam đúng?* Câu trả lời có tên riêng là matroid, và biết nó giúp bạn nhận ra ngay một lớp bài mà tham lam chắc chắn tối ưu. Chương cũng nói về những nơi tham lam sai nhưng vẫn hữu ích: xấp xỉ với bảo đảm chất lượng. Nếu chỉ nhớ một điều: tham lam không sai vì “nhìn ngắn”, nó sai vì lựa chọn hôm nay *phá huỷ* khả năng của tương lai — và nhiệm vụ của chứng minh là loại trừ đúng khả năng đó.

KIẾN THỨC CẦN CÓ TRƯỚC

- **thuật toán tham lam** — xây lời giải bằng cách lặp lại một lựa chọn tốt nhất theo tiêu chí cục bộ, không bao giờ xét lại.
- **tính chất lựa chọn tham lam** — tồn tại một lời giải tối ưu chứa lựa chọn mà tham lam đưa ra ở bước đầu. Đây là điều phải chứng minh.
- **lập luận đối chỗ** — biến một lời giải tối ưu bất kỳ thành lời giải tham lam qua các phép đổi không làm nó tệ đi (A/04).
- **matroid** — cấu trúc gồm một tập nền và một họ “tập độc lập” thỏa hai tính chất; đó chính xác là cấu trúc mà tham lam luôn cho kết quả tối ưu.
- **hàm dưới mô-đun (submodular)** — hàm mà lợi ích biên giảm dần khi tập đã chọn lớn lên. Tham lam trên hàm này cho bảo đảm xấp xỉ $1 - 1/e$.
- **tham lam có hối tiếc** — biến thể: cứ nhận tất cả, rồi khi vi phạm ràng buộc thì bỏ đi phần tử tệ nhất đã nhận — thường cài bằng heap.

CÂU HỎI MỞ ĐẦU

- Q1. Vì sao tham lam sai ở bài đổi tiền với mệnh giá $\{1, 3, 4\}$ mà lại đúng với mệnh giá tiền thật của hầu hết các nước?
- Q2. Kruskal và Prim đều là tham lam, đều cho cây khung nhỏ nhất, nhưng chọn cạnh theo hai tiêu chí hoàn toàn khác nhau. Vì sao cả hai cùng đúng?
- Q3. Có cách nào biết *trước* rằng tham lam sẽ đúng cho một bài, thay vì thử rồi hy vọng?
- Q4. Khi tham lam sai, vì sao câu trả lời thường là quy hoạch động chứ không phải một tiêu chí tham lam khôn hơn?
- Q5. Thuật toán tham lam cho bài phủ tập hợp không tối ưu nhưng vẫn được dùng rộng rãi. Điều gì làm nó đáng tin cậy dù không tối ưu?

Có một trải nghiệm phổ biến khi luyện thuật toán: bạn đọc đề, nghĩ ra một quy tắc đơn giản — “cứ chọn cái rẻ nhất”, “cứ chọn cái kết thúc sớm nhất” — cài trong ba phút, nộp, và sai. Bạn thử

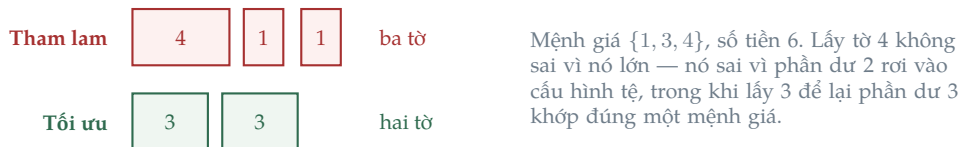
một quy tắc khác, cũng sai. Đến quy tắc thứ ba thì đúng, nhưng bạn không biết vì sao, và bài sau bạn lại phải đoán từ đầu.

Đó là trạng thái mà chương này muốn chấm dứt. Tham lam không phải trò may rủi; nó có lý thuyết, và lý thuyết đó trả lời được câu “bài này tham lam có đúng không” trước khi bạn gõ code. Nhưng phải nói ngay điều khó chịu: cái giá của việc dùng tham lam là bạn *phải chứng minh*, và không có cách nào né. Một lời giải tham lam không có chứng minh chỉ là một phỏng đoán được viết bằng ngôn ngữ lập trình.

3.1 Vì sao tham lam sai, nói cho chính xác

Trực giác thông thường nói tham lam sai vì nó “nhìn ngắn”. Cách nói đó đúng nhưng quá mơ hồ để dùng. Phát biểu chính xác hơn: tham lam sai khi lựa chọn hôm nay *phá huỷ những khả năng* mà một lựa chọn khác giữ lại được.

Hãy xem bài đổi tiền với mệnh giá $\{1, 3, 4\}$ và số tiền 6. Tham lam lấy 4, còn lại 2, phải lấy $1 + 1$ — ba tờ. Tối ưu là $3 + 3$ — hai tờ. Vấn đề không phải ở chỗ 4 lớn hơn 3, mà ở chỗ sau khi lấy 4 thì phần dư 2 rơi vào một cấu hình tệ, trong khi lấy 3 để lại phần dư 3 rơi đúng vào mệnh giá có sẵn.



Hình 3.1. Vì sao tham lam sai, nói cho chính xác. Vấn đề không nằm ở chỗ lựa chọn hiện tại “nhỏ” hay “lớn” mà ở chỗ nó *phá huỷ* cấu hình còn lại. Mọi chứng minh trong chương này đều nhằm loại trừ đúng khả năng đó.

Từ cách phát biểu này rút ra ngay một cách kiểm tra thực dụng: *sau khi tham lam chọn một phần tử, bài toán còn lại có còn cùng dạng và có tệ đi so với phương án khác không?* Nếu bạn chứng minh được rằng chọn theo tiêu chí tham lam để lại một bài toán con “không tệ hơn” mọi lựa chọn khác, thì tham lam đúng — và đó chính là nội dung của lập luận đổi chỗ.

3.2 Hai khuôn chứng minh

Chương A/04 đã giới thiệu hai khuôn; ở đây ta dùng chúng trên các bài thật.

Khuôn 1 — đổi chỗ. Lấy một lời giải tối ưu bất kỳ, xét chỗ khác đầu tiên so với tham lam, chứng minh rằng có thể sửa lời giải tối ưu tại đó cho giống tham lam mà không làm nó tệ đi. Áp dụng cho bài xếp lịch: tham lam chọn cuộc họp kết thúc sớm nhất; nếu lời giải tối ưu chọn cuộc khác thì đổi sang cuộc kết thúc sớm nhất vẫn hợp lệ vì nó kết thúc sớm hơn, nên không cản trở các cuộc sau.

Khuôn 2 — tham lam luôn đi trước. Chứng minh bằng quy nạp rằng sau k bước, lời giải tham lam “không kém” bất kỳ lời giải nào theo một đại lượng đo được. Với bài xếp lịch, đại lượng là thời điểm kết thúc của cuộc họp thứ k : tham lam luôn có thời điểm kết thúc sớm nhất có thể, nên nó không bao giờ bị kẹt trước lời giải khác.

Trong thực hành, khuôn 1 thường dễ áp hơn vì nó chỉ đòi hỏi một phép biến đổi cục bộ. Nhưng khuôn 2 có ưu điểm riêng: khi phép đổi chỗ không thực hiện được, chỗ vướng thường chính là phần ví dụ đang cần tìm.

Có một mẹo phụ rất hữu ích cho các bài mà tiêu chí tham lam là “sắp xếp theo cái gì đó”: nếu bạn nghi ngờ nên sắp theo tiêu chí f , hãy xét *hai phần tử liền kề* trong một lời giải tối ưu và hỏi “đổi chỗ hai phần tử này có làm kết quả tốt lên không”. Điều kiện để không đổi chỗ được sẽ cho bạn chính xác tiêu chí sắp xếp. Đây là cách người ta suy ra các tiêu chí lạ như “sắp theo tỉ số” hay “sắp theo hiệu” mà nhìn đề thì không đoán ra.

3.3 Matroid: cấu trúc mà tham lam luôn đúng

Đến đây có một câu hỏi tự nhiên: thay vì chứng minh lại cho từng bài, có đặc trưng chung nào cho các bài mà tham lam đúng không? Có, và nó là một trong những kết quả đẹp nhất của lý thuyết thuật toán.

Một *matroid* gồm một tập nền E và một họ các tập con được gọi là “độc lập”, thoả hai điều kiện: mọi tập con của một tập độc lập cũng độc lập; và nếu tập độc lập A nhỏ hơn tập độc lập B thì luôn tồn tại một phần tử trong $B \setminus A$ thêm được vào A mà vẫn giữ tính độc lập. Điều kiện thứ hai — gọi là tính chất trao đổi — chính là thứ đảm bảo rằng tham lam không bao giờ tự đưa mình vào ngõ cụt.

Định lý cốt lõi: *thuật toán tham lam cho kết quả tối ưu trên mọi hàm trọng số khi và chỉ khi cấu trúc là một matroid*. Chiều thuận nói tham lam đúng; chiều đảo còn mạnh hơn — nếu cấu trúc không phải matroid thì tồn tại một hàm trọng số làm tham lam sai.

Ví dụ điển hình: tập các tập cạnh không tạo chu trình của một đồ thị tạo thành một matroid. Đó là lý do Kruskal đúng — và cũng là câu trả lời cho câu hỏi mở đầu thứ hai: Kruskal và Prim khác nhau ở tiêu chí nhưng cùng khai thác một cấu trúc, nên cả hai cùng đúng. Một ví dụ khác: bài xếp lịch các công việc có hạn chót, mỗi việc một đơn vị thời gian, tối đa hoá lợi nhuận — tập các việc “xếp được” cũng tạo thành matroid.

Giá trị thực hành của khái niệm này không phải để bạn kiểm tra tiên đề mỗi lần gặp bài, mà là để có một danh mục: nếu bài của bạn quy về “chọn tập con độc lập trọng số lớn nhất” trên một cấu trúc kiểu rừng, kiểu không gian vector, hay kiểu phân hoạch, thì tham lam gần như chắc chắn đúng và bạn không cần đắn đo.

TRỰC GIÁC

Vì sao tính chất trao đổi lại là điều kiện đúng? Hãy nghĩ theo hướng ngược lại. Tham lam thất bại khi nó chọn xong k phần tử và rơi vào tình trạng “không thêm được gì nữa”, trong khi một lời giải khác chọn k phần tử khác lại còn mở rộng được. Tính chất trao đổi loại trừ đúng tình huống này: nếu lời giải kia lớn hơn thì luôn có phần tử của nó thêm được vào tập của tham lam. Nói cách khác, matroid là cách phát biểu hình thức cho điều kiện “tham lam không bao giờ tự bịt đường của chính mình”.

3.4 Danh mục các bài tham lam kinh điển

Bảng dưới đây gom các bài mà tiêu chí tham lam đã được chứng minh, kèm tiêu chí và lý do. Đáng thuộc vì các bài mới thường là biến thể của chúng.

Bài toán	Tiêu chí tham lam	Vì sao đúng
Chọn nhiều cuộc họp nhất	Kết thúc sớm nhất trước	Để lại nhiều thời gian nhất cho phần còn lại; đổi chỗ trực tiếp
Cây khung nhỏ nhất	Cạnh nhẹ nhất không tạo chu trình (Kruskal)	Cấu trúc rừng là matroid; tính chất cắt
Cây khung nhỏ nhất	Cạnh nhẹ nhất rời khỏi cây đang có (Prim)	Cùng tính chất cắt, duyệt theo thứ tự khác
Ba lô phân số	Tỉ số giá trị trên khối lượng	Được lấy phần lẻ nên không có “lãng phí chỗ”
Mã Huffman	Gộp hai tần suất nhỏ nhất	Hai ký hiệu hiếm nhất phải nằm sâu nhất; đổi chỗ
Xếp lịch giảm trễ tối đa	Hạn chót sớm nhất trước	Đổi chỗ hai việc liên kế không làm trễ tăng
Đổi tiền (hệ chuẩn)	Mệnh giá lớn nhất trước	Chỉ đúng với hệ mệnh giá đặc biệt — phải kiểm
Chọn việc có hạn chót, tối đa lợi nhuận	Lợi nhuận lớn trước, xếp muộn nhất có thể	Matroid phân hoạch theo thời điểm
Phủ tập hợp	Chọn tập phủ nhiều phần tử mới nhất	<i>Không</i> tối ưu, nhưng bảo đảm xấp xỉ $\ln n$

Có một mẫu đáng chú ý trong bảng: gần như mọi tiêu chí đều là một *thứ tự sắp xếp*. Điều đó nổi chương này với chương B/06: rất nhiều bài tham lam thực chất là “sắp xếp theo đúng khoá rồi quét một lượt”, và toàn bộ độ khó nằm ở việc tìm ra khoá.

Tham lam có hối tiếc

Một biến thể đáng thuộc vì nó giải một họ bài mà tham lam thuần tuý chịu. Ý tưởng: cứ nhận mọi thứ đến; khi vi phạm ràng buộc thì bỏ đi phần tử tệ nhất trong những cái đã nhận. Cấu trúc thực thi là heap (B/07).

Ví dụ: mỗi công việc có hạn chót và thời lượng, cần chọn nhiều việc nhất làm được. Duyệt theo hạn chót tăng dần, nhận mọi việc; nếu tổng thời gian vượt hạn chót hiện tại thì bỏ việc dài nhất đã nhận. Kết quả tối ưu, và chứng minh vẫn là đổi chỗ.

Điều đáng học ở đây có tính phương pháp: khi tiêu chí tham lam “chọn hay không chọn” quá khó, hãy đổi sang tiêu chí “nhận hết rồi loại dần”. Hai cách nhìn cho cùng bài toán, nhưng cách thứ hai thường có tiêu chí đơn giản hơn nhiều.

3.5 Khi tham lam sai: xấp xỉ có bảo đảm

Không phải cứ tham lam sai là vứt đi. Với nhiều bài NP-khó (D/23), tham lam là thuật toán tốt nhất trong thực tế, và điều làm nó đáng tin không phải là nó “thường đúng” mà là nó có *bảo đảm chất lượng chứng minh được*.

Hai kết quả kinh điển. Với bài phủ tập hợp, tham lam chọn tập phủ được nhiều phần tử mới nhất cho lời giải không tệ hơn $\ln n$ lần tối ưu — và người ta chứng minh được rằng dưới các giả thuyết chuẩn của lý thuyết độ phức tạp thì không thuật toán đa thức nào làm tốt hơn đáng kể. Nghĩa là tham lam ở đây không phải giải pháp tạm mà là gần như tốt nhất có thể.

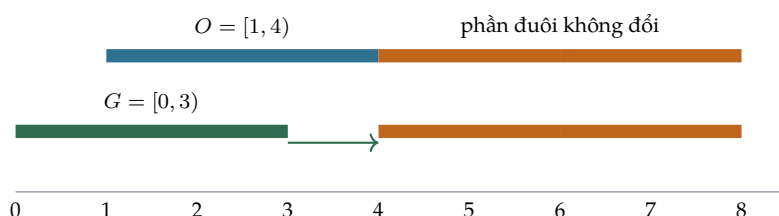
Kết quả thứ hai đẹp hơn và có ứng dụng rộng: khi hàm mục tiêu là *dưới mô-đun* — lợi ích biên giảm dần khi tập đã chọn lớn lên — và ta tối đa hoá dưới ràng buộc số lượng, thì tham lam cho ít nhất $1 - 1/e \approx 63\%$ giá trị tối ưu. Tính chất lợi ích biên giảm dần xuất hiện tự nhiên ở rất nhiều nơi: chọn vị trí đặt cảm biến, chọn tập ảnh đại diện, chọn đặc trưng cho mô hình, chọn khung hình chủ chốt trong một hệ thống bản đồ. Trong tất cả những bài đó, tham lam vừa đơn giản vừa có bảo đảm — một sự kết hợp hiếm.

CẠM BÃY

Bảo đảm xấp xỉ là bảo đảm về *trường hợp xấu nhất*, không phải mô tả hành vi thường gặp. Tham lam cho phủ tập hợp trong thực tế thường cho kết quả rất gần tối ưu, tốt hơn nhiều so với cận $\ln n$. Ngược lại, đừng lấy hành vi thực tế tốt làm cơ sở để bỏ qua việc kiểm tra: có những dữ liệu chạm đúng trường hợp xấu, và trong thi đấu thì dữ liệu *được cố ý* dựng như vậy.

3.6 Vẽ phép đổi chỗ: điều gì được bảo toàn

Trong bài chọn nhiều cuộc họp nhất, dùng đoạn nửa mở $[s, f)$ để hai cuộc chạm đầu mút vẫn tương thích. Hình 3.2 giữ nguyên phần đuôi của một lịch tối ưu, chỉ thay cuộc đầu tiên. Lý do đúng là $f(G) \leq f(O)$, không phải vì độ dài cuộc G ngắn hơn.



Hình 3.2. Đổi cuộc đầu tiên của lịch tối ưu sang cuộc kết thúc sớm nhất không làm mất cuộc nào ở phía sau. Đoạn trống từ 3 tới 4 cho thấy phần thời gian được nới ra sau phép đổi.

Nếu mỗi cuộc có lợi ích khác nhau, phép đổi vẫn giữ tính hợp lệ nhưng có thể giảm tổng lợi ích. Ví dụ $[0, 3)$ lợi ích 2 và $[0, 4)$ lợi ích 100: chọn kết thúc sớm cho giá trị 2 thay vì 100. Khi đó dùng quy hoạch động trên các cuộc đã sắp theo thời điểm kết thúc, với trạng thái có lựa chọn lấy hoặc bỏ cuộc hiện tại. Kinh nghiệm rút ra: mỗi lần đề thêm trọng số, hãy kiểm lại đại lượng được bảo toàn trong chứng minh cũ.

3.7 Gốc rễ: điện khí hoá Moravia và một bài tập về nhà

Bài toán cây khung nhỏ nhất có một nguồn gốc bất ngờ và rất cụ thể. Năm 1926, nhà toán học Séc Otakar Borůvka công bố thuật toán đầu tiên cho bài toán này, và động cơ là một nhu cầu kỹ thuật của thời đó: quy hoạch mạng lưới điện cho vùng Moravia sao cho tổng chiều dài dây dẫn nhỏ nhất. Thuật toán của ông — gộp đồng thời mọi thành phần với cạnh nhẹ nhất rời khỏi chúng — về sau được đánh giá cao vì rất dễ song song hoá, dù suốt nhiều thập kỷ nó ít nổi tiếng hơn hai thuật toán sau. Jarník tìm ra thuật toán kiểu Prim năm 1930, cũng ở Tiệp Khắc; Kruskal và Prim công bố độc lập vào năm 1956 và 1957 tại Mỹ, trong bối cảnh nghiên cứu mạng viễn thông.

Mã Huffman có một câu chuyện thường được kể trong giới: năm 1952, David Huffman là sinh viên cao học ở MIT, và giáo sư cho lớp chọn giữa việc thi cuối kỳ hoặc viết một tiểu luận về việc tìm mã tiền tố tối ưu — một bài toán mà chính giáo sư và các đồng nghiệp chưa giải được. Huffman chọn tiểu luận, gần như bỏ cuộc, rồi tìm ra ý tưởng gộp từ dưới lên. Điều đáng học không phải giai thoại mà là hướng suy nghĩ: mọi người trước đó cố xây cây *từ trên xuống*, còn lời giải nằm ở hướng ngược lại. Đây là một ví dụ trực tiếp cho heuristic “đảo ngược hướng nhìn” ở chương A/01.

Phần lý thuyết đến muộn hơn nhiều và từ một ngành khác. Khái niệm matroid do Hassler Whitney đưa ra năm 1935 trong bối cảnh toán học thuần túy — ông muốn trừu tượng hoá khái niệm

“độc lập tuyến tính” để nó dùng được cho cả đồ thị. Mãi tới đầu thập niên 1970, Jack Edmonds mới nối hai đầu lại và chứng minh rằng matroid *chính xác* là cấu trúc mà tham lam tối ưu. Đây là một mẫu lặp lại trong lịch sử ngành: một khái niệm toán học ra đời không vì ứng dụng, ba mươi năm sau trở thành câu trả lời chính xác cho một câu hỏi thuật toán.

I Gốc rễ: ý tưởng này ra đời từ bài toán nào

Mốc	Bài toán thúc ép lúc đó	Điều thay đổi sau đó
1926 — Borůvka	Quy hoạch mạng điện cho vùng Moravia với tổng chiều dài dây nhỏ nhất.	Thuật toán cây khung nhỏ nhất đầu tiên; ít nổi tiếng nhưng song song hoá tốt nhất trong ba thuật toán.
1930, 1956–57 — Jarník, Kruskal, Prim	Thiết kế mạng viễn thông chi phí thấp.	Hai tiêu chí tham lam khác nhau cho cùng bài toán; cả hai đều đúng vì cùng khai thác tính chất cắt.
1952 — Huffman	Tìm mã tiền tố tối ưu — bài toán mở mà thầy giao thay cho bài thi.	Xây cây từ dưới lên thay vì từ trên xuống; ví dụ trực tiếp của heuristic “đảo ngược hướng nhìn”.
1935 — Whitney	Trừu tượng hoá khái niệm độc lập tuyến tính cho cả đồ thị.	Matroid ra đời trong toán thuần túy, không nhằm ứng dụng nào.
Đầu thập niên 1970 — Edmonds	Vì sao tham lam đúng ở bài này mà sai ở bài kia?	Định lý: tham lam tối ưu trên mọi trọng số khi và chỉ khi cấu trúc là matroid — một điều kiện <i>cần và đủ</i> , hiếm gặp trong lý thuyết thuật toán.
1978 — Nemhauser và cộng sự	Tối đa hoá hàm có lợi ích biên giảm dần dưới ràng buộc số lượng.	Tham lam bảo đảm $1 - 1/e$; nền lý thuyết cho việc chọn cảm biến, chọn đặc trưng, chọn dữ liệu đại diện.

I 3.8 Liên hệ ngang

I Liên hệ ngang: ý này còn sống ở đâu nữa

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu
Kinh tế học	Quyết định cận biên, tối ưu hoá thiên cận	Cùng cấu trúc: chọn theo lợi ích tức thời. Cùng chỗ hỏng: khi lựa chọn hôm nay thay đổi tập lựa chọn ngày mai. Kinh tế học gọi đó là chi phí cơ hội động; ta gọi là “phá huỷ khả năng tương lai”.
Tiến hoá, tối ưu hoá cục bộ	Leo đồi: luôn đi theo hướng cải thiện	Tham lam là leo đồi trên không gian tổ hợp. Cùng thất bại: kẹt ở cực trị địa phương. Cách chữa cũng tương tự — cho phép bước lùi, tức là mô phỏng luyện kim (C/19).
Hệ điều hành, bộ nhớ đệm	Chính sách thay thế LRU, lập lịch tiến trình	LRU là tham lam trên thông tin quá khứ. Điểm khác quan trọng: ở đây tương lai <i>chưa biết</i> , nên tiêu chuẩn đánh giá không phải tối ưu mà là tỉ lệ cạnh tranh so với lời giải biết trước tương lai (D/24).
Nén dữ liệu	Mã Huffman, mã số học	Ứng dụng tham lam kinh điển nhất, và vẫn đang chạy trong mọi tệp ảnh nén bạn mở hôm nay.
Học máy	Chọn đặc trưng tham lam, cây quyết định, học tích cực	Cây quyết định chọn thuộc tính chia tốt nhất tại mỗi nút — tham lam thuần túy, và nó <i>không</i> tối ưu toàn cục nhưng chạy tốt trong thực tế. Nhiều tiêu chí chọn dữ liệu có tính dưới mô-đun nên tham lam có bảo đảm $1 - 1/e$.

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu (tiếp)
Robot và SLAM	Chọn khung hình chủ chốt, chọn điểm mốc, chọn cảm biến	Bài toán “chọn k quan sát cho nhiều thông tin nhất” thường có hàm mục tiêu dưới mô-đun — lợi ích của một quan sát mới giảm khi đã có nhiều quan sát tương tự. Đó là lý do tham lam ở đây không chỉ tiện mà còn có bảo đảm chất lượng.
Tài chính	Chiến lược thiên cận so với tối ưu động	Cùng bài học: tối ưu từng bước không cho tối ưu cả chuỗi khi có ràng buộc liên thời gian. Đây cũng chính là ranh giới dẫn sang quy hoạch động ở C/16.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì tính đúng của tham lam ở bài đổi tiền là tính chất của *hệ mệnh giá*, không phải của thuật toán. Với $\{1, 3, 4\}$ và số tiền 6, lấy 4 để lại phần dư 2 rơi vào cấu hình tệ, trong khi lấy 3 để lại phần dư 3 khớp đúng một mệnh giá. Các hệ tiền thật thường có cấu trúc bội số $(1, 2, 5, 10, \dots)$ làm cho việc lấy mệnh giá lớn nhất không bao giờ phá huỷ khả năng của phần dư. Có thuật toán kiểm tra một hệ mệnh giá có “tham lam được” hay không, và đó là bằng chứng rõ ràng rằng đây là tính chất của dữ liệu chứ không của phương pháp.
- A2. Vì cả hai đều dựa trên cùng một tính chất, gọi là tính chất cắt: với mọi cách chia đỉnh thành hai nhóm, cạnh nhẹ nhất nối hai nhóm luôn thuộc một cây khung nhỏ nhất nào đó. Kruskal áp dụng nó bằng cách duyệt cạnh theo trọng số tăng; Prim áp dụng bằng cách luôn xét cắt giữa cây đang có và phần còn lại. Nói theo ngôn ngữ mục 3: tập các tập cạnh không tạo chu trình là một matroid, và mọi cách duyệt tham lam hợp lệ trên matroid đều cho kết quả tối ưu.
- A3. Có, ít nhất cho một lớp lớn: nếu bài toán quy về “chọn tập con độc lập có trọng số lớn nhất” trên một cấu trúc thoả hai tiên đề matroid thì tham lam chắc chắn tối ưu, và định lý của Edmonds nói điều ngược lại cũng đúng — không phải matroid thì tồn tại trọng số làm tham lam sai. Trong thực hành, thay vì kiểm tiên đề, người ta nhận diện qua danh mục: cấu trúc rừng, không gian vector, phân hoạch theo nhóm. Ngoài lớp đó thì vẫn phải chứng minh bằng đổi chỗ cho từng bài.
- A4. Vì khi không có lựa chọn cục bộ nào an toàn, vấn đề không nằm ở việc tiêu chí chưa đủ khôn mà nằm ở chỗ *lời giải tối ưu phụ thuộc vào những gì đã chọn trước đó*. Nói cách khác, bài toán có trạng thái. Quy hoạch động là cách xử lý đúng cho tình huống đó: thay vì cam kết một lựa chọn, ta xét mọi lựa chọn nhưng nhớ lại kết quả theo trạng thái để khởi tính lại (C/16). Cố tìm tiêu chí tham lam khôn hơn trong tình huống này thường dẫn tới một tiêu chí phức tạp mà vẫn có phản ví dụ.
- A5. Bảo đảm chất lượng *chứng minh được*: nó không bao giờ tệ hơn tối ưu quá $\ln n$ lần cho bài phủ tập hợp, hoặc không kém $1 - 1/e$ lần tối ưu khi hàm mục tiêu có lợi ích biên giảm dần. Điều này khác hẳn với “thực nghiệm thấy tốt”: bảo đảm đúng cho mọi dữ liệu, kể cả dữ liệu do đối thủ dựng. Và với bài phủ tập hợp còn có kết quả mạnh hơn: dưới các giả thuyết chuẩn về độ phức tạp, không thuật toán đa thức nào cải thiện được cận đó đáng kể — nên tham lam gần như là tốt nhất có thể.

NÓI LẠI THẬT ĐƠN GIẢN

Tham lam là cách giải mà ai cũng thử đầu tiên: cứ chọn cái tốt nhất trước mắt, rồi tính tiếp. Nhiều khi nó đúng, nhiều khi nó sai, và chương này nói về cách phân biệt hai trường hợp đó. Hãy nghĩ tới việc trả tiền lẻ. Nếu có các tờ 1, 2, 5, 10 thì cứ lấy tờ to nhất còn dùng được là ổn. Nhưng nếu tiền chỉ có mệnh giá 1, 3 và 4, mà bạn phải trả 6, thì lấy tờ 4 trước là hỏng — còn 2 và bạn phải dùng hai tờ 1, tổng ba tờ; trong khi hai tờ 3 là đủ. Điều làm hỏng không phải vì tờ 4 nhỏ hay to, mà vì sau khi lấy nó thì phần còn lại rơi vào tình thế xấu. Đó chính là câu hỏi duy nhất cần hỏi mỗi lần định dùng tham lam: *sau khi tôi chọn cái này, phần còn lại có bị kẹt hơn so với chọn cái khác không?* Nếu chứng minh được là không bao giờ kẹt hơn, tham lam đúng. Cách chứng minh chuẩn cũng rất trực quan: đưa tôi bất kỳ cách làm tốt nhất nào của bạn, tôi sẽ sửa dần nó cho giống cách của tôi, mỗi lần sửa đều không làm nó xấu đi.

Có một điều thú vị mà các nhà toán học đã tìm ra: tồn tại một loại cấu trúc mà trên đó tham lam *luôn luôn* đúng, bất kể trọng số là gì. Nếu bài toán của bạn có dạng “chọn một nhóm phần tử sao cho không tạo thành vòng” — như chọn dây điện nối các thành phố mà không tạo vòng lặp — thì bạn ở trong loại cấu trúc đó và có thể yên tâm.

Cuối cùng, khi tham lam sai, đôi khi ta vẫn dùng nó — nhưng chỉ khi biết rõ nó sai tới mức nào. Có những bài mà tham lam bảo đảm cho kết quả ít nhất bằng khoảng hai phần ba mức tốt nhất. Biết con số đó là điều khác biệt giữa “dùng liều” và “dùng có căn cứ”.

Một câu: Tham lam không sai vì nhìn ngắn; nó sai khi lựa chọn hôm nay bịt mắt đường của ngày mai.

Chỗ không rút gọn được: Tìm ra tiêu chí tham lam đúng cho một bài mới vẫn là bước sáng tạo; matroid chỉ nhận diện được một lớp, không phải mọi bài.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
●●● Tham lam sai khi nào — phát biểu chính xác	Khi lựa chọn hiện tại phá hủy những khả năng mà lựa chọn khác giữ lại được, chứ không phải vì “nhìn ngắn”.
●●● Hai khuôn chứng minh	Đổi chỗ (biến lời giải tối ưu thành tham lam, từng bước không tệ đi) và “tham lam luôn đi trước” (quy nạp theo một đại lượng đo được).
●●● Mẹo tìm tiêu chí sắp xếp	Xét hai phần tử liền kề trong lời giải tối ưu, hỏi “đổi chỗ có tốt lên không”; điều kiện không đổi được cho ra chính tiêu chí sắp xếp.
●●● Matroid — định lý cốt lõi	Tham lam tối ưu trên mọi hàm trọng số <i>khi và chỉ khi</i> cấu trúc là matroid. Rừng trong đồ thị là matroid, nên Kruskal đúng.
●●● Tính chất cắt	Với mọi cách chia đỉnh thành hai nhóm, cạnh nhẹ nhất nối hai nhóm thuộc một cây khung nhỏ nhất — nền của cả Kruskal lẫn Prim.
●●● Khi tham lam sai thì làm gì	Chuyển sang quy hoạch động, vì bài toán có trạng thái: lời giải tối ưu phụ thuộc những gì đã chọn trước đó (C/16).
●● Tham lam có hối tiếc	Nhận hết, khi vi phạm thì bỏ phần tử tệ nhất đã nhận (dùng heap). Đổi tiêu chí “chọn hay không” thành “nhận rồi loại”, thường đơn giản hơn nhiều.
●● Huffman	Gộp hai tần suất nhỏ nhất; ý tưởng đến từ việc xây cây <i>từ dưới lên</i> thay vì <i>từ trên xuống</i> .
●● Ba lô phân số so với ba lô nguyên	Phân số: tham lam theo tỉ số, tối ưu. Nguyên: tham lam sai, cần quy hoạch động — khác nhau đúng ở chỗ “có lấy phần lẻ được không”.
●● Bảo đảm xấp xỉ của tham lam	Phủ tập hợp: không tệ hơn $\ln n$ lần tối ưu. Hàm dưới mô-đun dưới ràng buộc số lượng: ít nhất $1 - 1/e$ lần tối ưu.
●● Hàm dưới mô-đun	Lợi ích biên giảm dần khi tập đã chọn lớn lên; xuất hiện tự nhiên ở chọn cảm biến, chọn đặc trưng, chọn khung hình chủ chốt.
● Bảo đảm xấp xỉ nghĩa là gì	Bảo đảm cho trường hợp xấu nhất, không mô tả hành vi thường gặp — thực tế thường tốt hơn nhiều, nhưng dữ liệu thi đấu được dựng để chạm đúng trường hợp xấu.

●●● phải thuộc ●● nên nắm ● biết là có

Câu hỏi ôn tập có đáp án (che phần đáp án, tự trả lời trước)

1. Trong kỳ thi, tôi nên dành bao lâu để chứng minh một tiêu chí tham lam?

| Một quy trình thực dụng: dành hai phút thử đổi chỗ trong đầu; nếu ra thì cài. Nếu không ra sau hai phút, hãy viết bộ sinh test ngẫu nhiên nhỏ và so với vết cặn — năm phút và cho câu trả lời

chắc chắn hơn nhiều so với ngồi nghĩ thêm mười phút. Nếu stress test không tìm được phản ví dụ sau vài nghìn ca nhỏ, xác suất tiêu chí đúng là khá cao. Điều tuyệt đối không nên làm là cài rồi nộp mà không làm cả hai việc trên.

2. Bài của tôi có nhiều tiêu chí tham lam hợp lý. Chọn thế nào?

| Dùng mẹo hai phần tử liền kề ở mục 2: giả sử lời giải tối ưu xếp phần tử a ngay trước b ; viết ra điều kiện để việc đổi chỗ chúng *không* cải thiện kết quả. Bất đẳng thức thu được chính là tiêu chí so sánh cần dùng để sắp xếp, và nó thường có dạng bất ngờ như “sắp theo $a_i + b_i$ ” hoặc “sắp theo tỉ số”. Cách này biến việc đoán tiêu chí thành việc giải một bất đẳng thức — xác định hơn nhiều.

3. Nếu tham lam cho kết quả gần đúng thì có nên nộp không?

| Trong thi đấu thì không, vì chấm là đúng hoặc sai. Trong công việc thật thì tùy, và câu hỏi đúng là: bạn có *bảo đảm* nào không? “Thử vài ca thấy gần đúng” rất khác với “chứng minh được không tệ hơn $1 - 1/e$ lần tối ưu”. Nếu có bảo đảm và sai số chấp nhận được thì tham lam là lựa chọn tốt: đơn giản, nhanh, dễ bảo trì. Nếu không có bảo đảm, ít nhất hãy đo trên dữ liệu thật và biết phân phối sai số.

4. Tham lam và quy hoạch động có thể kết hợp không?

| Có, theo nhiều cách. Cách phổ biến nhất: dùng tham lam để giảm không gian trạng thái — chứng minh rằng một số lựa chọn không bao giờ tối ưu và loại chúng, rồi quy hoạch động trên phần còn lại. Cách thứ hai: dùng lời giải tham lam làm cận trên trong nhánh cận (C/13). Cách thứ ba, thường gặp trong bài khó: sắp xếp bằng tiêu chí tham lam trước, rồi quy hoạch động trên thứ tự đó — vì tiêu chí đúng thu hẹp bài toán từ “mọi thứ tự” xuống “một thứ tự”.

5. Vì sao Dijkstra được xếp vào tham lam?

| Vì nó cam kết một lựa chọn không bao giờ xét lại: mỗi bước chốt đỉnh có nhãn nhỏ nhất và coi khoảng cách đó là cuối cùng. Chứng minh tính đúng chính là chứng minh lựa chọn tham lam an toàn, và nó dùng giả thiết trọng số không âm đúng một lần — ở chỗ khẳng định rằng không có đường nào đi vòng qua các đỉnh chưa chốt rồi quay lại mà ngắn hơn. Đây là ví dụ tốt cho việc “giả thiết nằm ở đâu trong chứng minh” quyết định “thuật toán hỏng ở đâu”.

6. Tính dưới mô-đun kiểm tra thế nào trong thực tế?

| Định nghĩa là: thêm một phần tử vào tập nhỏ cho lợi ích ít nhất bằng thêm chính phần tử đó vào tập lớn hơn chứa nó. Trong thực hành, ba dấu hiệu gợi ý mạnh: hàm đếm số phần tử được phủ (phủ tập hợp); hàm đo lượng thông tin thu được từ các quan sát có phần trùng lặp; và hàm đo độ đa dạng của một tập đã chọn. Ba họ này phủ phần lớn ứng dụng thực tế. Nếu nghi ngờ, hãy kiểm tra bằng số trên các trường hợp nhỏ trước khi dựa vào bảo đảm lý thuyết.

7. Có bài nào mà tham lam đúng nhưng chứng minh rất khó không?

| Có, và một ví dụ nổi tiếng là bài xếp lịch với hạn chót và tiền phạt, hay các biến thể của bài toán ghép đôi ổn định. Thuật toán ghép đôi ổn định — mỗi bên lần lượt cầu hôn theo thứ tự ưu tiên — trông rất giống tham lam và luôn cho kết quả ổn định, nhưng chứng minh đòi hỏi lập luận tinh tế về việc không tồn tại cặp muốn bỏ nhau. Bài học: độ ngắn của code không nói gì về độ khó của chứng minh, và đó là chủ đề của cả chương này.

Bài tập (mức: ● khởi động ●● chuẩn ●●● khó)

Bài	Nguồn	Điều cần nhận ra
● Chọn nhiều cuộc họp nhất không chồng lấn	CSES “Movie Festival”[1]	Sắp theo thời điểm kết thúc; viết chứng minh đổi chỗ ra giấy.
● Ba lô phân số	Bài kinh điển	Sắp theo tỉ số; giải thích vì sao cách này sai với ba lô nguyên.
● Số tiền nhỏ nhất không tạo được từ các mệnh giá đã có	CSES “Coin Piles”-loại[1]	Sắp tăng dần và duy trì tổng đạt được; bất biến rất gọn.

Bài	Nguồn	Điều cần nhận ra (tiếp)
•• Mã Huffman	Bài kinh điển	Cài bằng heap; sau đó tự chứng minh bằng đổi chỗ hai ký hiệu hiếm nhất.
•• Cây khung nhỏ nhất bằng Kruskal và bằng Prim	CSES “Road Reparation”[1]	Cài cả hai, so thời gian trên đồ thị thưa và dày; giải thích khác biệt.
•• Xếp lịch giảm thiểu độ trễ lớn nhất	Bài kinh điển	Sắp theo hạn chót; chứng minh bằng đổi chỗ hai việc liền kề.
•• Chọn nhiều việc nhất khi mỗi việc có thời lượng và hạn chót	Bài kinh điển	Tham lam có hồi tiếc với heap; ví dụ mẫu cho mẫu “nhận hết rồi loại”.
••• Sắp thứ tự công việc trên hai máy để tổng thời gian nhỏ nhất	Bài kinh điển	Tiêu chí sắp xếp lại; hãy dùng mẹo đổi chỗ hai phần tử liền kề để tự dẫn ra.
••• Phủ tập hợp bằng tham lam, so với tối ưu trên dữ liệu nhỏ	Bài tập thực nghiệm	Đo tỉ số thực tế so với cận $\ln n$; đây là cách tốt nhất để hiểu “bảo đảm xấp xỉ” nghĩa là gì.
••• Chọn k vị trí đặt cảm biến để phủ nhiều nhất	Bài tập ứng dụng	Hàm mục tiêu dưới mô-đun; tham lam có bảo đảm $1 - 1/e$ — kiểm chứng bằng thực nghiệm.
••• Tìm một hệ mệnh giá làm tham lam đổi tiền sai, càng nhỏ càng tốt	Bài tập thiết kế	Cách rẻ nhất là stress test với vết cạn; bài này luyện đúng thói quen chương E/25.
••• Ghép đôi ổn định	Bài kinh điển	Cài thuật toán cầu hôn; thử chứng minh không tồn tại cặp muốn bỏ nhau — một chứng minh khó hơn về ngoài của thuật toán.

Kết chương và đọc tiếp

Tham lam là chiến lược ngắn nhất về code và đắt nhất về nghĩa vụ chứng minh. Ba điều đáng mang theo: phát biểu chính xác về lý do tham lam sai — phá huỷ khả năng tương lai; mẹo đổi chỗ hai phần tử liền kề để *suy ra* tiêu chí sắp xếp thay vì đoán; và ý thức rằng có một lớp cấu trúc (matroid) mà trên đó tham lam chắc chắn đúng.

Khi không lựa chọn cục bộ nào an toàn, ta buộc phải xét nhiều khả năng — nhưng không phải xét lại từ đầu mỗi lần. Chương tiếp theo là chương dài nhất và quan trọng nhất của phần C: quy hoạch động, kỹ thuật biến “xét mọi khả năng” từ hàm mũ thành đa thức bằng cách nhận ra rằng rất nhiều nhánh khác nhau dẫn tới cùng một trạng thái (C/16).

Nguồn

- [1] A. Laaksonen. “CSES Problem Set.” (2026), address: <https://cses.fi/problemset/> (visited on 09/06/2026).
- [2] M. Blum, R. W. Floyd, V. Pratt, et al., “Time Bounds for Selection,” *Journal of Computer and System Sciences*, vol. 7, no. 4, pp. 448–461, 1973.
- [3] V. Strassen, “Gaussian Elimination Is Not Optimal,” *Numerische Mathematik*, vol. 13, pp. 354–356, 1969.
- [4] D. Harvey and J. van der Hoeven, “Integer Multiplication in Time $O(n \log n)$,” *Annals of Mathematics*, vol. 193, no. 2, pp. 563–617, 2021.
- [5] J. Alman and V. Vassilevska Williams, “A Refined Laser Method and Faster Matrix Multiplication,” in *Proceedings of the 32nd ACM-SIAM Symposium on Discrete Algorithms (SODA)*, 2021.
- [6] J. Alman, R. Duan, V. Vassilevska Williams, et al. “Improving the Matrix Multiplication Exponent with Modern Optimization and AlphaEvolve.” bản tiền ấn arXiv:2608.16884, chưa qua phản biện. (2026), address: <https://arxiv.org/abs/2608.16884> (visited on 09/07/2026).
- [7] J. W. Cooley and J. W. Tukey, “An Algorithm for the Machine Calculation of Complex Fourier Series,” *Mathematics of Computation*, vol. 19, no. 90, pp. 297–301, 1965.
- [8] D. R. Musser, “Introspective Sorting and Selection Algorithms,” *Software: Practice and Experience*, vol. 27, no. 8, pp. 983–993, 1997.
- [9] J. L. Bentley and M. D. McIlroy, “Engineering a Sort Function,” *Software: Practice and Experience*, vol. 23, no. 11, pp. 1249–1265, 1993.
- [10] D. E. Knuth, “Optimum Binary Search Trees,” *Acta Informatica*, vol. 1, pp. 14–25, 1971.
- [11] F. F. Yao, “Speed-Up in Dynamic Programming,” *SIAM Journal on Algebraic and Discrete Methods*, vol. 3, no. 4, pp. 532–540, 1982.
- [12] A. Aggarwal, M. M. Klawe, S. Moran, et al., “Geometric Applications of a Matrix-Searching Algorithm,” *Algorithmica*, vol. 2, pp. 195–208, 1987.
- [13] D. S. Hirschberg, “A Linear Space Algorithm for Computing Maximal Common Subsequences,” *Communications of the ACM*, vol. 18, no. 6, pp. 341–343, 1975.
- [14] R. E. Tarjan, “Depth-First Search and Linear Graph Algorithms,” *SIAM Journal on Computing*, vol. 1, no. 2, pp. 146–160, 1972.
- [15] M. L. Fredman and R. E. Tarjan, “Fibonacci Heaps and Their Uses in Improved Network Optimization Algorithms,” *Journal of the ACM*, vol. 34, no. 3, pp. 596–615, 1987.
- [16] M. Thorup, “Undirected Single-Source Shortest Paths with Positive Integer Weights in Linear Time,” *Journal of the ACM*, vol. 46, no. 3, pp. 362–394, 1999.
- [17] R. Duan, J. Mao, X. Mao, et al., “Breaking the Sorting Barrier for Directed Single-Source Shortest Paths,” in *Proceedings of the 57th Annual ACM Symposium on Theory of Computing (STOC)*, Best Paper Award, 2025.
- [18] E. W. Dijkstra, “A Note on Two Problems in Connexion with Graphs,” *Numerische Mathematik*, vol. 1, pp. 269–271, 1959.

- [19] KTH Royal Institute of Technology. “KACTL: KTH Algorithm Competition Template Library.” (2026), address: <https://github.com/kth-competitive-programming/kactl> (visited on 09/06/2026).
- [20] H. W. Kuhn, “The Hungarian Method for the Assignment Problem,” *Naval Research Logistics Quarterly*, vol. 2, no. 1-2, pp. 83–97, 1955.
- [21] J. B. Orlin, “Max Flows in $O(nm)$ Time, or Better,” in *Proceedings of the 45th Annual ACM Symposium on Theory of Computing (STOC)*, 2013, pp. 765–774.
- [22] L. Chen, R. Kyng, Y. P. Liu, et al., “Maximum Flow and Minimum-Cost Flow in Almost-Linear Time,” in *Proceedings of the 63rd IEEE Symposium on Foundations of Computer Science (FOCS)*, Best Paper Award, 2022.
- [23] J. E. Hopcroft and R. M. Karp, “An $n^{5/2}$ Algorithm for Maximum Matchings in Bipartite Graphs,” *SIAM Journal on Computing*, vol. 2, no. 4, pp. 225–231, 1973.
- [24] J. Bentley, *Programming Pearls*, 2nd ed. Addison-Wesley, 2000.
- [25] R. Motwani and P. Raghavan, *Randomized Algorithms*. Cambridge University Press, 1995.

Quy hoạch động (Dynamic Programming)

PHỤ THUỘC C/13 cho lời giải vết cạn làm điểm xuất phát; C/15 cho ranh giới “khi nào tham lam đủ”; A/04 cho lập luận cắt-dán chứng minh cấu trúc con tối ưu.

TÓM TẮT

Quy hoạch động là kỹ thuật quan trọng nhất của phần C và cũng là kỹ thuật bị dạy tệ nhất. Nó không phải một danh sách bài mẫu cần thuộc; nó là một quy trình năm bước — định nghĩa trạng thái, viết công thức chuyển, xác định cơ sở, chọn thứ tự tính, đọc đáp án — trong đó bước một chiếm chín phần mười độ khó. Chương đi qua sáu họ trạng thái hay gặp (một chiều, ba lô, hai chuỗi, khoảng, trên cây, mặt nạ bit), cách giảm bộ nhớ, cách truy vết lời giải, và bốn kỹ thuật tăng tốc khi số trạng thái nhân số chuyển quá lớn. Nếu chỉ nhớ một điều: trạng thái phải *đủ* — biết trạng thái thì phần tương lai không cần biết gì thêm về quá khứ — và mọi lỗi quy hoạch động đều quy về việc vi phạm đúng câu này.

KIẾN THỨC CẦN CÓ TRƯỚC

- **trạng thái (state)** — bản tóm tắt của quá khứ, đủ để quyết định tương lai. Nếu hai lịch sử khác nhau dẫn tới cùng trạng thái thì phần còn lại của bài toán là như nhau.
- **cấu trúc con tối ưu** — lời giải tối ưu của bài lớn chứa lời giải tối ưu của các bài con; chứng minh bằng lập luận cắt-dán (A/04).
- **bài toán con chồng lặp** — các nhánh khác nhau của cây đệ quy gặp lại cùng bài toán con — đây là lý do việc ghi nhớ có tác dụng.
- **ghi nhớ (memoization)** — cách viết từ trên xuống: hàm đệ quy lưu kết quả đã tính vào bảng, lần sau tra thay vì tính lại.
- **bảng từ dưới lên (tabulation)** — cách viết lặp: điền bảng theo một thứ tự bảo đảm mọi phụ thuộc đã có sẵn.
- **mảng cuộn (rolling array)** — kỹ thuật giảm bộ nhớ: nếu tầng i chỉ phụ thuộc tầng $i - 1$ thì chỉ cần giữ hai tầng.

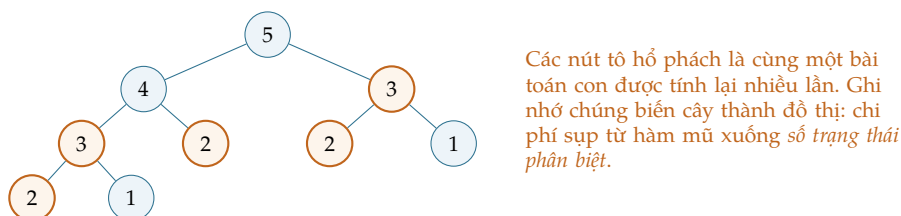
CÂU HỎI MỞ ĐẦU

- Q1. Quy hoạch động chỉ là đệ quy cộng một cái bảng. Vì sao nó lại khó tới vậy với hầu hết người học?
- Q2. Vì sao “chọn sai trạng thái” là lỗi phổ biến nhất, và làm sao biết trạng thái của mình *đủ* trước khi cài?
- Q3. Bài ba lô có lời giải $O(nW)$, nhìn thì đa thức. Vì sao bài ba lô vẫn được xếp vào NP-khó?
- Q4. Viết từ trên xuống (ghi nhớ) và từ dưới lên (điền bảng) cho cùng kết quả. Khi nào phải chọn cách nào?
- Q5. Khi số trạng thái đã tối thiểu mà vẫn quá chậm, còn hướng nào ngoài việc đổi thuật toán?

Người học quy hoạch động thường đi qua ba giai đoạn. Giai đoạn một: đọc lời giải thấy hiển nhiên, tự làm thì không nghĩ ra. Giai đoạn hai: nhận ra một số dạng quen thuộc và giải được các bài gần với chúng, nhưng gặp bài lạ thì lại đứng. Giai đoạn ba: nhìn bài mới và tự đặt ra được

trạng thái. Khoảng cách giữa giai đoạn hai và ba là khoảng cách giữa “thuộc bài mẫu” và “nắm phương pháp”, và chương này viết cho đúng khoảng cách đó.

Điểm khởi đầu phải là chương C/13: quy hoạch động không phải kỹ thuật độc lập mà là *vết cạn cộng trí nhớ*. Bạn viết lời giải đệ quy duyệt mọi khả năng — đúng nhưng chậm — rồi quan sát rằng rất nhiều nhánh khác nhau của cây đệ quy dẫn tới cùng một tình huống. Nếu tình huống đó mô tả được gọn thì lưu kết quả lại, và chi phí sụp từ hàm mũ xuống bằng *số tình huống phân biệt*. Cả chương này chỉ là các cách trả lời câu hỏi “tình huống được mô tả gọn bằng cái gì”.



Các nút tô hổ phách là cùng một bài toán con được tính lại nhiều lần. Ghi nhớ chúng biến cây thành đồ thị: chi phí sụp từ hàm mũ xuống *số trạng thái phân biệt*.

Hình 4.1. Cây đệ quy của một lời giải vét cạn. Quy hoạch động không phải kỹ thuật mới mà chỉ là quan sát rằng cây này có rất nhiều nút lặp lại. Toàn bộ độ khó nằm ở chỗ mô tả “nút nào giống nút nào” cho gọn — tức là ở việc chọn trạng thái.

4.1 Quy trình năm bước

Mọi lời giải quy hoạch động đều được viết theo cùng năm bước. Viết chúng ra thành lời *trước khi gõ code* là thói quen tạo ra khác biệt lớn nhất, đúng tinh thần chương A/04.

Bước 1 — định nghĩa trạng thái. Viết một câu hoàn chỉnh dạng “ $f[i][j]$ là ...”. Câu này phải rõ tới mức người khác đọc xong tính được ví dụ nhỏ bằng tay. Đây là bước chiếm chín phần mười độ khó, và mục 2 dành riêng cho nó.

Bước 2 — công thức chuyển. Từ trạng thái hiện tại, liệt kê *mọi* quyết định khả dĩ và trạng thái mà mỗi quyết định dẫn tới. Yêu cầu quan trọng: các nhánh phải *phủ hết* và tốt nhất là *không chồng nhau* (nếu đang đếm).

Bước 3 — cơ sở. Trạng thái nhỏ nhất và giá trị của nó. Rất nhiều lỗi nằm ở đây, đặc biệt việc phân biệt “không có cách nào” với “có đúng một cách rỗng”.

Bước 4 — thứ tự tính. Phải bảo đảm khi tính một trạng thái thì mọi trạng thái nó phụ thuộc đã xong. Nói cách khác, đồ thị phụ thuộc phải không có chu trình — và khi nó có chu trình thì bạn đang ở bài toán khác, cần Dijkstra hoặc giải hệ phương trình.

Bước 5 — đáp án và truy vết. Đáp án nằm ở trạng thái nào; nếu cần chỉ ra lời giải cụ thể chứ không chỉ giá trị, hãy lưu quyết định tại mỗi trạng thái hoặc dò ngược bằng cách so sánh giá trị.

4.2 Bước khó nhất: trạng thái phải đủ

Tiêu chuẩn duy nhất cho một trạng thái tốt: *biết trạng thái thì phần còn lại của bài toán hoàn toàn xác định, không cần biết thêm bất cứ điều gì về cách ta tới đó*. Trong ngôn ngữ xác suất, đây là tính chất Markov; trong ngôn ngữ chương A/04, đây chính là một bất biến.

Có ba cách phát hiện trạng thái thiếu, và cả ba đều rẻ hơn việc cài rồi mới sai.

Cách thứ nhất — thử kể lại tương lai. Đặt câu hỏi: “tôi đang ở trạng thái $f[i]$, tôi cần quyết định gì tiếp theo, và tôi có đủ thông tin để quyết định không?” Nếu câu trả lời là “còn tùy vào việc trước đó tôi đã chọn gì” thì trạng thái thiếu đúng cái đó, và cái đó phải trở thành một chiều mới.

Cách thứ hai — tìm hai lịch sử cùng trạng thái nhưng khác tương lai. Nếu dựng được hai cách đi tới cùng $f[i]$ mà phần tối ưu còn lại khác nhau thì trạng thái chắc chắn thiếu. Đây là phản ví dụ, và như chương A/04 đã nói, một phản ví dụ đủ để bác bỏ.

Cách thứ ba — kiểm tra cắt-dán. Nếu bạn không chứng minh được rằng thay một phần lời giải bằng lời giải tối ưu của bài con thì kết quả không tệ đi, thì cấu trúc con tối ưu không có — và chỗ chứng minh hỏng chỉ ra chiều còn thiếu. Ví dụ kinh điển: đường đi *dài nhất* không lập đỉnh trong đồ thị tổng quát; ghép hai đường dài nhất có thể trùng đỉnh, nên trạng thái phải mang theo tập đỉnh đã đi — và đó là lý do bài này chỉ giải được bằng mặt nạ bit với n nhỏ.

CẦN NHỚ

Ba câu hỏi để kiểm trạng thái trước khi cài: (i) tôi phát biểu được $f[\cdot]$ thành một câu đầy đủ không; (ii) từ trạng thái, tôi quyết định được bước tiếp theo mà không cần nhìn lại quá khứ chứ; (iii) hai lịch sử khác nhau tới cùng trạng thái có thật sự cho cùng tương lai không. Trả lời được cả ba thì phần còn lại là kỹ thuật.

4.3 Sáu họ trạng thái hay gặp

Thay vì học từng bài, hãy học sáu họ này — phần lớn bài quy hoạch động trong thi đấu và phỏng vấn là biến thể của chúng.

Họ 1 — một chiều theo vị trí. $f[i]$ là đáp án tối ưu xét tới phần tử i , thường kèm điều kiện “có dùng phần tử i ”. Đại diện: dây con tăng dài nhất, chia dây thành các đoạn, bài trộm nhà không lấy hai nhà liền kề. Dấu hiệu: quyết định tại mỗi vị trí chỉ phụ thuộc vài vị trí gần trước đó.

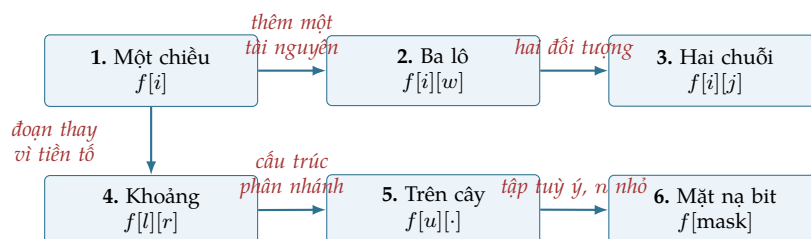
Họ 2 — ba lô: vị trí kèm một tài nguyên. $f[i][w]$ với w là lượng tài nguyên đã dùng. Đại diện: ba lô 0/1, đổi tiền, chia tập thành hai phần cân bằng. Dấu hiệu: có một đại lượng tích lũy bị giới hạn.

Họ 3 — hai chuỗi. $f[i][j]$ xét i ký tự đầu của chuỗi này và j ký tự đầu của chuỗi kia. Đại diện: dây con chung dài nhất, khoảng cách chỉnh sửa, so khớp có ký tự đại diện. Dấu hiệu: bài toán so hai dãy với nhau.

Họ 4 — khoảng. $f[l][r]$ là đáp án cho đoạn $[l, r]$, tính theo độ dài tăng dần. Đại diện: nhân dãy ma trận, cắt gậy, chuỗi đối xứng, trò chơi lấy đồ ở hai đầu. Dấu hiệu: bài toán có cấu trúc lồng nhau, hoặc quyết định chia đoạn tại một điểm giữa.

Họ 5 — trên cây. $f[u][\cdot]$ là đáp án cho cây con gốc u ; tính từ lá lên. Đại diện: tập độc lập lớn nhất trên cây, phủ đỉnh, đường kính, đếm cách tô màu. Dấu hiệu: dữ liệu là cây, và điều kiện chỉ ràng buộc giữa cha và con.

Họ 6 — mặt nạ bit. $f[\text{mask}][\cdot]$ với mask mô tả tập đã dùng. Đại diện: người bán hàng, ghép cặp hoàn hảo, phủ hình. Dấu hiệu: $n \leq 20$ và bài yêu cầu “mỗi phần tử dùng đúng một lần”.



Hình 4.2. Sáu họ trạng thái, xếp theo mức độ thông tin phải mang theo. Đi từ trái sang phải và từ trên xuống dưới, trạng thái ngày càng nặng và số trạng thái ngày càng lớn. Nguyên tắc chọn: dùng họ *nhẹ nhất* còn đủ mô tả tương lai — cùng nguyên tắc “dùng cấu trúc yếu nhất còn đủ dùng” của phần B.

Ngoài sáu họ trên còn ba họ chuyên hơn, nên biết là có: *quy hoạch động trên chữ số* cho các bài đếm số trong khoảng thoả điều kiện; *quy hoạch động kỳ vọng* cho các bài xác suất, ở đó thứ tự tính đôi khi phải đi ngược hoặc phải giải hệ phương trình; và *quy hoạch động trò chơi*, ở đó giá trị của trạng thái được định nghĩa qua cực đại và cực tiểu xen kẽ.

4.4 Ba lô và câu hỏi “vì sao vẫn NP-khó”

Bài ba lô 0/1: n món, món i nặng w_i và có giá trị v_i ; chọn tập con nặng không quá W sao cho tổng giá trị lớn nhất. Trạng thái $f[i][w]$ = giá trị lớn nhất khi xét i món đầu và sức chứa còn w . Chuyển: lấy hoặc không lấy món i . Chi phí $O(nW)$.

Đây là chỗ nên dừng lại vì nó dạy một điều quan trọng về độ phức tạp. $O(nW)$ nhìn thì đa thức, nhưng W được ghi trong dữ liệu vào bằng $\log W$ bit. Nếu $W = 10^{18}$ thì kích thước dữ liệu vào chỉ tăng thêm vài chục bit trong khi chi phí tăng lên 10^{18} . Nói cách khác, thuật toán đa thức theo *giá trị* nhưng mũ theo *kích thước biểu diễn* — người ta gọi đó là **giả đa thức** (*pseudo-polynomial*). Bài ba lô vẫn là NP-khó, và điều đó hoàn toàn nhất quán với sự tồn tại của lời giải $O(nW)$. Chi tiết ở chương D/23.

Hệ quả thực hành: khi thấy trạng thái chứa một *giá trị* thay vì một chỉ số, hãy kiểm tra ngay giá trị đó lớn tới đâu. Nếu W lớn nhưng tổng giá trị nhỏ thì hãy đảo vai trò — đặt trạng thái theo giá trị và tối thiểu hoá khối lượng. Mẹo “đảo vai trò của mục tiêu và ràng buộc” này giải được rất nhiều bài trông bề bét.

4.5 Từ trên xuống hay từ dưới lên

Hai cách viết cho cùng kết quả, nhưng khác nhau ở bốn điểm thực dụng.

Ghi nhớ (từ trên xuống) gần với cách bạn nghĩ, nên viết nhanh và ít sai logic hơn; nó chỉ tính những trạng thái thực sự cần, điều rất quý khi không gian trạng thái thừa; và nó không đòi bạn nghĩ ra thứ tự tính. Nhược điểm: chi phí gọi hàm, nguy cơ tràn ngăn xếp khi độ sâu lớn, và khó áp dụng các tối ưu bộ nhớ.

Điền bảng (từ dưới lên) nhanh hơn theo hàng số, không tràn ngăn xếp, và cho phép giảm bộ nhớ bằng mảng cuộn. Nhược điểm: bạn phải tự xác định thứ tự tính đúng, và nó tính cả những trạng thái không cần.

Quy tắc thực dụng: nghĩ bằng ghi nhớ, và chuyển sang điền bảng khi cần tốc độ hoặc cần giảm bộ nhớ. Trong kỳ thi, nếu không gian trạng thái dày và thứ tự rõ ràng thì viết thẳng bảng.

Giảm bộ nhớ. Nếu tầng i chỉ phụ thuộc tầng $i - 1$ thì chỉ giữ hai tầng — hoặc thậm chí một tầng nếu duyệt đúng chiều, như trong ba lô 0/1 khi duyệt w giảm dần. Kỹ thuật này biến $O(nW)$ bộ nhớ thành $O(W)$ và thường là thứ quyết định bài có chạy nổi hay không. Cái giá: bạn mất khả năng truy vết lời giải cụ thể, vì các tầng cũ đã bị ghi đè. Khi cần cả hai, dùng kỹ thuật chia để trị của Hirschberg cho họ hai chuỗi, hoặc lưu quyết định dưới dạng bit.

4.6 Bốn kỹ thuật tăng tốc

Chi phí quy hoạch động bằng *số trạng thái nhân chi phí mỗi chuyển*. Khi số trạng thái đã tối thiểu mà vẫn chậm, phải tấn công thừa số thứ hai. Bốn kỹ thuật, xếp theo mức độ hay gặp:

Hàng đợi đơn điệu. Khi chuyển có dạng “lấy cực trị của $f[j]$ với j trong một cửa sổ trượt”, dùng deque đơn điệu (B/07) để mỗi trạng thái tốn $O(1)$ khấu hao thay vì $O(k)$.

Cấu trúc dữ liệu cho truy vấn đoạn. Khi chuyển lấy cực trị hoặc tổng trên một đoạn tùy ý, dùng cây phân đoạn hoặc Fenwick (B/10) để giảm từ $O(n)$ xuống $O(\log n)$. Đây là cách tăng tốc dãy con tăng dài nhất từ $O(n^2)$ xuống $O(n \log n)$.

Bao lồi. Khi chuyển có dạng $f[i] = \min_j (f[j] + a_j \cdot b_i)$ — tức là mỗi j đóng góp một đường thẳng theo biến b_i — thì việc lấy cực tiểu trên tập đường thẳng chính là bài bao lồi, và có cấu trúc trả lời trong $O(\log n)$ hoặc $O(1)$ khi các hệ số đơn điệu.

Tối ưu chia để trị và tối ưu Knuth. Khi điểm chuyển tối ưu đơn điệu theo trạng thái — tức là tối ưu của trạng thái sau không bao giờ nằm bên trái tối ưu của trạng thái trước — ta tính các trạng thái theo kiểu chia để trị và giảm một thừa số n . Điều kiện đơn điệu này thường phải chứng minh hoặc kiểm bằng thực nghiệm.

Có một hướng thứ năm ít được nhắc nhưng rất mạnh: *giảm số trạng thái bằng một quan sát tham lam*. Nếu chứng minh được rằng một lớp lựa chọn không bao giờ tối ưu, hãy loại chúng khỏi không gian trạng thái. Đây là chỗ chương C/15 và chương này gặp nhau, và nó thường hiệu quả hơn mọi tối ưu kỹ thuật.

4.7 Vì sao điểm tối ưu lại đơn điệu: bất đẳng thức tứ giác

Kỹ thuật thứ tư ở mục trên có một câu bỏ lửng: “điều kiện đơn điệu này thường phải chứng minh hoặc kiểm bằng thực nghiệm”. Có một điều kiện đủ, phát biểu được trong một dòng, và biết nó thì bạn không còn phải đoán.

Xét lớp truy hồi rất hay gặp ở họ khoảng:

$$f(i, j) = w(i, j) + \min_{i < k < j} (f(i, k) + f(k, j)),$$

với w là chi phí gộp. Nói w thoả **bất đẳng thức tứ giác** (*quadrangle inequality*) nghĩa là với mọi $a \leq b \leq c \leq d$,

$$w(a, c) + w(b, d) \leq w(a, d) + w(b, c).$$

Đọc bằng lời: *hai đoạn lồng nhau không đắt hơn hai đoạn cắt nhau*. Đây đúng là hình ảnh của lập luận đổi chỗ ở chương A/04 — nếu hai lựa chọn tối ưu “bắt chéo” nhau, ta gỡ chéo được mà không làm tệ đi — và chính nó kéo theo tính đơn điệu của điểm chia tối ưu.

Lịch sử của điều kiện này minh hoạ cách một quan sát riêng lẻ trở thành công cụ chung. Knuth[10] nhận ra tính đơn điệu ấy khi tính cây tìm kiếm nhị phân tối ưu và nhờ đó hạ $O(n^3)$ xuống $O(n^2)$; hơn mười năm sau, Yao[11] tách bất đẳng thức tứ giác ra khỏi bài toán cụ thể và chứng minh nó là điều kiện đủ cho cả lớp truy hồi trên. Từ chỗ “một mẹo cho một bài”, nó thành “một định lý cho một họ bài” — và đó là lý do đáng học nó ở dạng tổng quát thay vì học thuộc bài cây tìm kiếm.

Đi xa hơn một bước là thuật toán SMAWK[12], đặt theo chữ đầu của năm tác giả. Khi ma trận chi phí *đơn điệu toàn phần* — một điều kiện họ hàng với bất đẳng thức tứ giác — thì cực tiểu của mọi hàng tìm được trong thời gian *tuyến tính* theo kích thước ma trận, không có cả thừa số lôgarit. Chi tiết đáng chú ý: bài báo này ra đời trong hình học tính toán, cho bài tìm đỉnh xa nhất từ mỗi đỉnh của một đa giác lồi, rồi mới di cư sang quy hoạch động. Lại một lần nữa, cùng cấu trúc toán học xuất hiện ở hai miền không liên quan — đúng điều mà mục liên hệ ngang của mọi chương trong quyển này cố chỉ ra.

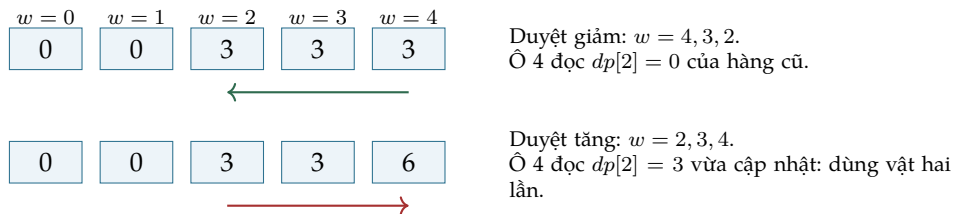
Còn một mảnh nữa hoàn thiện bức tranh, và nó trả lời câu hỏi “đã nén bộ nhớ thì truy vết thế nào” đã hẹn ở mục 5. Hirschberg[13] lấy họ hai chuỗi làm ví dụ: tính bảng từ trái sang tới cột

giữa và từ phải sang tới cột giữa, cộng hai kết quả lại thì biết lời giải tối ưu đi qua ô nào của cột giữa; cắt bài toán tại ô đó và đệ quy trên hai nửa. Tổng chi phí vẫn $O(nm)$ vì mỗi tầng đệ quy làm việc trên một nửa diện tích, còn bộ nhớ chỉ $O(\min(n, m))$. Ý tưởng “không lưu vết, mà tính lại có tổ chức” này dùng được cho nhiều họ trạng thái khác, không riêng hai chuỗi.

Lời khuyên thực hành khép lại mục: trước khi tin vào tính đơn điệu, hãy *kiểm* nó — viết một đoạn vết cận nhỏ tính điểm chia tối ưu cho mọi (i, j) với n cỡ vài chục và kiểm tra trực tiếp bất đẳng thức tứ giác trên hàm w của bạn. Năm phút đó rẻ hơn nhiều so với việc gỡ một lời giải sai chỉ trên vài phần trăm bộ test (E/25).

4.8 Cùng một công thức, đổi chiều duyệt là đổi bài toán

Xét một vật duy nhất có trọng lượng 2, giá trị 3 và sức chứa 4. Với trạng thái “giá trị tốt nhất khi sức chứa không quá w ”, khởi tạo mọi ô bằng 0. Công thức nén là $dp[w] \leftarrow \max(dp[w], dp[w-2]+3)$. Hình 4.3 cho thấy chiều duyệt quyết định ô bên phải đọc giá trị của hàng cũ hay hàng mới.



Hình 4.3. Hai mảng sau khi xử lý cùng một vật. Hàng trên đúng cho ba lô 0/1; hàng dưới đúng cho ba lô không giới hạn. Giá trị 6 không phải lỗi cộng mà là lỗi phụ thuộc nếu đề chỉ cho lấy vật một lần.

Bảng 4.1. Ba dạng ba lô: cùng tài nguyên nhưng khác tập quyết định hợp lệ. Giả sử trọng lượng nguyên dương.

Dạng	Giới hạn mỗi vật	Cách duyệt / xử lý	Test phân biệt
0/1	Tối đa một lần	Duyệt sức chứa giảm, $O(nW)$	Một vật, $W = 2w_i$: không được lấy hai lần
Không giới hạn	Bao nhiêu lần cũng được	Duyệt sức chứa tăng, $O(nW)$	Một vật, $W = 2w_i$: được lấy hai lần
Có số lượng	Tối đa c_i lần	Tách nhị phân thành vật 0/1, $O(W \sum_i \log(c_i + 1))$	Sức chứa đủ cho $c_i + 1$ vật: vẫn chỉ lấy c_i

Bảng 4.1 giúp chọn biến thể trước khi nén bộ nhớ. Nếu trạng thái yêu cầu trọng lượng *đúng bằng* w , chỉ $dp[0] = 0$, các ô còn lại là không đạt được ($-\infty$ trong bài cực đại), và không cộng vào trạng thái không đạt được. Khi chưa chắc, giữ bảng hai chiều làm bản đối chiếu; chỉ nén sau khi xác định rõ mỗi phép đọc thuộc hàng nào.

4.9 Gốc rễ: một cái tên được chọn để nguy trang

Quy hoạch động do Richard Bellman phát triển tại RAND Corporation trong thập niên 1950, và câu chuyện về cái tên của nó đáng kể lại vì nó nói nhiều về bối cảnh nghiên cứu thời đó. Trong hồi ký của mình, Bellman kể rằng ông làm việc dưới sự tài trợ của Không quân Mỹ, và vị bộ trưởng quốc phòng khi đó có ác cảm với từ “nghiên cứu” và đặc biệt với “toán học”. Ông cần một cái tên nghe có vẻ thực dụng và không thể phản đối. “Dynamic” nghe tích cực và có nghĩa thời gian; “programming” thời đó mang nghĩa lập kế hoạch, như trong “quy hoạch tuyến tính”.

Kết quả là một cái tên gần như không mô tả gì về kỹ thuật — điều gây khó khăn cho người học suốt bảy chục năm sau.

Nội dung thật sự thì nghiêm túc hơn cái tên nhiều. Bellman đưa ra *nguyên lý tối ưu*: một chính sách tối ưu có tính chất là, dù trạng thái ban đầu và quyết định đầu tiên là gì, các quyết định còn lại phải tạo thành chính sách tối ưu cho trạng thái sinh ra bởi quyết định đầu. Phát biểu đó chính là cấu trúc con tối ưu, và phương trình Bellman đi kèm là công thức chuyển ở dạng tổng quát nhất. Điều đáng nói là nó ra đời cho các bài toán *điều khiển liên tục theo thời gian* — quản lý tồn kho, điều khiển tên lửa — chứ không phải cho bài tập tổ hợp; các ứng dụng rời rạc là hệ quả về sau.

Ba nhánh ứng dụng lớn ra đời gần như đồng thời trong thập niên 1960–70, mỗi nhánh từ một ngành khác nhau, và cả ba đều là quy hoạch động dù người phát minh không luôn dùng từ đó. Trong *viễn thông*, Viterbi công bố năm 1967 thuật toán giải mã cho mã chập, về sau trở thành thuật toán chuẩn để tìm chuỗi trạng thái khả dĩ nhất của mô hình Markov ẩn — nền của nhận dạng tiếng nói suốt nhiều thập kỷ. Trong *tin sinh học*, Needleman và Wunsch công bố năm 1970 thuật toán căn chỉnh hai chuỗi protein — chính là bài khoảng cách chỉnh sửa với ma trận điểm số sinh học. Trong *ngôn ngữ học tính toán*, thuật toán phân tích cú pháp CYK dùng quy hoạch động trên khoảng để phân tích câu.

Sự trùng hợp này không phải ngẫu nhiên. Cả ba bài toán đều có cùng cấu trúc: một chuỗi quyết định, mỗi quyết định phụ thuộc trạng thái tóm tắt được, và mục tiêu cộng dồn. Bất cứ khi nào ba điều kiện đó xuất hiện — ở bất kỳ ngành nào — quy hoạch động sẽ được phát minh lại.

■ Gốc rễ: ý tưởng này ra đời từ bài toán nào

Mốc	Bài toán thúc ép lúc đó	Điều thay đổi sau đó
Thập niên 1950 — Bellman (RAND)	Điều khiển tối ưu nhiều giai đoạn: tồn kho, phân bổ tài nguyên, dẫn đường.	Nguyên lý tối ưu và phương trình Bellman; cái tên “dynamic programming” được chọn để tránh bị cắt tài trợ vì nghe giống “toán học”.
1962 — Held & Karp	Bài người bán hàng với n vừa phải.	Quy hoạch động trên mặt nạ bit, $O(2^n n^2)$ — vẫn là thuật toán chính xác tốt nhất đã biết cho bài này.
1965–70 — Levenshtein; Needleman & Wunsch	So sánh chuỗi: mã sửa lỗi, rồi căn chỉnh chuỗi protein.	Khoảng cách chỉnh sửa; nền của cả kiểm tra chính tả lẫn tin sinh học hiện đại.
1967 — Viterbi	Giải mã mã chập trong truyền tin có nhiễu.	Thuật toán Viterbi; về sau thành công cụ chuẩn cho mô hình Markov ẩn trong nhận dạng tiếng nói và xử lý tín hiệu.
Thập niên 1960 — CYK	Phân tích cú pháp câu theo văn phạm phi ngữ cảnh.	Quy hoạch động trên khoảng; cùng khuôn với bài nhân dãy ma trận.
Thập niên 1980–nay — học tăng cường	Ra quyết định tuần tự khi mô hình môi trường không biết trước.	Phương trình Bellman trở thành nền của học tăng cường; “hàm giá trị” chính là bảng quy hoạch động, chỉ khác là nó được <i>học</i> thay vì được điền.

■ 4.10 Liên hệ ngang

■ Liên hệ ngang: ý này còn sống ở đâu nữa

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu
Học tăng cường	Phương trình Bellman, hàm giá trị, lập giá trị	Cùng một phương trình. Khác: ở đây bảng được điền bằng cách duyệt hết trạng thái, ở đó hàm giá trị được <i>xấp xỉ</i> bằng mạng nơ-ron vì không gian trạng thái quá lớn để liệt kê.
Lý thuyết điều khiển	Điều khiển tối ưu, bộ điều khiển toàn phương tuyến tính	Bản liên tục của cùng nguyên lý; “trạng thái” ở đây đúng nghĩa trạng thái vật lý. Điều kiện Markov trong điều khiển tương ứng chính xác với yêu cầu “trạng thái phải đủ” ở mục 2.
Tin sinh học	Căn chỉnh chuỗi, gấp RNA	Khoảng cách chỉnh sửa với ma trận điểm số sinh học; bài gấp RNA là quy hoạch động trên khoảng. Đây là nơi quy hoạch động chạy ở quy mô lớn nhất trong khoa học.
Xử lý tiếng nói và tín hiệu	Viterbi trong mô hình Markov ẩn, xoắn thời gian động	Cùng thuật toán, khác cách gọi. Xoắn thời gian động là bài hai chuỗi với chi phí căn chỉnh — dùng để so hai chuỗi tín hiệu có tốc độ khác nhau.
Thị giác máy tính, stereo	So khớp ảnh theo dòng quét bằng quy hoạch động	Bài toán tìm độ lệch cho từng điểm ảnh trên một dòng, với ràng buộc trơn giữa các điểm kề — đúng khuôn họ một chiều với chi phí chuyển. Cùng bài toán về sau được giải bằng cắt đồ thị (C/18), một ví dụ đẹp về hai công cụ cho cùng mô hình.
Tài chính	Định giá quyền chọn kiểu Mỹ bằng cây nhị thức	Đi ngược từ thời điểm đáo hạn về hiện tại, mỗi nút lấy cực đại giữa thực hiện ngay và giữ tiếp — đúng quy hoạch động trò chơi với thứ tự tính ngược thời gian.
Vận trù học	Quản lý tồn kho, lập lịch sản xuất	Bài toán gốc mà Bellman nhắm tới. Ràng buộc thực tế thêm vào là bất định về nhu cầu, dẫn tới quy hoạch động ngẫu nhiên — trạng thái mang theo phân phối thay vì giá trị.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì phần khó không nằm ở kỹ thuật mà nằm ở việc *đặt tên cho đúng thứ*. Đệ quy và bảng là chuyện cơ khí; cái khó là nhìn một bài toán và nhận ra “mọi lịch sử dẫn tới đây đều tương đương với nhau nếu ta chỉ nhớ hai con số này”. Đó là một bước trừu tượng hoá, và nó không có quy trình máy móc. Thêm nữa, cách dạy phổ biến — trình bày công thức đã hoàn chỉnh — che mất chính bước này, khiến người học thấy lời giải hiển nhiên mà không tái tạo được.
- A2. Vì trạng thái thiếu vẫn cho code chạy và vẫn ra số, chỉ là số sai — một lỗi im lặng. Ba cách kiểm trước khi cài: thử kể lại tương lai từ trạng thái và xem có phải nói “còn tùy vào trước đó” không; tìm hai lịch sử cùng trạng thái nhưng khác tương lai tối ưu; và kiểm lập luận cắt-dán, vì chỗ nó hỏng chỉ ra đúng chiều còn thiếu. Sau khi cài, cách kiểm rẻ nhất vẫn là stress test với vét cạn trên n nhỏ.
- A3. Vì W xuất hiện trong chi phí như một *giá trị*, trong khi kích thước dữ liệu vào chỉ chứa $\log W$ bit để biểu diễn nó. Với $W = 10^{18}$, dữ liệu vào vẫn ngắn nhưng thuật toán cần 10^{18} bước — tức là mũ theo kích thước biểu diễn. Loại thuật toán này gọi là giả đa thức, và sự tồn tại của nó hoàn toàn nhất quán với việc bài toán là NP-khó (D/23). Hệ quả thực hành: khi trạng thái chứa một giá trị chứ không phải chỉ số, hãy kiểm tra ngay biên của giá trị đó.
- A4. Chọn ghi nhớ khi không gian trạng thái *thưa* — chỉ một phần nhỏ trạng thái thực sự được dùng — hoặc khi thứ tự tính khá xác định, hoặc khi bạn đang trong giai đoạn nghĩ và muốn code gần với suy nghĩ. Chọn điền bảng khi cần tốc độ (không có chi phí gọi hàm), khi độ sâu đệ quy có thể gây tràn ngăn xếp, hoặc khi cần giảm bộ nhớ bằng mảng cuộn — kỹ thuật gần như không áp được cho bản ghi nhớ.
- A5. Còn ba hướng. Thứ nhất, tấn công chi phí mỗi chuyển bằng cấu trúc dữ liệu hoặc bằng các tối ưu ở mục 6: hàng đợi đơn điệu, cây phân đoạn, bao lồi, tính đơn điệu của điểm tối ưu. Thứ hai, giám sát trạng thái bằng một quan sát tham lam — chứng minh rằng cả một lớp lựa chọn không

bao giờ tối ưu và loại chúng; hướng này thường hiệu quả nhất. Thứ ba, đổi vai trò các chiều, ví dụ đặt trạng thái theo giá trị thay vì theo khối lượng khi giá trị nhỏ hơn nhiều.

NÓI LẠI THẬT ĐƠN GIẢN

Quy hoạch động chỉ là hai chuyện gộp lại: thử mọi khả năng, và *ghi lại những gì đã tính để khỏi tính lại*.

Hãy nghĩ tới việc leo cầu thang mỗi bước một hoặc hai bậc, và bạn muốn đếm số cách leo lên bậc thứ mười. Nếu ngồi vẽ cây mọi khả năng thì rất nhiều nhánh trùng nhau: dù bạn tới bậc thứ bảy bằng đường nào, số cách đi tiếp từ đó vẫn y hệt. Vậy thay vì vẽ lại, hãy ghi “từ bậc bảy còn bao nhiêu cách” vào một tờ giấy và dùng lại. Số việc từ chỗ tăng gấp đôi mỗi bậc trở thành mỗi bậc một dòng ghi chép.

Câu quan trọng nhất trong chuyện trên là câu “dù bạn tới bậc thứ bảy bằng đường nào”. Đó chính là điều kiện để cách làm này chạy được: những gì đã xảy ra không còn ảnh hưởng gì tới tương lai, ngoài việc bạn đang đứng ở bậc nào. Nếu bài toán có thêm luật kiểu “không được đi hai bước dài liên tiếp” thì đứng ở bậc bảy chưa đủ — bạn còn phải nhớ bước vừa rồi dài hay ngắn. Lúc đó tờ giấy ghi chép phải có hai cột thay vì một.

Toàn bộ khó khăn của quy hoạch động nằm ở đúng chỗ đó: quyết định *tờ giấy cần ghi những gì*. Ghi thiếu thì kết quả sai một cách im lặng; ghi thừa thì tờ giấy to quá và chương trình chạy chậm. Mọi kỹ thuật còn lại trong chương — cách điền bảng, cách tiết kiệm giấy, cách tăng tốc — đều là chuyện phụ so với câu hỏi này.

Một mẹo kiểm tra rất hiệu quả: hãy tự kể lại phần còn lại của bài toán chỉ dựa vào những gì đã ghi. Nếu bạn thấy mình phải nói “còn tùy trước đó tôi làm gì”, thì đúng cái “tùy” đó là thứ còn thiếu.

Một câu: *Trạng thái phải đủ: biết trạng thái thì tương lai không cần biết gì thêm về quá khứ.*

Chỗ không rút gọn được: Việc nghĩ ra trạng thái đúng cho một bài mới vẫn là bước sáng tạo, không có quy trình máy móc — chỉ có các cách kiểm tra sau khi đã đoán.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Quy hoạch động là gì	Vết cặn cộng trí nhớ: các nhánh khác nhau gộp lại cùng bài toán con, nên lưu kết quả để khỏi tính lại. Chỉ phí bằng số trạng thái nhân chi phí mỗi chuyển.
••• Quy trình năm bước	Trạng thái → công thức chuyển → cơ sở → thử tự tính → đáp án và truy vết. Bước một chiếm chín phần mười độ khó.
••• Tiêu chuẩn của trạng thái tốt	Biết trạng thái thì tương lai hoàn toàn xác định, không cần biết cách tới đó. Đây là tính chất Markov, cũng là một bất biến.
••• Ba cách phát hiện trạng thái thiếu	Kể lại tương lai và xem có phải nói “còn tùy”; tìm hai lịch sử cùng trạng thái khác tương lai; kiểm lập luận cắt-dán và xem nó hỏng ở đâu.
••• Sáu họ trạng thái	Một chiều $f[i]$; ba lô $f[i][w]$; hai chuỗi $f[i][j]$; khoảng $f[l][r]$; trên cây $f[u][v]$; mặt nạ bit $f[mask]$.
••• Vì sao ba lô vẫn NP-khó	$O(nW)$ là <i>giả đa thức</i> : đa thức theo giá trị W nhưng mũ theo số bit biểu diễn W . Dấu hiệu: trạng thái chứa một giá trị chứ không phải chỉ số.
•• Ghi nhớ so với điền bảng	Ghi nhớ: gần cách nghĩ, chỉ tính trạng thái cần, không phải tìm thứ tự. Điền bảng: nhanh hơn, không tràn ngăn xếp, cho phép mảng cuộn.
•• Mảng cuộn	Giữ hai tầng (hoặc một, nếu duyệt đúng chiều) khi tầng i chỉ phụ thuộc tầng $i - 1$; cái giá là mất khả năng truy vết.
•• Bốn kỹ thuật tăng tốc chuyển	Deque đơn điệu (cửa sổ trượt); cây phân đoạn/Fenwick (đoạn tùy ý); bao lồi (chuyển dạng đường thẳng); tính đơn điệu điểm tối ưu (chia để trị, Knuth).

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●● Hường tăng tốc mạnh nhất	Giảm <i>số trạng thái</i> bằng một quan sát tham lam: chứng minh cả một lớp lựa chọn không bao giờ tối ưu rồi loại.
●● Mẹo đảo vai trò	Khi W quá lớn nhưng tổng giá trị nhỏ, đặt trạng thái theo giá trị và tối thiểu hoá khối lượng.
●● Ba họ chuyên hơn	Quy hoạch động trên chữ số (đếm số thoả điều kiện); kỳ vọng (có thể phải giải hệ); trò chơi (cực đại và cực tiểu xen kẽ).
● Truy vết lời giải	Lưu quyết định tại mỗi trạng thái, hoặc dò ngược bằng so sánh giá trị; nếu đã dùng mảng cuộn thì cần kỹ thuật chia để trị kiểu Hirschberg.
●● Bất đẳng thức tứ giác	$w(a, c) + w(b, d) \leq w(a, d) + w(b, c)$ với $a \leq b \leq c \leq d$ — “lồng nhau không đắt hơn cắt nhau” — kéo theo điểm chia tối ưu đơn điệu, tức giảm được một thừa số n . Kiểm bằng vết cạnh nhỏ trước khi tin.

●●● phải thuộc ●● nên nắm ● biết là có

I Câu hỏi ôn tập có đáp án (che phần đáp án, tự trả lời trước)

1. Tôi luôn bí ở bước đặt trạng thái. Có quy trình nào không?

| Có một quy trình bốn bước dùng được cho phần lớn bài. Một, viết lời giải vết cạnh đệ quy *trước*, với các tham số tự nhiên. Hai, nhìn danh sách tham số của hàm đệ quy đó — chứng chính là ứng viên trạng thái. Ba, loại các tham số suy ra được từ những tham số khác (ví dụ nếu biết i thì biết luôn tổng đã dùng thì bỏ tham số tổng). Bốn, kiểm tra tính đủ bằng ba câu hỏi ở khối *Cần nhớ*. Quy trình này biến việc “nghĩ ra trạng thái” thành việc “viết vết cạnh rồi rút gọn tham số”, dễ hơn nhiều.

2. Làm sao biết bài này nên dùng tham lam hay quy hoạch động?

| Thử tham lam trước vì nó rẻ hơn nhiều: nghĩ một tiêu chí, và stress test với vết cạnh trên n nhỏ trong năm phút. Nếu tìm được phản ví dụ thì chuyển sang quy hoạch động, và điều đáng chú ý là chính phản ví dụ đó thường chỉ ra chiều trạng thái cần thêm — nó cho thấy quyết định tối ưu phụ thuộc vào cái gì. Đây là một trong những cách hiệu quả nhất để “đọc ra” trạng thái từ bài toán.

3. Số trạng thái của tôi là 10^7 và mỗi chuyển $O(1)$. Có chạy nổi không?

| Theo bảng ở A/03 thì 10^7 phép toán đơn giản là thoải mái, nhưng hãy kiểm hai điều mà bảng không nói. Thứ nhất là bộ nhớ: 10^7 số nguyên 32 bit là 40 MB, thường vẫn được, nhưng 10^7 số 64 bit là 80 MB và có thể vượt giới hạn. Thứ hai là kiểu truy cập: nếu bạn duyệt bảng hai chiều theo cột thay vì theo hàng thì mỗi lần truy cập là một lần trượt cache và chương trình có thể chậm gấp năm lần. Đổi thứ tự vòng lặp là tối ưu rẻ nhất và hay bị bỏ qua nhất.

4. Quy hoạch động của tôi có chu trình phụ thuộc — trạng thái A cần B mà B lại cần A. Làm sao?

| Đó là dấu hiệu bài toán không còn là quy hoạch động thuần túy. Ba hướng xử lý. Nếu các chuyển có “chi phí không âm” và bạn tìm cực tiểu, hãy dùng Dijkstra trên đồ thị trạng thái (C/17) — thứ tự tính được xác định động thay vì tĩnh. Nếu là bài kỳ vọng có chu trình, hãy lập và giải hệ phương trình tuyến tính. Nếu chu trình đến từ việc trạng thái thiếu chiều, hãy thêm chiều để phá chu trình — ví dụ thêm chiều “số bước đã đi”.

5. Tôi cần in ra lời giải cụ thể chứ không chỉ giá trị tối ưu. Làm thế nào?

| Hai cách. Cách đơn giản: lưu thêm mảng `choice[state]` ghi quyết định đã chọn tại mỗi trạng thái, rồi đi ngược từ trạng thái đáp án. Cách tiết kiệm bộ nhớ: không lưu gì, và khi truy vết thì tính lại — tại mỗi trạng thái, thử từng quyết định và xem quyết định nào cho ra đúng giá trị đã lưu. Nếu bạn đã dùng mảng cuộn thì không truy vết trực tiếp được, và cách chuẩn cho họ hai chuỗi là kỹ thuật chia đôi của Hirschberg, cho $O(nm)$ thời gian với $O(\min(n, m))$ bộ nhớ.

6. Vì sao dãy con tăng dài nhất lại có lời giải $O(n \log n)$ trông chẳng giống quy hoạch động?

| Vì nó là quy hoạch động đã được biến đổi. Bản $O(n^2)$ có trạng thái “độ dài dãy tăng dài nhất kết thúc tại i ”. Bản $O(n \log n)$ đảo lại: với mỗi độ dài, giữ *giá trị kết thúc nhỏ nhất* có thể. Mảng các giá trị đó luôn tăng, nên cập nhật được bằng tìm kiếm nhị phân. Đây là ví dụ điển hình của mẹo đảo vai trò ở mục 4 — đổi chỗ giữa cái được tối ưu và cái làm chỉ số — và nó đáng học vì mẹo này áp dụng được cho nhiều bài khác.

7. Quy hoạch động trên cây khác gì so với trên dãy?

| Khác chủ yếu ở thứ tự tính và ở cách gộp. Thứ tự là duyệt sâu, tính cây con trước rồi gộp lên cha — tương đương với việc “từ dưới lên” trên một cấu trúc phân nhánh. Cách gộp thường có dạng “kết hợp kết quả của các con”, và khi có ràng buộc về số lượng thì phép gộp giống bài ba lô trên từng cây con. Một chi tiết kỹ thuật quan trọng: với cây sâu tới 10^5 , đệ quy có thể tràn ngăn xếp, nên hãy dùng bản lặp hoặc tăng giới hạn ngăn xếp (E/25).

8. Có bài nào nhìn không giống quy hoạch động mà thực ra là không?

| Rất nhiều, và nhận ra chúng là dấu hiệu đã nắm phương pháp. Ba ví dụ: bài đếm đường đi trên lưới có vật cản (thực ra là $f[i][j]$ một cách hiển nhiên); bài chia dãy thành k đoạn tối ưu (họ khoảng, nhưng thường phải kèm tối ưu chia để trị); và bài xác suất thắng của một trò chơi (quy hoạch động kỳ vọng, với chú ý về chu trình). Ngoài ra, mọi bài tìm đường đi ngắn nhất trên đồ thị không chu trình đều là quy hoạch động — và ngược lại, mọi quy hoạch động đều là bài đường đi ngắn nhất trên đồ thị trạng thái, một cách nhìn rất đáng có (C/17).

Bài tập (mức: ● khởi động ●● chuẩn ●●● khó)

Bài	Nguồn	Điều cần nhận ra
● Số cách leo cầu thang, số cách đi trên lưới có vật cản	AtCoder DP “Frog 1”; CSES “Grid Paths”[1]	Bài nhập môn; mục tiêu là viết đủ năm bước ra giấy trước khi gõ.
● Đổi tiền: số cách và số đồng ít nhất	CSES “Coin Combinations I/II”[1]	Hai bài khác nhau đúng ở thứ tự hai vòng lặp — hãy giải thích vì sao.
● Dãy con tăng dài nhất	CSES “Increasing Subsequence”[1]	Cài cả $O(n^2)$ và $O(n \log n)$; hiểu mẹo đảo vai trò.
●● Ba lô 0/1 và ba lô không giới hạn	CSES “Book Shop”[1]	Giảm bộ nhớ bằng mảng cuộn; chú ý chiều duyệt w cho hai biến thể.
●● Khoảng cách chỉnh sửa	CSES “Edit Distance”[1]	Họ hai chuỗi; sau đó thử truy vết ra dãy thao tác cụ thể.
●● Dãy con chung dài nhất của hai chuỗi	Bài kinh điển	Cùng họ; so sánh cách truy vết với và không có mảng cuộn.
●● Tập độc lập lớn nhất trên cây	Bài kinh điển	Quy hoạch động trên cây với hai trạng thái mỗi đỉnh; chú ý độ sâu đệ quy.
●● Cắt gậy để tổng chi phí nhỏ nhất	Bài kinh điển	Họ khoảng; nhận ra sự tương đồng với bài nhân dãy ma trận.
●●● Người bán hàng với $n \leq 20$	CSES “Hamiltonian Flights”-loại[1]	Quy hoạch động mặt nạ bit; ước lượng $2^n n^2$ trước khi cài.
●●● Chia dãy thành k đoạn sao cho tổng chi phí nhỏ nhất	Bài thi đấu kinh điển	Bản $O(kn^2)$ trước, rồi thêm tối ưu chia để trị; kiểm tính đơn điệu điểm tối ưu bằng thực nghiệm.
●●● Đếm số nguyên trong $[a, b]$ thoả một điều kiện về chữ số	Bài kinh điển	Quy hoạch động trên chữ số với cờ “đang bám biên”; đây là họ có nhiều trường hợp biên nhất.
●●● Xác suất thắng của một trò chơi có chu trình trạng thái	Bài kinh điển	Lập hệ phương trình tuyến tính; ví dụ cho thấy ranh giới của quy hoạch động thuần túy.
●●● Ghép cặp hoàn hảo trọng số nhỏ nhất với $n \leq 20$	Bài kinh điển	Mặt nạ bit; so với thuật toán Hungarian ở C/18 về cả độ phức tạp lẫn độ dài code.
●●● Bài toán ba lô với W tới 10^{18} nhưng tổng giá trị nhỏ	Bài kinh điển	Đảo vai trò hai chiều; bài mẫu cho mẹo ở mục 4.

Kết chương và đọc tiếp

Quy hoạch động là kỹ thuật có tỉ lệ xuất hiện cao nhất trong mọi kỳ thi và phỏng vấn, và toàn bộ độ khó của nó tập trung vào một câu: trạng thái phải đủ. Ba điều đáng mang theo: quy trình năm bước viết ra thành lời trước khi gõ; ba cách kiểm tra tính đủ của trạng thái; và cách nhìn “chi phí bằng số trạng thái nhân chi phí chuyển”, vì nó chỉ ra ngay phải tấn công thừa số nào khi quá chậm.

Chương tiếp theo đổi ngôn ngữ. Rất nhiều bài toán trông rất khác nhau trở thành cùng một bài khi được phát biểu lại bằng đỉnh và cạnh — và như câu hỏi cuối ở mục hỏi đáp đã gợi ý, quy hoạch động và đường đi ngắn nhất là hai cách nói về cùng một thứ. Đồ thị là chương của kỹ năng *mô hình hoá*, kỹ năng đáng giá hơn việc thuộc thêm mười thuật toán (C/17).

Đọc thêm: công trình gốc (chỉ liệt kê nguồn đã qua phản biện; bản tiền ấn được ghi rõ)

Mức	Công trình và tác giả	Nơi công bố, năm	Vì sao đọc, đọc phần nào
nền tảng	Yao, “Speed-Up in Dynamic Programming”	SIAM Journal on Algebraic and Discrete Methods, 1982	Nguồn của điều kiện tứ giác ở dạng tổng quát. Đọc phần phát biểu định lý và ví dụ; đây là bài nên đọc nếu bạn hay gặp quy hoạch động trên khoảng.
nền tảng	Hirschberg, “A Linear Space Algorithm for Computing Maximal Common Subsequences”	Communications of the ACM, 1975	Ba trang. Đọc để có kỹ thuật truy vết trong bộ nhớ tuyến tính — thứ mà mảng cuộn lấy đi.
tra khi cần	Knuth, “Optimum Binary Search Trees”	Acta Informatica, 1971	Nơi tính đơn điệu của điểm chia xuất hiện lần đầu, trong một bài toán cụ thể.
tra khi cần	Aggarwal, Klawe, Moran, Shor, Wilber, “Geometric Applications of a Matrix-Searching Algorithm”	Algorithmica, 1987	Thuật toán SMAWK. Đọc khi truy hồi của bạn viết được thành “cực tiểu theo hàng của một ma trận đơn điệu toàn phần”.
tra khi cần	Backurs & Indyk, “Edit Distance Cannot Be Computed in Strongly Subquadratic Time”	SIAM Journal on Computing, 2018 (bản hội nghị STOC 2015)	Đọc khi bạn tự hỏi vì sao khoảng cách chỉnh sửa mãi vẫn là $O(nm)$: có lý do, và nó nằm ở D/24.

Nguồn

- [1] A. Laaksonen. “CSES Problem Set.” (2026), address: <https://cses.fi/problemset/> (visited on 09/06/2026).
- [2] M. Blum, R. W. Floyd, V. Pratt, et al., “Time Bounds for Selection,” *Journal of Computer and System Sciences*, vol. 7, no. 4, pp. 448–461, 1973.
- [3] V. Strassen, “Gaussian Elimination Is Not Optimal,” *Numerische Mathematik*, vol. 13, pp. 354–356, 1969.
- [4] D. Harvey and J. van der Hoeven, “Integer Multiplication in Time $O(n \log n)$,” *Annals of Mathematics*, vol. 193, no. 2, pp. 563–617, 2021.
- [5] J. Alman and V. Vassilevska Williams, “A Refined Laser Method and Faster Matrix Multiplication,” in *Proceedings of the 32nd ACM-SIAM Symposium on Discrete Algorithms (SODA)*, 2021.
- [6] J. Alman, R. Duan, V. Vassilevska Williams, et al. “Improving the Matrix Multiplication Exponent with Modern Optimization and AlphaEvolve.” bản tiền ấn arXiv:2608.16884, chưa qua phản biện. (2026), address: <https://arxiv.org/abs/2608.16884> (visited on 09/07/2026).
- [7] J. W. Cooley and J. W. Tukey, “An Algorithm for the Machine Calculation of Complex Fourier Series,” *Mathematics of Computation*, vol. 19, no. 90, pp. 297–301, 1965.
- [8] D. R. Musser, “Introspective Sorting and Selection Algorithms,” *Software: Practice and Experience*, vol. 27, no. 8, pp. 983–993, 1997.
- [9] J. L. Bentley and M. D. McIlroy, “Engineering a Sort Function,” *Software: Practice and Experience*, vol. 23, no. 11, pp. 1249–1265, 1993.
- [10] D. E. Knuth, “Optimum Binary Search Trees,” *Acta Informatica*, vol. 1, pp. 14–25, 1971.
- [11] F. F. Yao, “Speed-Up in Dynamic Programming,” *SIAM Journal on Algebraic and Discrete Methods*, vol. 3, no. 4, pp. 532–540, 1982.
- [12] A. Aggarwal, M. M. Klawe, S. Moran, et al., “Geometric Applications of a Matrix-Searching Algorithm,” *Algorithmica*, vol. 2, pp. 195–208, 1987.
- [13] D. S. Hirschberg, “A Linear Space Algorithm for Computing Maximal Common Subsequences,” *Communications of the ACM*, vol. 18, no. 6, pp. 341–343, 1975.
- [14] R. E. Tarjan, “Depth-First Search and Linear Graph Algorithms,” *SIAM Journal on Computing*, vol. 1, no. 2, pp. 146–160, 1972.
- [15] M. L. Fredman and R. E. Tarjan, “Fibonacci Heaps and Their Uses in Improved Network Optimization Algorithms,” *Journal of the ACM*, vol. 34, no. 3, pp. 596–615, 1987.
- [16] M. Thorup, “Undirected Single-Source Shortest Paths with Positive Integer Weights in Linear Time,” *Journal of the ACM*, vol. 46, no. 3, pp. 362–394, 1999.
- [17] R. Duan, J. Mao, X. Mao, et al., “Breaking the Sorting Barrier for Directed Single-Source Shortest Paths,” in *Proceedings of the 57th Annual ACM Symposium on Theory of Computing (STOC)*, Best Paper Award, 2025.
- [18] E. W. Dijkstra, “A Note on Two Problems in Connexion with Graphs,” *Numerische Mathematik*, vol. 1, pp. 269–271, 1959.

- [19] KTH Royal Institute of Technology. “KACTL: KTH Algorithm Competition Template Library.” (2026), address: <https://github.com/kth-competitive-programming/kactl> (visited on 09/06/2026).
- [20] H. W. Kuhn, “The Hungarian Method for the Assignment Problem,” *Naval Research Logistics Quarterly*, vol. 2, no. 1-2, pp. 83–97, 1955.
- [21] J. B. Orlin, “Max Flows in $O(nm)$ Time, or Better,” in *Proceedings of the 45th Annual ACM Symposium on Theory of Computing (STOC)*, 2013, pp. 765–774.
- [22] L. Chen, R. Kyng, Y. P. Liu, et al., “Maximum Flow and Minimum-Cost Flow in Almost-Linear Time,” in *Proceedings of the 63rd IEEE Symposium on Foundations of Computer Science (FOCS)*, Best Paper Award, 2022.
- [23] J. E. Hopcroft and R. M. Karp, “An $n^{5/2}$ Algorithm for Maximum Matchings in Bipartite Graphs,” *SIAM Journal on Computing*, vol. 2, no. 4, pp. 225–231, 1973.
- [24] J. Bentley, *Programming Pearls*, 2nd ed. Addison-Wesley, 2000.
- [25] R. Motwani and P. Raghavan, *Randomized Algorithms*. Cambridge University Press, 1995.

Đồ thị: mô hình hoá và duyệt (Graphs: Modeling and Traversal)

PHỤ THUỘC B/07 cho ba loại kho chờ — chương này là nơi chúng sinh ra ba thuật toán; B/11 cho DSU; C/15 cho chứng minh tham lam của Dijkstra và Kruskal; C/16 cho quan hệ giữa quy hoạch động và đường đi ngắn nhất.

TÓM TẮT

Kỹ năng quan trọng nhất của chương này không phải thuộc thuật toán mà là *mô hình hoá*: nhận ra rằng bài toán trước mặt, dù nói về gì, thực chất là bài toán về đỉnh và cạnh. Khi làm được điều đó, hàng chục bài trông khác nhau sụp xuống thành vài bài đã có lời giải. Chương đi qua hai phép duyệt nền tảng và những thứ chúng để lộ ra — thứ tự tô-pô, thành phần liên thông mạnh, cầu và khớp — rồi tới bốn thuật toán đường đi ngắn nhất, mỗi cái ứng với một giả thiết khác nhau về trọng số. Phần cuối là các mẹo mô hình hoá: đồ thị trạng thái, đồ thị nhiều tầng, tách đỉnh. Nếu chỉ nhớ một điều: khi bí, hãy hỏi “cái gì là đỉnh, cái gì là cạnh” — câu hỏi đó giải được nhiều bài hơn bất kỳ thuật toán riêng lẻ nào.

KIẾN THỨC CẦN CÓ TRƯỚC

- **đồ thị** — tập đỉnh và tập cạnh nối chúng; có hướng nếu cạnh có chiều, có trọng số nếu cạnh mang một giá trị.
- **danh sách kề** — cách lưu phổ biến nhất: với mỗi đỉnh, một danh sách các đỉnh kề. Bộ nhớ $O(n+m)$, hợp với đồ thị thưa.
- **duyet rộng (BFS)** — duyệt theo lớp khoảng cách bằng hàng đợi; cho đường đi ngắn nhất trên đồ thị không trọng số.
- **duyet sâu (DFS)** — đi sâu hết một nhánh rồi quay lại, dùng ngăn xếp hoặc đệ quy; thời điểm vào/ra của mỗi đỉnh mang thông tin cấu trúc.
- **thứ tự tô-pô** — cách xếp các đỉnh của một đồ thị có hướng không chu trình sao cho mọi cạnh đi từ trái sang phải.
- **thành phần liên thông mạnh** — tập đỉnh mà từ đỉnh nào cũng tới được mọi đỉnh khác trong tập; nén chúng lại cho một đồ thị không chu trình.

CÂU HỎI MỞ ĐẦU

- Q1. Nhiều bài toán không hề nhắc tới “đồ thị” vẫn giải bằng đồ thị. Dấu hiệu nào cho biết nên phát biểu lại bài toán theo đỉnh và cạnh?
- Q2. Duyệt rộng và duyệt sâu chỉ khác nhau ở một dòng code. Vì sao chúng để lộ ra những thông tin hoàn toàn khác nhau?
- Q3. Dijkstra sai với cạnh trọng số âm. Chỗ nào trong lập luận của nó dùng tới giả thiết không âm, và Bellman–Ford làm khác thế nào?
- Q4. Vì sao “đường đi ngắn nhất” và “quy hoạch động” lại là hai cách nói về cùng một thứ?
- Q5. Khi đồ thị có tới 10^5 đỉnh và cần trả lời hàng trăm nghìn truy vấn về tổ tiên chung trên cây, làm sao trả lời mỗi truy vấn trong $O(\log n)$?

Có một khoảnh khắc rất đặc trưng khi luyện thuật toán: bạn đọc một đề bài nói về những người bạn trao đổi quà, hoặc về các công việc phụ thuộc nhau, hoặc về các trạng thái của một trò xếp hình — và đột nhiên nhận ra đó là bài toán tìm chu trình, hoặc sắp thứ tự tô-pô, hoặc đường đi ngắn nhất. Sau khoảnh khắc đó, bài toán gần như đã xong: phần còn lại là gọi một thuật toán bạn đã biết.

Chương này viết cho chính khoảnh khắc đó. Các thuật toán đồ thị đều đã được chuẩn hoá và bạn tra được ở bất cứ đâu; cái không tra được là kỹ năng nhìn ra rằng bài toán của mình *là* một bài đồ thị. Vì vậy tôi sẽ trình bày thuật toán ở mức đủ dùng, và dành trọng lượng cho câu hỏi mô hình hoá — cái gì là đỉnh, cái gì là cạnh, và cạnh mang trọng số gì.

5.1 Mô hình hoá: cái gì là đỉnh, cái gì là cạnh

Ba dấu hiệu cho thấy bài toán nên được phát biểu lại thành đồ thị.

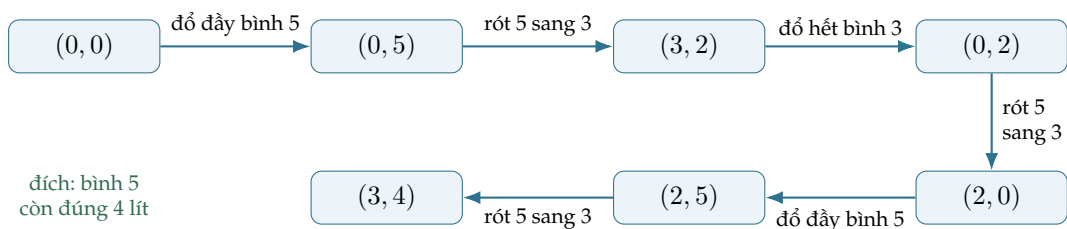
Dấu hiệu thứ nhất — quan hệ đôi một. Đề nói về những cặp có quan hệ với nhau: hai người quen nhau, hai thành phố có đường nối, hai công việc xung khắc. Đỉnh là đối tượng, cạnh là quan hệ. Đây là trường hợp hiển nhiên nhất.

Dấu hiệu thứ hai — phụ thuộc và thứ tự. Đề nói “việc A phải làm trước việc B”. Đỉnh là việc, cạnh có hướng là ràng buộc; câu hỏi “có xếp được thứ tự không” trở thành “đồ thị có chu trình không”.

Dấu hiệu thứ ba — và đây là dấu hiệu đáng giá nhất — trạng thái và bước chuyển. Đề mô tả một hệ có thể ở nhiều tình huống, và có các thao tác đưa từ tình huống này sang tình huống khác. Lúc đó *đỉnh là trạng thái, cạnh là một thao tác hợp lệ*, và câu hỏi “ít thao tác nhất để tới đích” chính là đường đi ngắn nhất.

Dấu hiệu thứ ba mở ra rất nhiều bài mà nhìn bề ngoài không có đồ thị nào: trò xếp hình, các bài đổ nước giữa các bình, bài biến chuỗi này thành chuỗi kia bằng các phép biến đổi, bài robot di chuyển trên lưới có luật. Điều cần cẩn thận là *kích thước không gian trạng thái*: nếu số trạng thái là 10^6 thì duyệt rộng chạy tốt; nếu là 10^{15} thì phải tìm cấu trúc khác hoặc dùng tìm kiếm có định hướng.

Hình 5.1 ghi rõ từng thao tác; kiểm từng cạnh là cách kiểm mô hình trước khi viết BFS.



Hình 5.1. Một đường đi hợp lệ gồm sáu thao tác từ hai bình rỗng tới 4 lít trong bình 5 lít. Trạng thái là (lượng trong bình 3, lượng trong bình 5). Hình chỉ vẽ một đường đi, không phải toàn bộ đồ thị BFS.

CẦN NHỚ

Ba câu hỏi khi nghi ngờ bài là bài đồ thị: (i) đỉnh là gì — đối tượng hay *trạng thái*; (ii) cạnh là gì, và có hướng không; (iii) cạnh có trọng số gì — không trọng số, trọng số 0/1, trọng số dương, hay có thể âm. Câu thứ ba quyết định thuật toán, và bảng ở mục 4 là bản đồ.

5.2 Hai phép duyệt và những gì chúng để lộ

Duyệt rộng và duyệt sâu chỉ khác nhau ở kho chờ — hàng đợi hay ngăn xếp, đúng như chương B/07 đã nói — nhưng thông tin chúng cho lại rất khác.

Duyệt rộng xử lý các đỉnh theo lớp khoảng cách tăng dần. Hệ quả trực tiếp: nó cho *đường đi ngắn nhất trên đồ thị không trọng số*, và khoảng cách tìm được là chính xác chứ không phải xấp xỉ. Hai biến thể đáng thuộc: *duyet rộng nhiều nguồn* — đẩy tất cả các nguồn vào hàng đợi lúc đầu, cho “khoảng cách tới nguồn gần nhất” trong một lượt; và *duyet rộng 0-1* — khi trọng số chỉ là 0 hoặc 1, dùng deque và đẩy vào đầu khi cạnh trọng số 0, cho $O(n + m)$ thay vì $O(m \log n)$ của Dijkstra.

Duyệt sâu không cho khoảng cách, nhưng nó cho một thứ khác có giá trị hơn: *cấu trúc*. Khi ghi lại thời điểm vào và thời điểm ra của mỗi đỉnh, ta được một bộ khung để đọc ra rất nhiều tính chất. Đây chính là đóng góp khái niệm lớn của Tarjan năm 1972[14]: biến duyệt sâu từ “một cách đi lòng vòng” thành một công cụ phân tích có hệ thống.

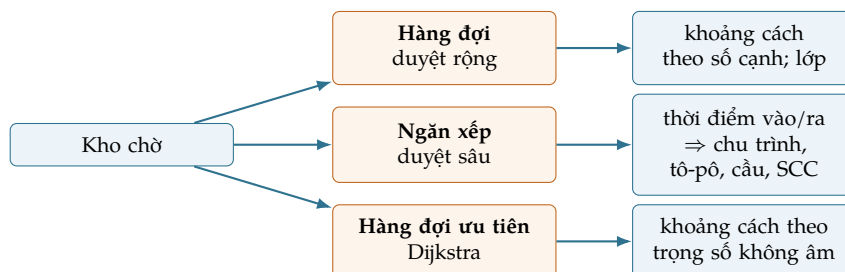
Bốn thứ mà thời điểm vào/ra của duyệt sâu để lộ ra:

Chu trình. Trong đồ thị có hướng, có chu trình khi và chỉ khi tồn tại cạnh đi ngược về một đỉnh đang nằm trên ngăn xếp hiện tại.

Thứ tự tô-pô. Trên đồ thị có hướng không chu trình (DAG), xếp các đỉnh theo thời điểm *ra* giảm dần cho một thứ tự tô-pô hợp lệ. Nếu phát hiện chu trình thì không có thứ tự tô-pô.

Cầu và khớp. Một cạnh là cầu khi cây con bên dưới nó không có đường ngược nào lên phía trên — điều kiện này viết được bằng thời điểm vào và một giá trị “lên cao nhất có thể”.

Thành phần liên thông mạnh. Cả hai thuật toán chuẩn — Kosaraju với hai lượt duyệt, Tarjan với một lượt — đều dựa trên cùng bộ thông tin đó. Sau khi nén mỗi thành phần thành một đỉnh, ta được đồ thị không chu trình, và mọi kỹ thuật quy hoạch động trên đồ thị không chu trình dùng lại được.



Hình 5.2. Cùng một khung “lấy ra, xử lý, đẩy vào”, đôi kho chờ thì được ba thuật toán với ba loại kết quả khác nhau. Nhận ra điều này giúp nhớ ít hơn và suy được nhiều hơn — và nó cũng giải thích ngay vì sao Dijkstra cần trọng số không âm (B/07).

5.3 Bốn thuật toán đường đi ngắn nhất, bốn giả thiết

Chọn thuật toán đường đi ngắn nhất không phải chuyện sở thích: mỗi thuật toán ứng với một giả thiết về trọng số, và dùng sai giả thiết cho kết quả sai chứ không phải chậm.

Không trọng số → duyệt rộng, $O(n + m)$. Đơn giản nhất, nhanh nhất; đừng dùng Dijkstra ở đây.

Trọng số 0 hoặc 1 → duyệt rộng 0-1 với deque, $O(n + m)$.

Trọng số không âm → Dijkstra với hàng đợi ưu tiên, $O(m \log n)$. Nó là thuật toán tham lam: chốt đỉnh có nhãn nhỏ nhất và không xét lại. Chứng minh dùng giả thiết không âm đúng một lần — ở chỗ khẳng định rằng một đường đi vòng qua các đỉnh chưa chốt không thể ngắn hơn — và đó chính là chỗ nó hỏng khi có cạnh âm.

Có trọng số âm $\rightarrow$ Bellman–Ford, $O(nm)$. Nó không chốt gì cả: nó thư giãn mọi cạnh $n - 1$ lần, và ý tưởng là “đường đi ngắn nhất có tối đa $n - 1$ cạnh, nên sau $n - 1$ vòng mọi nhãn đã ổn định”. Lợi thế kèm theo: nếu ở vòng thứ n vẫn còn cạnh thư giãn được thì đồ thị có chu trình âm — một thứ mà Dijkstra không phát hiện được.

Mọi cặp đỉnh $\rightarrow$ Floyd–Warshall, $O(n^3)$. Đáng chú ý vì cách nghĩ của nó là quy hoạch động thuần túy: $f[k][i][j]$ là đường ngắn nhất từ i tới j chỉ dùng các đỉnh trung gian trong k đỉnh đầu tiên. Ba vòng lặp lồng nhau, và thứ tự của chúng không hoán đổi được — vòng k phải nằm ngoài cùng, một lỗi rất phổ biến.

Ngoài bốn cái trên còn hai công cụ nên biết. A^* là Dijkstra cộng một hàm ước lượng khoảng cách còn lại; nếu hàm đó không bao giờ đánh giá quá thực tế thì kết quả vẫn tối ưu, còn nếu đánh giá tốt thì số đỉnh phải xét giảm rất nhiều. Đây là thuật toán chuẩn cho tìm đường của robot và trò chơi. Johnson biến đổi trọng số để loại bỏ số âm rồi chạy Dijkstra từ mọi đỉnh — hợp cho đồ thị thưa có cạnh âm.

Giả thiết trọng số	Thuật toán	Chi phí	Điều cần nhớ
Không trọng số	Duyệt rộng	$O(n + m)$	Nhanh nhất; nhiều nguồn cùng lúc rất tiện
Chỉ 0 và 1	Duyệt rộng 0–1	$O(n + m)$	Deque: cạnh 0 đẩy vào đầu, cạnh 1 vào cuối
Không âm	Dijkstra	$O(m \log n)$	Nhớ bỏ qua bản ghi lỗi thời khi lấy ra
Có thể âm	Bellman–Ford	$O(nm)$	Phát hiện được chu trình âm ở vòng thứ n
Mọi cặp, n nhỏ	Floyd–Warshall	$O(n^3)$	Vòng k bắt buộc ở ngoài cùng
Có ước lượng tốt	A^*	phụ thuộc heuristic	Heuristic không được đánh giá quá thực tế

5.4 Cây: các kỹ thuật riêng

Cây là đồ thị liên thông không chu trình, và chính vì thiếu chu trình mà nhiều bài trở nên dễ hơn hẳn. Ba kỹ thuật đáng thuộc.

Nhảy nhị phân cho tổ tiên chung gần nhất. Tiền xử lý $up[v][k] =$ tổ tiên thứ 2^k của v , tốn $O(n \log n)$; sau đó mỗi truy vấn tổ tiên chung gần nhất trả lời trong $O(\log n)$ bằng cách nâng hai đỉnh về cùng độ sâu rồi nhảy đồng thời. Kỹ thuật này còn dùng để tính nhanh cực trị hoặc tổng trên đường đi giữa hai đỉnh — chỉ cần lưu thêm thông tin tổng hợp cùng với bảng nhảy.

Dãy Euler. Duyệt sâu và ghi lại đỉnh mỗi khi đi qua, ta được một dãy mà bài toán tổ tiên chung trở thành bài toán cực tiểu trên đoạn — tức là chuyển hẳn sang công cụ của chương B/10. Đây là một trong những phép quy dẫn đẹp nhất giữa hai miền, và nó đi cả hai chiều: mọi bài cực tiểu trên đoạn cũng dựng được thành bài tổ tiên chung trên cây Cartesian.

Phân rã theo cạnh nặng. Chia cây thành các chuỗi dọc sao cho mọi đường đi từ gốc xuống lá chỉ cắt qua $O(\log n)$ chuỗi; nhờ đó truy vấn và cập nhật trên đường đi giữa hai đỉnh quy về $O(\log n)$ truy vấn đoạn trên các chuỗi. Xếp vào loại tham khảo nhưng rất mạnh khi cần.

5.5 Ba mẹo mô hình hoá đáng giá nhất

Ba mẹo dưới đây biến những bài trông không giải được thành bài quen thuộc.

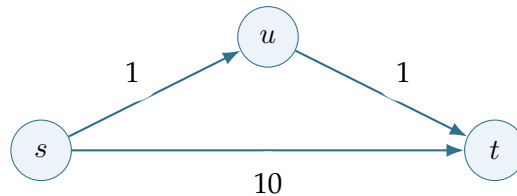
Đồ thị trạng thái. Đã nói ở mục 1, nhưng đáng nhắc lại kèm ví dụ: bài “có hai bình dung tích 3 và 5 lít, đong ra đúng 4 lít” có đỉnh là cặp (lượng nước bình 1, lượng nước bình 2) và cạnh là các thao tác đổ. Bài trở thành duyệt rộng trên vài chục đỉnh.

Đồ thị nhiều tầng. Khi bài cho phép một số hành động đặc biệt với số lần hạn chế — “được đi miễn phí tối đa k cạnh” — hãy nhân bản đồ thị thành $k + 1$ tầng, mỗi tầng ứng với số lần đã dùng, và cho cạnh đặc biệt nối tầng i sang tầng $i + 1$. Đường đi ngắn nhất trên đồ thị mở rộng chính là đáp án. Mẹo này là ứng dụng trực tiếp của nguyên tắc “trạng thái phải đủ” ở chương C/16: số lần đã dùng là một phần của trạng thái.

Tách đỉnh. Khi ràng buộc nằm ở *đỉnh* chứ không ở *cạnh* — “mỗi thành phố chỉ đi qua một lần”, “mỗi máy chỉ xử lý một việc” — hãy tách mỗi đỉnh thành hai nửa nối bởi một cạnh mang đúng ràng buộc đó. Kỹ thuật này đặc biệt quan trọng trong các bài luồng ở chương C/18.

5.6 Phản ví dụ ba đỉnh để chọn đúng đường đi ngắn nhất

Hình 5.3 có đường một cạnh $s \rightarrow t$ giá 10 và đường hai cạnh $s \rightarrow u \rightarrow t$ giá 2. BFS tối thiểu số cạnh sẽ chọn đường giá 10; Dijkstra tối thiểu tổng trọng số chọn đường giá 2. Vì vậy thay queue bằng heap không đủ: phải thay cả luật cập nhật khoảng cách bằng phép nối lỏng.



Hình 5.3. Ít cạnh nhất không đồng nghĩa tổng trọng số nhỏ nhất. Cả ba cạnh có hướng và trọng số không âm; Dijkstra tìm khoảng cách 2 tới t .

Khi dùng heap có phần tử cũ, mỗi lần lấy (d, u) , bỏ qua nếu $d \neq \text{dist}[u]$. Chỉ dừng sớm khi lấy đích ra với khoảng cách còn hợp lệ, không dừng lúc vừa nhìn thấy cạnh tới đích. Muốn thấy vì sao cấm cạnh âm, đổi $s \rightarrow t$ thành 2, $s \rightarrow u$ thành 5 và $u \rightarrow t$ thành -10 : chốt t ở 2 sẽ bỏ lỡ đường giá -5 . Đồ thị này là DAG nên vẫn giải được bằng thứ tự tô-pô.

Bảng 5.1. Bổ sung hai trường hợp dễ bị bỏ sót khi chọn đường đi ngắn nhất. n là số đỉnh, m là số cạnh, dùng danh sách kề.

Thuật toán	Giả thiết	Chi phí	Lỗi cần tránh
BFS	Mọi cạnh cùng giá dương	$O(n + m)$	Tối thiểu số cạnh trên đồ thị có giá khác nhau
0–1 BFS	Giá cạnh chỉ 0 hoặc 1	$O(n + m)$	Dùng visited để chặn mọi lần cải thiện
DP trên DAG	Có hướng, không chu trình; cho phép cạnh âm	$O(n + m)$	Dùng thứ tự DFS khi chưa loại chu trình
Dijkstra + heap nhị phân	Giá cạnh không âm	$O((n + m) \log n)$ trên đồ thị đơn	Chốt đỉnh lúc thêm vào heap
Bellman–Ford	Cho phép cạnh âm	$O(nm)$	Bỏ qua chu trình âm tới được từ nguồn

Bảng 5.1 tách giả thiết về trọng số khỏi giả thiết về cấu trúc. Nếu chu trình âm tới được từ nguồn và đi tiếp được tới đích, không tồn tại khoảng cách hữu hạn nhỏ nhất tới đích.

5.7 Dijkstra có phải là tận cùng không: rào cản sắp xếp

Bảng trên ghi Dijkstra là $O(m \log n)$ và dừng ở đó, như mọi giáo trình. Nhưng cái log ấy đến từ đâu, và có bắt buộc không? Câu hỏi này có một lịch sử dài và một câu trả lời rất mới, đáng kể lại vì nó cho thấy cách một “sự thật hiển nhiên” bị lật.

Nguồn của log là hàng đợi ưu tiên: Dijkstra rút các đỉnh ra *theo đúng thứ tự khoảng cách tăng dần*, nghĩa là chạy xong thì nó đã sắp xếp n đỉnh theo khoảng cách. Mà sắp xếp n số bằng so sánh cần $\Omega(n \log n)$ (D/24). Vậy mọi thuật toán làm đúng như Dijkstra — chốt đỉnh theo thứ tự — đều bị chặn dưới bởi chi phí sắp xếp. Người ta gọi đó là *rào cản sắp xếp*, và với hàng đợi Fibonacci[15] thì cận $O(m + n \log n)$ chính là chạm đúng rào cản ấy.

Cách vượt rào thứ nhất là bỏ mô hình so sánh. Nếu trọng số là số nguyên và máy được phép làm số học trên chúng, cận dưới sắp xếp không còn áp dụng — đúng như sắp xếp đếm không mâu thuẫn với cận $n \log n$. Thorup[16] khai thác đúng chỗ đó và cho thuật toán *tuyến tính* $O(m)$ cho đồ thị *vô hướng* với trọng số nguyên dương, bằng cách xây một cấu trúc phân tầng theo bit và chấp nhận xử lý các đỉnh “theo nhóm không cần đúng thứ tự”. Chi tiết đáng nhớ là chỗ giả thiết được dùng: vô hướng và nguyên, cả hai đều thiết yếu.

Cách vượt rào thứ hai mới xuất hiện gần đây và mạnh hơn về mặt khái niệm, vì nó *không* bỏ mô hình so sánh. Duan, Mao, Mao, Shu và Yin[17] đưa ra thuật toán tất định $O(m \log^{2/3} n)$ cho đồ thị *có hướng* với trọng số thực không âm, trong mô hình chỉ có so sánh và cộng — tức là phá vỡ cận $O(m + n \log n)$ của Dijkstra trên đồ thị thưa và chứng minh rằng Dijkstra *không* tối ưu cho bài toán đường đi ngắn nhất. Ý tưởng cốt lõi là làm ít hơn cái Dijkstra làm: thuật toán tránh việc sắp xếp đầy đủ mọi đỉnh theo khoảng cách, và chỉ giữ đủ thứ tự để các nhãn vẫn đúng. Bài được trao giải bài báo xuất sắc tại STOC 2025.

Đừng cài các thuật toán này. Giá trị của mục này nằm ở ba câu hỏi mà nó dạy bạn tự đặt cho *bất kỳ* thuật toán quen thuộc nào. Một, thuật toán này có đang làm nhiều hơn việc đề bài yêu cầu không — Dijkstra sắp xếp toàn bộ đỉnh trong khi đề chỉ hỏi khoảng cách. Hai, cận dưới mà tôi vừa viện dẫn thuộc mô hình nào, và bài toán của tôi có thật sự nằm trong mô hình đó không. Ba, khi một cận đứng yên hàng chục năm, đó là dấu hiệu của một định lý hay chỉ là dấu hiệu chưa ai thử đúng chỗ? Với đường đi ngắn nhất, câu trả lời hoá ra là về sau — sau sáu mươi sáu năm.

5.8 Gốc rễ: bảy cây cầu, một quán cà phê, và một con robot

Lý thuyết đồ thị có một ngày sinh khá rõ ràng: năm 1736, Euler giải bài toán bảy cây cầu ở Königsberg — có đi qua mỗi cây cầu đúng một lần rồi về chỗ cũ được không. Điều làm bài báo đó thành khởi đầu của cả một ngành không phải câu trả lời (không) mà là *cách trả lời*: Euler bỏ đi toàn bộ hình dạng địa lý, giữ lại đúng quan hệ “vùng đất nào nối với vùng đất nào”, và chứng minh kết luận chỉ từ tính chẵn lẻ của số cầu tại mỗi vùng. Đó chính xác là thao tác mô hình hoá mà mục 1 nói tới, thực hiện lần đầu tiên.

Thuật toán Dijkstra có một nguồn gốc rất đời và được chính tác giả kể lại nhiều lần: năm 1956, ông đang ngồi ở một quán cà phê tại Amsterdam cùng vợ chưa cưới, và cần một bài toán trình diễn cho máy tính ARMAC sao cho khán giả không chuyên hiểu được. Ông nghĩ ra bài đường đi ngắn nhất giữa hai thành phố Hà Lan, và tìm ra thuật toán trong khoảng hai mươi phút, không giấy bút. Ông xuất bản nó ba năm sau trong một bài báo hai trang[18]. Chi tiết đáng chú ý về mặt phương pháp: ràng buộc “phải giải thích được cho người không chuyên” đã đẩy ông tới một thuật toán đơn giản — và sự đơn giản đó là lý do nó vẫn được dùng khắp nơi sau bảy mươi năm.

Duyệt sâu thì cổ hơn nhiều — nó xuất hiện trong các phương pháp giải mê cung từ thế kỷ 19 — nhưng đóng góp quyết định là của Tarjan năm 1972[14]. Ông chỉ ra rằng nếu ghi lại thời điểm vào và ra của mỗi đỉnh thì duyệt sâu trở thành một *khung phân tích*: cùng một lượt duyệt cho ra

thành phần liên thông mạnh, cầu, khớp, và nhiều tính chất khác. Trước đó mỗi bài toán có một thuật toán riêng; sau đó chúng trở thành các cách đọc khác nhau của cùng một cấu trúc dữ liệu.

Cuối cùng, A^* ra đời năm 1968 tại Viện Nghiên cứu Stanford, từ một nhu cầu rất cụ thể: robot Shakey — một trong những robot di động đầu tiên có khả năng lập kế hoạch — cần tìm đường trong một môi trường thật với tài nguyên tính toán rất hạn chế. Hart, Nilsson và Raphael thêm vào Dijkstra một hàm ước lượng khoảng cách còn lại và chứng minh điều kiện để kết quả vẫn tối ưu. Đây là lý do A^* tới nay vẫn là thuật toán lập kế hoạch đường đi mặc định trong robot học và trong trò chơi.

I Gốc rễ: ý tưởng này ra đời từ bài toán nào

Mốc	Bài toán thúc ép lúc đó	Điều thay đổi sau đó
1736 — Euler	Bảy cây cầu Königsberg: đi qua mỗi cầu đúng một lần được không?	Khai sinh lý thuyết đồ thị; và quan trọng hơn, khai sinh thao tác <i>bỏ chi tiết, giữ quan hệ</i> — chính là mô hình hoá.
1956 — Dijkstra	Cần một bài trình diễn máy tính mà khán giả không chuyên hiểu được.	Thuật toán đường đi ngắn nhất, nghĩ ra trong 20 phút không giấy bút; ràng buộc “phải dễ hiểu” dẫn tới sự đơn giản khiến nó sống lâu.
1958–62 — Bellman, Ford, Floyd, Warshall 1968 — Hart, Nilsson, Raphael (SRI)	Mạng có chi phí âm (chiết khấu, chênh lệch tỉ giá) và nhu cầu tính mọi cặp. Robot Shakey phải lập kế hoạch đường đi thật với tài nguyên rất hạn chế.	Bellman–Ford và Floyd–Warshall; cả hai là quy hoạch động thuần túy trên đồ thị. A^* : Dijkstra cộng ước lượng khoảng cách còn lại; vẫn là thuật toán mặc định trong robot học và trò chơi.
1972 — Tarjan	Mỗi tính chất của đồ thị lại có một thuật toán riêng.	Duyệt sâu với thời điểm vào/ra trở thành khung phân tích chung: SCC, cầu, khớp, tô-pô đều đọc ra từ cùng một lượt duyệt.
Thập niên 2000 — định tuyến bản đồ	Tìm đường trên mạng đường bộ hàng trăm triệu cạnh, trả lời trong mili-giây.	Các kỹ thuật tiền xử lý như phân cấp co rút; cho thấy Dijkstra thuần túy không đủ ở quy mô thật, nhưng vẫn là nền của mọi cải tiến.

I 5.9 Liên hệ ngang

I Liên hệ ngang: ý này còn sống ở đâu nữa

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu
SLAM và robot	Đồ thị tư thế: đỉnh là tư thế, cạnh là ràng buộc đo được	Đây là mô hình hoá đúng nghĩa: bài toán ước lượng trở thành bài toán trên đồ thị, và việc đóng vòng chính là thêm một cạnh. Khác với chương này: ở đó ta không tìm đường đi mà <i>tối ưu vị trí các đỉnh</i> sao cho tổng sai lệch nhỏ nhất — một bài bình phương tối thiểu trên cấu trúc đồ thị.
Lập kế hoạch đường đi	A^* , lưới chiếm dụng, đồ thị khả kiến	Đúng thuật toán của mục 3, với đỉnh là ô lưới hoặc trạng thái tư thế. Bài học chuyển được: chất lượng hàm ước lượng quyết định số đỉnh phải xét, và một hàm đánh giá quá thực tế sẽ phá vỡ tính tối ưu.
Trình biên dịch	Đồ thị luồng điều khiển, đồ thị phụ thuộc, tô màu để cấp phát thanh ghi	Sắp thứ tự tô-pô dùng để xếp lịch lệnh; phát hiện chu trình dùng để tìm vòng lặp; tô màu đồ thị dùng để gán biến vào thanh ghi — một bài NP-khó được giải bằng heuristic trong mọi trình biên dịch bạn dùng.

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu (tiếp)
Mạng máy tính	Giao thức định tuyến	Định tuyến theo trạng thái liên kết chính là Dijkstra chạy phân tán; định tuyến theo vector khoảng cách chính là Bellman–Ford chạy phân tán. Đây là ví dụ hiếm hoi mà hai thuật toán sách giáo khoa tương ứng đúng với hai họ giao thức thực tế.
Quản lý dự án	Sơ đồ mạng công việc, đường găng	Đường dài nhất trên đồ thị không chu trình — giải được vì không có chu trình, đúng như C/16 đã lưu ý. Đường găng là các công việc mà chậm một ngày thì cả dự án chậm một ngày.
Web và mạng xã hội	Xếp hạng trang, phát hiện cộng đồng, đường đi ngắn nhất xã hội	Xếp hạng trang là bước ngẫu nhiên trên đồ thị, tức là chuỗi Markov — nổi chương này với xác suất ở A/05. Quy mô ở đây buộc phải dùng thuật toán xấp xỉ và tính toán phân tán.
Hoá học và sinh học	Đồ thị phân tử, mạng phản ứng, mạng tương tác protein	Bài toán đẳng cấu đồ thị — hai phân tử có cùng cấu trúc không — là một trong số ít bài toán chưa được xếp dứt khoát vào P hay NP-đầy đủ (D/23).

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Ba dấu hiệu. Một, đề nói về quan hệ đôi một giữa các đối tượng. Hai, đề nói về ràng buộc thứ tự hoặc phụ thuộc — lúc đó câu hỏi “có xếp được không” chính là “có chu trình không”. Ba, và đáng giá nhất, đề mô tả một hệ có nhiều *trạng thái* và các thao tác chuyển giữa chúng; lúc đó đỉnh là trạng thái, cạnh là thao tác, và “ít bước nhất” là đường đi ngắn nhất. Dấu hiệu thứ ba mở ra các bài không nhắc gì tới đồ thị: trò xếp hình, đong nước, biến đổi chuỗi.
- A2. Vì thứ tự xử lý quyết định thông tin được đảm bảo. Hàng đợi cho thứ tự theo lớp khoảng cách, nên lần đầu chạm tới một đỉnh là đường ngắn nhất — một tính chất mạnh và trực tiếp. Ngăn xếp cho thứ tự lồng nhau: một đỉnh chỉ ra khỏi ngăn xếp sau khi mọi thứ dưới nó đã xong, nên khoảng thời gian vào/ra của các đỉnh có quan hệ bao nhau đúng theo cấu trúc cây duyệt. Chính tính bao nhau đó là thứ cho ra thứ tự tô-pô, cầu và thành phần liên thông mạnh.
- A3. Dijkstra dùng giả thiết không âm ở đúng một chỗ: khi chốt đỉnh có nhãn nhỏ nhất, nó khẳng định rằng không có đường nào tới đỉnh đó rẻ hơn, vì mọi đường khác phải đi qua một đỉnh chưa chốt có nhãn lớn hơn hoặc bằng, và các cạnh sau đó *chỉ làm tăng* chi phí. Với cạnh âm, các cạnh sau có thể làm giảm chi phí nên lập luận sụp. Bellman–Ford không chốt gì cả: nó thư giãn mọi cạnh $n - 1$ lần, dựa trên nhận xét đường đi ngắn nhất có tối đa $n - 1$ cạnh — và nếu vòng thứ n vẫn cải thiện được thì tồn tại chu trình âm.
- A4. Vì mọi bài quy hoạch động đều là bài đường đi ngắn nhất trên *đồ thị trạng thái*: đỉnh là trạng thái, cạnh là chuyển, trọng số là chi phí của chuyển đó, và thứ tự tính hợp lệ tồn tại đúng khi đồ thị không có chu trình. Ngược lại, đường đi ngắn nhất trên đồ thị không chu trình là quy hoạch động theo thứ tự tô-pô. Cách nhìn này có giá trị thực dụng: khi quy hoạch động của bạn có chu trình phụ thuộc, hãy chuyển sang Dijkstra trên đồ thị trạng thái thay vì cố ép một thứ tự tính.
- A5. Bằng nhảy nhị phân: tiền xử lý bảng $up[v][k]$ — tổ tiên thứ 2^k của v — trong $O(n \log n)$, rồi mỗi truy vấn nâng hai đỉnh về cùng độ sâu và nhảy đồng thời từ bước lớn xuống bước nhỏ, tốn $O(\log n)$. Cách thứ hai là chuyển sang dãy Euler và biến bài toán thành cực tiểu trên đoạn $(B/10)$, sau đó dùng sparse table cho truy vấn $O(1)$ — lựa chọn tốt khi số truy vấn rất lớn.

NÓI LẠI THẬT ĐƠN GIẢN

Đồ thị chỉ là cách vẽ lại một bài toán thành các điểm và các đường nối. Điều đáng nói không phải bản thân hình vẽ mà là việc rất nhiều bài toán khác nhau, sau khi vẽ lại, hoá ra là cùng một bài. Hãy nghĩ tới bản đồ tàu điện. Các ga là điểm, các tuyến là đường nối. Câu hỏi “đi từ ga A tới ga B ít trạm nhất” được trả lời bằng cách loang dần: đánh dấu mọi ga cách A một trạm, rồi hai trạm, rồi ba — lần đầu chạm tới B là đáp án. Đó là duyệt rộng, và nó đúng vì bạn xét theo đúng thứ tự tăng dần của số trạm.

Nếu mỗi tuyến mất thời gian khác nhau thì cách loang đều không còn đúng, và bạn phải luôn đi tiếp từ ga có *thời gian tích lũy nhỏ nhất* trong số các ga đang chờ. Đó là Dijkstra. Điều kiện để nó đúng: đi thêm một tuyến không bao giờ làm giảm tổng thời gian. Nếu có tuyến “thường” làm giảm tổng thời gian — tức trọng số âm — thì cách này sai, và phải dùng cách chậm hơn nhưng chắc chắn: cứ cải thiện mọi tuyến nhiều lượt cho tới khi không cải thiện được nữa.

Còn mẹo mạnh nhất của chương này thì thế này: đỉnh *không nhất thiết là địa điểm*. Nếu bài toán nói bạn có hai cái bình 3 lít và 5 lít và cần đo đúng 4 lít, thì hãy coi mỗi “tình huống” — ví dụ bình một có 2 lít, bình hai có 5 lít — là một điểm, và mỗi thao tác đổ là một đường nối. Bỗng nhiên bài đổ mёo trở thành bài tìm đường trên một bản đồ vài chục điểm, và bạn giải nó bằng đúng cách loang ở trên.

Đó là toàn bộ ý của mô hình hoá, và nó cũng chính là điều Euler làm năm 1736 khi ông xoá bản đồ thành phố đi và chỉ giữ lại xem vùng đất nào nối với vùng đất nào.

Một câu: Khi bí, hãy hỏi “cái gì là đỉnh, cái gì là cạnh” — và nhớ rằng đỉnh có thể là một trạng thái, không chỉ là một địa điểm.

Chỗ không rút gọn được: Với đồ thị trạng thái, việc ước lượng số trạng thái phải làm bằng tay cho từng bài; không có cách nào tự động biết nó là 10^6 hay 10^{15} .

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
●●● Ba dấu hiệu bài đồ thị	Quan hệ đôi một; ràng buộc thứ tự/phụ thuộc; và <i>trạng thái rời bước chuyển</i> — dấu hiệu thứ ba đáng giá nhất.
●●● Bảng chọn thuật toán đường đi ngắn nhất	Không trọng số: BFS. Trọng số 0/1: BFS 0–1 với deque. Không âm: Dijkstra. Có âm: Bellman–Ford. Mọi cặp, n nhỏ: Floyd–Warshall.
●●● Chỗ Dijkstra dùng giả thiết không âm	Khi chốt đỉnh nhãn nhỏ nhất, nó tin rằng đường khác đi qua đỉnh chưa chốt chỉ có thể đắt hơn — đúng vì cạnh không làm giảm chi phí.
●●● Bellman–Ford và chu trình âm	Thư giãn mọi cạnh $n - 1$ lần (đường ngắn nhất có $\leq n - 1$ cạnh); nếu vòng thứ n vẫn cải thiện thì có chu trình âm.
●●● Duyệt sâu để lộ những gì	Thời điểm vào/ra cho: phát hiện chu trình, thứ tự tô-pô (ra giảm dần), cầu và khớp, thành phần liên thông mạnh.
●●● Quy hoạch động và đường đi ngắn nhất	Là hai cách nói về cùng một thứ: đỉnh là trạng thái, cạnh là chuyển. Quy hoạch động có chu trình phụ thuộc $\Rightarrow$ dùng Dijkstra trên đồ thị trạng thái.
●● Ba mёo mô hình hoá	Đồ thị trạng thái; đồ thị nhiều tầng (nhân bản theo số lần dùng khả năng đặc biệt); tách đỉnh (khi ràng buộc nằm ở đỉnh).
●● Duyệt rộng nhiều nguồn	Đẩy mọi nguồn vào hàng đợi lúc khởi tạo $\Rightarrow$ khoảng cách tới nguồn gần nhất trong một lượt $O(n + m)$.
●● Nhảy nhị phân	$up[v][k] =$ tổ tiên thứ 2^k ; tiền xử lý $O(n \log n)$, truy vấn tổ tiên chung $O(\log n)$; mở rộng được cho cực trị trên đường đi.
●● Dây Euler	Biến bài tổ tiên chung thành bài cực tiểu trên đoạn $(B/10)$; phép quy dẫn đi được cả hai chiều.
●● A^*	Dijkstra cộng ước lượng khoảng cách còn lại; tối ưu nếu ước lượng không bao giờ vượt quá thực tế.
●● Lỗi Floyd–Warshall phổ biến	Vòng k phải ở ngoài cùng; đảo thứ tự vòng lặp cho kết quả sai.
● Phân rã theo cạnh nặng	Chia cây thành các chuỗi để truy vấn trên đường đi quy về $O(\log n)$ truy vấn đoạn; mạnh nhưng thuộc loại tham khảo.
● Rào cản sắp xếp	Dijkstra chốt đỉnh theo đúng thứ tự khoảng cách nên thừa hưởng cận dưới của sắp xếp. Vượt được bằng cách đổi mô hình (trọng số nguyên, đồ thị vô hướng) hoặc bằng cách không sắp xếp đầy đủ — $O(m \log^{2/3} n)$, 2025.

■ Câu hỏi ôn tập có đáp án (che phần đáp án, tự trả lời trước)

1. Khi nào tôi nên dùng ma trận kề thay vì danh sách kề?

| Chỉ khi n nhỏ (vài nghìn trở xuống) và đồ thị dày, hoặc khi bạn cần hỏi “có cạnh giữa u và v không” rất nhiều lần trong $O(1)$. Ma trận tốn $O(n^2)$ bộ nhớ nên với $n = 10^5$ là không khả thi. Trong hầu hết bài thi đấu, danh sách kề là lựa chọn mặc định, và nếu cần kiểm tra cạnh nhanh thì dùng thêm một tập băm các cặp.

2. Dijkstra của tôi cho kết quả sai trên đồ thị có cạnh trọng số 0. Có phải vì trọng số 0?

| Không, trọng số 0 hoàn toàn hợp lệ với Dijkstra — giả thiết là *không âm*, và 0 không âm. Lỗi gần như chắc chắn nằm ở chỗ khác: quên bỏ qua bản ghi lỗi thời khi lấy ra khỏi hàng đợi ưu tiên, hoặc cập nhật nhãn nhưng không đẩy vào hàng đợi, hoặc so sánh sai khi hai nhãn bằng nhau. Nếu đồ thị chỉ có trọng số 0 và 1 thì hãy dùng duyệt rộng 0-1, vừa nhanh hơn vừa ít chỗ sai hơn.

3. Làm sao đếm số đường đi ngắn nhất chứ không chỉ độ dài?

| Thêm một mảng $\text{cnt}[v]$. Khi thư giãn cạnh (u, v) : nếu tìm được đường ngắn hơn thì $\text{cnt}[v] = \text{cnt}[u]$; nếu bằng đúng khoảng cách hiện tại thì $\text{cnt}[v] += \text{cnt}[u]$. Cách này hoạt động với cả duyệt rộng và Dijkstra, với điều kiện bạn xử lý mỗi đỉnh theo đúng thứ tự khoảng cách tăng dần. Nhớ lấy modulo nếu số đường đi lớn.

4. Bài của tôi có 10^6 đỉnh trạng thái nhưng tôi không muốn sinh trước toàn bộ đồ thị. Làm sao?

| Sinh cạnh *khi cần* — gọi là đồ thị ẩn. Thay vì lưu danh sách kề, viết một hàm “cho tôi các trạng thái kề của trạng thái này” và gọi nó trong vòng lặp duyệt. Cách này tiết kiệm bộ nhớ rất nhiều và là cách chuẩn cho các bài trò xếp hình hay bài đong nước. Điều cần thêm là một cấu trúc để đánh dấu trạng thái đã thăm — thường là bảng băm nếu trạng thái không ánh xạ gọn về số nguyên.

5. Thứ tự tô-pô có thể có nhiều kết quả. Làm sao lấy kết quả nhỏ nhất theo thứ tự từ điển?

| Dùng thuật toán Kahn nhưng thay hàng đợi bằng hàng đợi ưu tiên: luôn lấy ra đỉnh có chỉ số nhỏ nhất trong số các đỉnh không còn phụ thuộc. Chi phí thành $O(m \log n)$. Lưu ý đây là một câu hỏi khác với “thứ tự tô-pô bất kỳ”, và nhiều bài cổ tình hỏi bản khó hơn — hãy đọc kỹ đề.

6. Vì sao cùng một bài lại có bài giải bằng luồng cực đại, bài giải bằng Dijkstra?

| Vì hai họ bài toán trả lời hai câu hỏi khác nhau. Đường đi ngắn nhất hỏi về *một* đường; luồng hỏi về *nhiều* đường đồng thời với ràng buộc sức chứa. Dấu hiệu phân biệt trên đề: nếu có khái niệm “sức chứa”, “mỗi thứ chỉ dùng một lần”, “phân công” thì đó là luồng hoặc ghép đôi (C/18); nếu chỉ có chi phí đi lại thì đó là đường đi ngắn nhất.

7. Trong hệ thống thật, Dijkstra có đủ cho định tuyến bản đồ không?

| Không ở quy mô thật. Mạng đường bộ một quốc gia có hàng chục triệu đỉnh, và Dijkstra thuần túy mất hàng giây cho một truy vấn — quá chậm cho dịch vụ trực tuyến. Các hệ thống thật dùng tiền xử lý nặng: phân cấp co rút, nhãn khoảng cách, hoặc phân vùng — những kỹ thuật đổi hàng giờ tiền xử lý lấy truy vấn dưới mili-giây. Nhưng tất cả đều xây trên Dijkstra và đều dùng đúng tính chất tham lam của nó, nên hiểu Dijkstra vẫn là điều kiện cần.

8. Phát hiện chu trình trong đồ thị vô hướng và có hướng khác nhau thế nào?

| Khác ở tiêu chí. Trong đồ thị vô hướng, gặp một đỉnh đã thăm mà không phải cha trực tiếp là có chu trình — hoặc đơn giản hơn, dùng DSU (B/11): cạnh nối hai đỉnh cùng nhóm tạo chu trình. Trong đồ thị có hướng, phải phân biệt “đã thăm xong” với “đang trên ngăn xếp”: chỉ cạnh trở về một đỉnh *đang trên ngăn xếp* mới tạo chu trình. Nhầm hai trạng thái này là một trong những lỗi phổ biến nhất khi cài.

■ Bài tập (mức: ● khởi động ●● chuẩn ●●● khó)

Bài	Nguồn	Điều cần nhận ra
• Đếm số thành phần liên thông	CSES “Counting Rooms”[1]	Duyệt sâu hoặc rộng trên lưới; chú ý độ sâu đệ quy với lưới lớn.
• Đường đi ngắn nhất trên lưới có vật cản	CSES “Labyrinth”[1]	Duyệt rộng kèm truy vết đường đi; bài mẫu cho việc lưu “đỉnh trước”.
• Kiểm tra đồ thị hai phía	CSES “Building Teams”[1]	Tô màu bằng duyệt rộng; so với cách DSU chuẩn lẽ ở B/11.
•• Đường đi ngắn nhất có trọng số	CSES “Shortest Routes I”[1]	Dijkstra với hàng đợi ưu tiên; nhớ dòng bỏ qua bản ghi lỗi thời.
•• Phát hiện chu trình âm	CSES “Cycle Finding”[1]	Bellman–Ford và truy vết chu trình; chú ý đỉnh không tới được từ nguồn.
•• Sắp thứ tự tô-pô và đường đi dài nhất trên đồ thị không chu trình	CSES “Course Schedule”, “Longest Flight Route”[1]	Quy hoạch động theo thứ tự tô-pô — ví dụ trực tiếp cho quan hệ ở mục 3.
•• Bài đong nước với hai bình	Bài kinh điển	Đồ thị trạng thái; ước lượng số trạng thái trước khi cài.
•• Số đường đi ngắn nhất giữa hai đỉnh	CSES “Investigation”[1]	Dijkstra kèm bốn đại lượng cùng lúc; luyện việc mở rộng thuật toán chuẩn.
••• Thành phần liên thông mạnh và nén đồ thị	CSES “Planets and Kingdoms”[1]	Kosaraju hoặc Tarjan; sau khi nén thì chạy quy hoạch động trên đồ thị không chu trình.
••• Tìm cầu và khớp	Bài kinh điển	Duyệt sâu với giá trị “lên cao nhất”; chú ý xử lý cạnh song song.
••• Tổ tiên chung gần nhất với 10^5 truy vấn	CSES “Company Queries II”[1]	Nhảy nhị phân; rồi thử lại bằng dãy Euler cộng sparse table và so thời gian.
••• Đường đi ngắn nhất khi được đi miễn phí tối đa k cạnh	Bài thi đấu kinh điển	Đồ thị nhiều tầng; đây là bài mẫu cho mẹo ở mục 5.
••• A^* cho bài xếp hình 15 ô	Bài kinh điển	So số đỉnh phải xét giữa duyệt rộng, Dijkstra và A^* với hai hàm ước lượng khác nhau.

Kết chương và đọc tiếp

Đồ thị là chương của kỹ năng mô hình hoá, và ba câu hỏi ở khối *Cần nhớ* — đỉnh là gì, cạnh là gì, trọng số là gì — đáng thành phần xạ. Hai điều nữa đáng mang theo: đổi kho chờ thì đổi thuật toán, nên duyệt rộng, duyệt sâu và Dijkstra là ba biến thể của một khung; và quy hoạch động chính là đường đi ngắn nhất trên đồ thị trạng thái, nên hai chương này là hai cách nói về cùng một thứ.

Chương tiếp theo mở rộng mô hình đồ thị theo một hướng mới: thay vì hỏi về *một* đường đi, ta hỏi về *nhiều* đường đi đồng thời với ràng buộc sức chứa. Đó là luồng cực đại, và điều bất ngờ là rất nhiều bài không liên quan gì tới “dòng chảy” — phân công công việc, chọn tập đại diện, một số bài chia đôi tối ưu — đều quy về đúng mô hình đó (C/18).

Đọc thêm: công trình gốc (chỉ liệt kê nguồn đã qua phản biện; bản tiền ấn được ghi rõ)

Mức	Công trình và tác giả	Nơi công bố, năm	Vì sao đọc, đọc phần nào
nền tảng	Dijkstra, “A Note on Two Problems in Connexion with Graphs”	Numerische Mathematik, 1959	Ba trang, không có ký hiệu tiệm cận nào; đọc để thấy thuật toán được nghĩ ra như thế nào trước khi ngành có ngôn ngữ để nói về nó.

Mức	Công trình và tác giả	Nơi công bố, năm	Vì sao đọc (tiếp)
nền tảng	Duan, Mao, Mao, Shu, Yin, “Breaking the Sorting Barrier for Directed Single-Source Shortest Paths”	STOC, 2025 (Best Paper)	Đọc phần mở đầu: nó giải thích rào cản sắp xếp và ý tưởng tránh sắp xếp rõ hơn bất kỳ bài giới thiệu nào.
tra khi cần	Thorup, “Undirected Single-Source Shortest Paths with Positive Integer Weights in Linear Time”	Journal of the ACM, 1999	Ví dụ mẫu mực về việc đổi mô hình (số nguyên, vô hướng) để thoát một cận dưới.
tra khi cần	Tarjan, “Depth-First Search and Linear Graph Algorithms”	SIAM Journal on Computing, 1972	Bài gốc của thành phần liên thông mạnh; đọc kèm mục 2 của chương này.
tra khi cần	Fredman & Tarjan, “Fibonacci Heaps...”	Journal of the ACM, 1987	Nguồn của cận $O(m + n \log n)$; xem thêm phần đo thực nghiệm ở B/07 trước khi quyết định dùng.

Nguồn

- [1] A. Laaksonen. “CSES Problem Set.” (2026), address: <https://cses.fi/problemset/> (visited on 09/06/2026).
- [2] M. Blum, R. W. Floyd, V. Pratt, et al., “Time Bounds for Selection,” *Journal of Computer and System Sciences*, vol. 7, no. 4, pp. 448–461, 1973.
- [3] V. Strassen, “Gaussian Elimination Is Not Optimal,” *Numerische Mathematik*, vol. 13, pp. 354–356, 1969.
- [4] D. Harvey and J. van der Hoeven, “Integer Multiplication in Time $O(n \log n)$,” *Annals of Mathematics*, vol. 193, no. 2, pp. 563–617, 2021.
- [5] J. Alman and V. Vassilevska Williams, “A Refined Laser Method and Faster Matrix Multiplication,” in *Proceedings of the 32nd ACM-SIAM Symposium on Discrete Algorithms (SODA)*, 2021.
- [6] J. Alman, R. Duan, V. Vassilevska Williams, et al. “Improving the Matrix Multiplication Exponent with Modern Optimization and AlphaEvolve.” bản tiền ấn arXiv:2608.16884, chưa qua phản biện. (2026), address: <https://arxiv.org/abs/2608.16884> (visited on 09/07/2026).
- [7] J. W. Cooley and J. W. Tukey, “An Algorithm for the Machine Calculation of Complex Fourier Series,” *Mathematics of Computation*, vol. 19, no. 90, pp. 297–301, 1965.
- [8] D. R. Musser, “Introspective Sorting and Selection Algorithms,” *Software: Practice and Experience*, vol. 27, no. 8, pp. 983–993, 1997.
- [9] J. L. Bentley and M. D. McIlroy, “Engineering a Sort Function,” *Software: Practice and Experience*, vol. 23, no. 11, pp. 1249–1265, 1993.
- [10] D. E. Knuth, “Optimum Binary Search Trees,” *Acta Informatica*, vol. 1, pp. 14–25, 1971.
- [11] F. F. Yao, “Speed-Up in Dynamic Programming,” *SIAM Journal on Algebraic and Discrete Methods*, vol. 3, no. 4, pp. 532–540, 1982.
- [12] A. Aggarwal, M. M. Klawe, S. Moran, et al., “Geometric Applications of a Matrix-Searching Algorithm,” *Algorithmica*, vol. 2, pp. 195–208, 1987.
- [13] D. S. Hirschberg, “A Linear Space Algorithm for Computing Maximal Common Subsequences,” *Communications of the ACM*, vol. 18, no. 6, pp. 341–343, 1975.
- [14] R. E. Tarjan, “Depth-First Search and Linear Graph Algorithms,” *SIAM Journal on Computing*, vol. 1, no. 2, pp. 146–160, 1972.
- [15] M. L. Fredman and R. E. Tarjan, “Fibonacci Heaps and Their Uses in Improved Network Optimization Algorithms,” *Journal of the ACM*, vol. 34, no. 3, pp. 596–615, 1987.
- [16] M. Thorup, “Undirected Single-Source Shortest Paths with Positive Integer Weights in Linear Time,” *Journal of the ACM*, vol. 46, no. 3, pp. 362–394, 1999.
- [17] R. Duan, J. Mao, X. Mao, et al., “Breaking the Sorting Barrier for Directed Single-Source Shortest Paths,” in *Proceedings of the 57th Annual ACM Symposium on Theory of Computing (STOC)*, Best Paper Award, 2025.
- [18] E. W. Dijkstra, “A Note on Two Problems in Connexion with Graphs,” *Numerische Mathematik*, vol. 1, pp. 269–271, 1959.

- [19] KTH Royal Institute of Technology. “KACTL: KTH Algorithm Competition Template Library.” (2026), address: <https://github.com/kth-competitive-programming/kactl> (visited on 09/06/2026).
- [20] H. W. Kuhn, “The Hungarian Method for the Assignment Problem,” *Naval Research Logistics Quarterly*, vol. 2, no. 1-2, pp. 83–97, 1955.
- [21] J. B. Orlin, “Max Flows in $O(nm)$ Time, or Better,” in *Proceedings of the 45th Annual ACM Symposium on Theory of Computing (STOC)*, 2013, pp. 765–774.
- [22] L. Chen, R. Kyng, Y. P. Liu, et al., “Maximum Flow and Minimum-Cost Flow in Almost-Linear Time,” in *Proceedings of the 63rd IEEE Symposium on Foundations of Computer Science (FOCS)*, Best Paper Award, 2022.
- [23] J. E. Hopcroft and R. M. Karp, “An $n^{5/2}$ Algorithm for Maximum Matchings in Bipartite Graphs,” *SIAM Journal on Computing*, vol. 2, no. 4, pp. 225–231, 1973.
- [24] J. Bentley, *Programming Pearls*, 2nd ed. Addison-Wesley, 2000.
- [25] R. Motwani and P. Raghavan, *Randomized Algorithms*. Cambridge University Press, 1995.

Luồng cực đại và ghép đôi (Max Flow, Matching, Min Cut)

PHỤ THUỘC C/17 bắt buộc — toàn bộ chương này là mô hình đồ thị mở rộng; A/04 cho tính đúng của Ford-Fulkerson và cho chứng minh định lý luồng cực đại-lát cắt nhỏ nhất.

TÓM TẮT

Luồng cực đại là ví dụ mạnh nhất trong quyển sách này cho sức mạnh của việc mô hình hoá: rất nhiều bài không hề nói tới “dòng chảy” — phân công người vào việc, chọn dự án có lợi nhất, phân đoạn ảnh, kiểm tra một đội bóng còn cơ hội vô địch — đều quy về đúng một mô hình. Trung tâm lý thuyết của chương là định lý luồng cực đại bằng lát cắt nhỏ nhất, một sự trùng khớp giữa bài toán tối đa và bài toán tối thiểu; đó cũng là lý do luồng vừa cho lời giải vừa cho *chứng cứ* rằng lời giải là tối ưu. Chương tập trung vào mô hình hoá và các mẹo dựng đồ thị, còn phần cài đặt chỉ ở mức đủ dùng. Nếu chỉ nhớ một điều: khi đề bài có “sức chứa”, “mỗi thứ dùng một lần”, hoặc “chọn một trong hai phía sao cho ít mâu thuẫn nhất”, hãy nghĩ tới luồng.

KIẾN THỨC CẦN CÓ TRƯỚC

- **mạng luồng** — đồ thị có hướng với một đỉnh nguồn, một đỉnh đích, mỗi cạnh có sức chứa. Luồng là cách gán giá trị cho các cạnh sao cho không vượt sức chứa và mọi đỉnh trung gian có lượng vào bằng lượng ra.
- **đồ thị thặng dư** — đồ thị phụ ghi lại phần sức chứa còn lại và các cạnh ngược cho phép “rút lại” luồng đã đẩy — ý tưởng then chốt làm thuật toán đúng.
- **đường tăng luồng** — đường từ nguồn tới đích trong đồ thị thặng dư; còn đường như vậy thì luồng còn tăng được.
- **lát cắt** — cách chia tập đỉnh thành hai phần, một chứa nguồn một chứa đích; giá của lát cắt là tổng sức chứa các cạnh đi từ phần nguồn sang phần đích.
- **ghép đôi** — tập cạnh đôi một không chung đỉnh. Ghép đôi cực đại trên đồ thị hai phía quy về luồng cực đại với mọi sức chứa bằng 1.
- **phủ đỉnh** — tập đỉnh chạm tới mọi cạnh. Trên đồ thị hai phía, phủ đỉnh nhỏ nhất bằng ghép đôi lớn nhất — định lý König.

CÂU HỎI MỞ ĐẦU

- Q1. Vì sao luồng cực đại lại đúng bằng lát cắt nhỏ nhất — hai bài toán trông không liên quan gì tới nhau?
- Q2. Cạnh ngược trong đồ thị thặng dư cho phép “hoàn tác” luồng đã đẩy. Nếu bỏ chúng đi thì thuật toán hỏng ở đâu?
- Q3. Bài phân công n người vào n việc không nhắc gì tới dòng chảy. Vì sao nó lại là bài luồng?
- Q4. Ford-Fulkerson có thể không dừng với sức chứa vô tỉ. Điều đó nói gì về vai trò của “đại lượng giảm” trong chứng minh tính đúng?
- Q5. Vì sao trong phân đoạn ảnh, người ta lại đi tìm lát cắt nhỏ nhất chứ không tìm luồng?

Chương trước kết thúc bằng nhận xét rằng đường đi ngắn nhất hỏi về *một* đường. Nhưng rất nhiều bài toán thực tế hỏi về nhiều đối tượng cùng lúc, cạnh tranh nhau một nguồn lực có hạn: nhiều gói tin cùng chạy qua một mạng có băng thông giới hạn, nhiều người cùng được phân vào nhiều việc mà mỗi việc chỉ nhận một người, nhiều đường ống cùng dẫn nước qua các van có sức chứa khác nhau.

Mô hình luồng ra đời cho đúng lớp bài toán đó, và điều làm nó có giá trị vượt xa mục đích ban đầu là một hiện tượng lặp đi lặp lại: những bài toán trông chẳng liên quan gì tới dòng chảy hoá ra lại có cùng cấu trúc. Vì vậy chương này sẽ dành phần lớn trọng lượng cho câu hỏi *nhận diện và dựng mô hình*, và chỉ nói về thuật toán ở mức đủ để bạn chọn đúng và cài được.

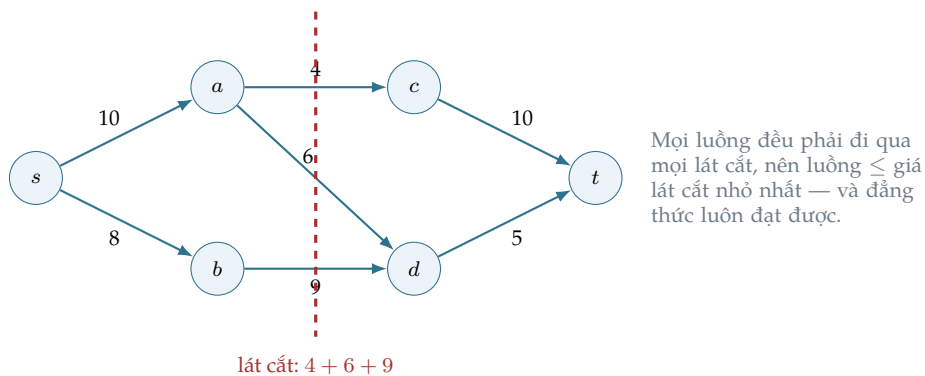
6.1 Luồng, lát cắt, và một sự trùng khớp đáng kinh ngạc

Bài toán: cho mạng với sức chứa trên mỗi cạnh, đẩy nhiều nhất bao nhiêu đơn vị từ nguồn tới đích?

Thuật toán cơ bản rất trực giác: tìm một đường từ nguồn tới đích còn chỗ trống, đẩy thêm luồng dọc đường đó, lặp lại. Điều duy nhất không hiển nhiên — và là ý tưởng then chốt của cả lý thuyết — là mỗi khi đẩy x đơn vị qua một cạnh, ta phải thêm một *cạnh ngược* sức chứa x vào đồ thị thặng dư. Cạnh ngược cho phép các bước sau “rút lại” quyết định cũ nếu nó hoá ra không tối ưu.

Vì sao cần? Vì nếu không có nó, thuật toán trở thành tham lam không hồi tiếc, và như chương C/15 đã chỉ ra, tham lam sai khi lựa chọn hôm nay phá huỷ khả năng tương lai. Đường đi đầu tiên có thể chiếm mất một cạnh hẹp mà lẽ ra nên để dành cho luồng khác. Cạnh ngược biến thuật toán tham lam thành thuật toán có khả năng tự sửa, và đó chính xác là điều làm nó tối ưu.

Kết quả trung tâm là *định lý luồng cực đại bằng lát cắt nhỏ nhất*: giá trị luồng lớn nhất đúng bằng giá của lát cắt nhỏ nhất. Chiều dễ hiểu ngay: mọi luồng đều phải đi qua mọi lát cắt, nên luồng không thể vượt giá của lát cắt nào. Chiều khó là chứng minh tồn tại một lát cắt *đạt* giá trị đó, và nó có một chứng minh rất đẹp: khi thuật toán dừng vì không còn đường tăng, hãy lấy tập đỉnh còn tới được từ nguồn trong đồ thị thặng dư — tập đó chính là một lát cắt có giá bằng luồng hiện tại.



Hình 6.1. Một mạng luồng và một lát cắt. Lát cắt vừa là chặn trên cho luồng vừa là *lời giải* của nhiều bài toán mô hình hoá (mục 4). Khi thuật toán dừng, tập đỉnh còn tới được từ s trong đồ thị thặng dư cho đúng một lát cắt đạt giá trị luồng.

TRỰC GIÁC

Vì sao một bài toán tối đa lại bằng một bài toán tối thiểu? Đây là hiện tượng *đối ngẫu*, và nó phổ biến hơn nhiều người nghĩ. Trực giác: luồng là “đẩy được bao nhiêu”, lát cắt là “chặn tồn ít nhất bao nhiêu”. Mỗi lát cắt là một *bằng chứng* chặn trên cho luồng; nếu tìm được lát

cắt có giá bằng đúng luồng hiện tại thì hai bên kẹp nhau và cả hai đều tối ưu. Đây là lý do luồng có tính chất hiếm: nó không chỉ cho lời giải mà còn cho *chứng cứ kiểm tra được* rằng lời giải là tối ưu. Cùng hiện tượng xuất hiện trong quy hoạch tuyến tính, trong lý thuyết trò chơi, và trong nhiều bài tối ưu tổ hợp khác.

6.2 Ba thuật toán và giả thiết của chúng

Ford–Fulkerson là khung chung: cứ tìm đường tăng bất kỳ. Chi phí phụ thuộc cách chọn đường: với sức chứa nguyên, mỗi vòng tăng ít nhất một đơn vị nên số vòng bị chặn bởi giá trị luồng — đó là *đại lượng giảm* theo nghĩa chương A/04. Nhưng nếu sức chứa là số vô tỉ thì các bước tăng có thể nhỏ dần vô hạn và thuật toán *không dừng*. Đây là một ví dụ hiếm và rất đáng nhớ về việc chứng minh tính dừng phụ thuộc vào giả thiết số học chứ không chỉ vào cấu trúc.

Edmonds–Karp chỉ định thêm một quy tắc: luôn chọn đường tăng *ngắn nhất* bằng duyệt rộng. Điều đó cho chặn $O(nm^2)$ độc lập với sức chứa, và giải quyết luôn vấn đề tính dừng. Bài học phương pháp rất đáng chú ý: một thuật toán không có đảm bảo tốt trở nên có đảm bảo chỉ nhờ *cổ định thứ tự lựa chọn*.

Dinic nhanh hơn nhiều và là lựa chọn thực tế: nó xây đồ thị phân tầng bằng duyệt rộng, rồi đẩy luồng chặn trên đồ thị đó bằng duyệt sâu. Chi phí $O(n^2m)$ nói chung, và $O(m\sqrt{n})$ trên mạng đơn vị của bài ghép đôi hai phía. Chỉ giả thiết mọi sức chứa bằng 1 chưa đủ để suy ra chặn này trên mạng tổng quát; cấu trúc vào/ra của các đỉnh trung gian cũng quan trọng[19]. Trong thực hành thi đấu, *Dinic* gần như luôn là lựa chọn đúng.

6.3 Ghép đôi hai phía và ba định lý anh em

Trường hợp riêng quan trọng nhất: đồ thị hai phía, mọi sức chứa bằng 1. Bài toán ghép đôi cực đại — phân công nhiều người vào nhiều việc sao cho số cặp lớn nhất — quy về luồng bằng cách thêm nguồn nối tới mọi đỉnh trái và đích nhận từ mọi đỉnh phải.

Ba định lý đi kèm, và chúng đáng thuộc vì chúng biến bài toán này thành bài toán khác:

Định lý König: trên đồ thị hai phía, ghép đôi lớn nhất bằng phủ đỉnh nhỏ nhất. Hệ quả: bài “chọn ít đỉnh nhất chạm mọi cạnh” — vốn NP-khó trên đồ thị tổng quát — lại giải được trong thời gian đa thức trên đồ thị hai phía.

Hệ quả về tập độc lập: tập độc lập lớn nhất bằng n trừ ghép đôi lớn nhất. Rất nhiều bài “chọn nhiều ô nhất trên bàn cờ sao cho không có hai ô xung khắc” rơi vào đúng đây.

Định lý Hall: tồn tại ghép đôi bão hoà phía trái khi và chỉ khi mọi tập con S của phía trái có số láng giềng ít nhất bằng $|S|$. Đây là công cụ để *chứng minh* một phân công là bất khả thi, chứ không chỉ để tìm phân công.

Với bài phân công có *chi phí* — mỗi người làm mỗi việc tốn một giá khác nhau, tìm phân công tổng chi phí nhỏ nhất — ta cần luồng chi phí nhỏ nhất hoặc thuật toán Hungarian[20]. Với $n \leq 20$, quy hoạch động mặt nạ bit (C/16) thường ngắn hơn và đủ nhanh; với n lớn hơn thì phải dùng thuật toán chuyên.

6.4 Mô hình hoá: nơi chương này thật sự trả công

Đây là mục quan trọng nhất. Dưới đây là các mẫu dựng mô hình hay gặp, kèm dấu hiệu nhận biết trên đề.

Mẫu 1 — phân công. Dấu hiệu: hai loại đối tượng, mỗi phần tử một loại ghép với một phần tử loại kia. Đồ thị hai phía, sức chứa 1.

Mẫu 2 — ràng buộc ở đỉnh. Dấu hiệu: “mỗi thành phố chỉ đi qua một lần”, “mỗi máy xử lý tối đa k việc”. Tách đỉnh thành hai nửa nối bởi một cạnh có sức chứa bằng đúng giới hạn đó — kỹ thuật đã nhắc ở C/17 và ở đây nó thành công cụ chính.

Mẫu 3 — chọn dự án có lợi nhất. Dấu hiệu: có các dự án cho lợi nhuận, mỗi dự án đòi hỏi một số thiết bị tốn chi phí, thiết bị dùng chung được. Đây là bài đóng lớn nhất, và nó quy về lát cắt nhỏ nhất: nối nguồn tới dự án với sức chứa bằng lợi nhuận, nối thiết bị tới đích với sức chứa bằng chi phí, nối dự án tới thiết bị nó cần với sức chứa vô hạn. Lợi nhuận lớn nhất bằng tổng lợi nhuận trừ lát cắt nhỏ nhất.

Mẫu 4 — chia hai phía sao cho ít mâu thuẫn nhất. Dấu hiệu: mỗi đối tượng phải được gán một trong hai nhãn; gán sai nhãn cho một đối tượng tốn một giá; hai đối tượng kề nhau mà khác nhãn cũng tốn một giá. Đây chính là mô hình lát cắt nhỏ nhất, và nó là nền của phân đoạn ảnh bằng cắt đồ thị.

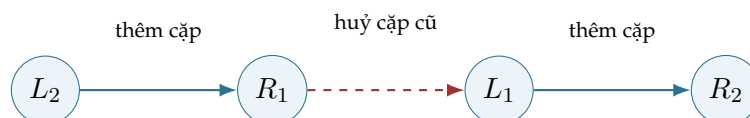
Mẫu 4 đáng được nói kỹ vì nó là cầu nối đẹp giữa chương này và thị giác máy tính. Khi ta muốn tách vật thể khỏi nền trong một bức ảnh, mỗi điểm ảnh phải nhận nhãn “vật” hoặc “nền”. Mỗi điểm ảnh có một chi phí gán nhãn dựa trên màu của nó, và hai điểm ảnh kề nhau có một chi phí nếu chúng nhận nhãn khác nhau — phần này thể hiện giả định rằng vùng ảnh thường liền mạch. Tổng chi phí đúng bằng giá của một lát cắt trong đồ thị mà đỉnh là điểm ảnh, nên tìm cách gán nhãn tốt nhất chính là tìm lát cắt nhỏ nhất. Đây là lý do câu hỏi mở đầu thứ năm có câu trả lời: người ta quan tâm tới lát cắt vì lát cắt chính là lời giải, còn luồng chỉ là phương tiện tính ra nó.

CẠM BẮY

Cạm bẫy phổ biến nhất khi dùng luồng không phải cài sai thuật toán mà là *dựng sai mô hình*, và cái sai đó im lặng: chương trình chạy, cho ra một con số, và con số đó trả lời một bài toán khác. Hai cách phòng. Thứ nhất, sau khi dựng xong, hãy kiểm tra bằng lời: “một đơn vị luồng trong mô hình này tương ứng với cái gì trong bài toán thật?” — nếu không trả lời được bằng một câu thì mô hình sai. Thứ hai, stress test với vết cặn trên dữ liệu rất nhỏ ($E/25$), vì mô hình sai gần như luôn lộ ra ở $n \leq 5$.

6.5 Cạnh ngược là quyền sửa một quyết định cũ

Cho hai người L_1, L_2 và hai việc R_1, R_2 ; các cặp hợp lệ là $L_1 R_1, L_1 R_2, L_2 R_1$. Nếu ghép $L_1 R_1$ trước, L_2 chưa có việc. Hình 6.2 cho thấy đường tăng dùng cạnh ngược để sửa cặp cũ.



Hình 6.2. Đường tăng xen kẽ $L_2 \rightarrow R_1 \rightarrow L_1 \rightarrow R_2$, lược bỏ nguồn và đích. Cạnh nét đứt là cạnh dư ngược của cặp đang ghép. Kết quả có hai cặp $L_2 R_1, L_1 R_2$, tăng từ 1 lên 2.

Khi đẩy Δ qua cạnh dư $u \rightarrow v$, giảm sức chứa dư cạnh đó và tăng cạnh đối ứng cùng lượng. Chỉ xoá bớt sức chứa chiều thuận sẽ biến thuật toán thành lựa chọn không thể sửa và có thể dừng ở luồng chưa cực đại. Trong cài đặt, lưu chỉ số cạnh đối ứng; hai cạnh gốc ngược chiều vẫn phải có các cặp cạnh dư riêng để xử lý chính xác đa cạnh. Sau khi chạy, kiểm sức chứa, bảo toàn luồng ở đỉnh trung gian và giá trị lát cắt từ các đỉnh còn tới được trong mạng dư.

6.6 Biên giới hiện nay: từ đường tăng tới tối ưu hoá liên tục

Dinic là lựa chọn thực hành, nhưng câu hỏi “luồng cực đại thật sự tốn bao nhiêu” có một lịch sử dài và vừa có bước ngoặt, và bước ngoặt ấy đáng kể lại vì nó đến từ một hướng hoàn toàn khác.

Suốt nửa thế kỷ, mọi cải tiến đều nằm trong khuôn tổ hợp: vẫn là đường tăng, luồng chặn, đẩy và gán nhãn, chỉ khác ở cách chọn và cách tổ chức dữ liệu. Đỉnh của dòng ấy là kết quả của Orlin[21], đưa luồng cực đại về $O(nm)$ cho mọi cặp n, m — con số mà nhiều người xem là điểm dừng tự nhiên của lối tiếp cận tổ hợp.

Bước ngoặt đến khi người ta ngừng coi luồng là bài toán tổ hợp. Nhìn từ tối ưu hoá, tìm luồng cực đại là giải một bài quy hoạch tuyến tính có cấu trúc rất đặc biệt, và người ta có thể đi tới lời giải bằng những bước *liên tục* — mỗi bước giải một bài toán dòng điện trên mạng, rồi sửa dần. Chen, Kyng, Liu, Peng, Probst Gutenberg và Sachdeva[22] hoàn thiện hướng này: luồng cực đại và luồng chi phí nhỏ nhất trên đồ thị có hướng với sức chứa và chi phí nguyên bị chặn đa thức, giải được trong thời gian $m^{1+o(1)}$ — *gần tuyến tính*, tức là chỉ hơn chi phí đọc dữ liệu vào một thừa số dưới mọi lũy thừa. Bài được trao giải bài báo xuất sắc tại FOCS 2022.

Hai điều đáng học ở đây, và không điều nào là “hãy cài thuật toán này”.

Thứ nhất, về phương pháp: bước nhảy có được nhờ *đổi ngôn ngữ* — rời khuôn tổ hợp sang khuôn tối ưu hoá liên tục — chứ không nhờ tinh chỉnh khuôn cũ. Và điều mĩ mai đẹp đẽ là cái làm cho hướng liên tục chạy được lại chính là cấu trúc dữ liệu động: mỗi bước liên tục chỉ rẽ nếu ta duy trì được thông tin về đồ thị đang thay đổi mà không tính lại từ đầu, đúng loại công việc của phần B. Lý thuyết thuật toán hiện đại có hình dạng như vậy: tối ưu hoá liên tục ở tầng trên, cấu trúc dữ liệu ở tầng dưới.

Thứ hai, về cách đọc kết quả: $m^{1+o(1)}$ giấu một thừa số dưới đa thức có thể rất lớn ở kích thước thật, nên trong mọi kỳ thi và gần như mọi hệ thống, Dinic vẫn là lựa chọn đúng, và ghép đôi hai phía vẫn nên dùng Hopcroft–Karp[23] với $O(m\sqrt{n})$. Nhưng kết quả này đổi một điều quan trọng cho *người mô hình hoá*: phát biểu lại bài toán của mình thành luồng chi phí nhỏ nhất giờ đây là một phép quy dẫn “gần như miễn phí” về mặt lý thuyết, nên rào cản còn lại chỉ là mô hình có đúng hay không — đúng thông điệp của mục 4.

6.7 Gốc rễ: mạng đường sắt Liên Xô và một bài toán về hôn nhân

Bài toán luồng cực đại có một nguồn gốc rất cụ thể và ít người biết. Năm 1955, trong bối cảnh Chiến tranh Lạnh, hai nhà nghiên cứu tại RAND Corporation là Harris và Ross viết một báo cáo phân tích năng lực vận chuyển của mạng đường sắt Liên Xô và các nước Đông Âu. Bài toán họ quan tâm có hai mặt: mạng này chuyển được tối đa bao nhiêu hàng, và — mặt được nhấn mạnh hơn — *cần cắt những tuyến nào, tốn ít nhất bao nhiêu, để chặn dòng vận chuyển*. Nghĩa là bài toán lát cắt nhỏ nhất được đặt ra như một bài toán quân sự, đồng thời với bài toán luồng cực đại.

Ford và Fulkerson, cũng tại RAND, đưa ra thuật toán và chứng minh định lý luồng cực đại bằng lát cắt nhỏ nhất vào năm 1956. Sự trùng khớp giữa hai bài toán — một của bên vận chuyển, một của bên phá hoại — vì thế không phải phát hiện lý thuyết thuần túy mà là câu trả lời cho một

câu hỏi thực tế được đặt ra từ cả hai phía cùng lúc. Đây là ví dụ mẫu mực cho việc bối cảnh sinh ra khái niệm: hai nửa của định lý đối ngẫu ứng với hai bên của một tình huống thật.

Nhánh ghép đôi thì cổ hơn và đến từ toán học thuần túy. König chứng minh định lý về ghép đôi và phủ đỉnh trên đồ thị hai phía năm 1931; Hall đưa ra điều kiện tồn tại ghép đôi bão hoà năm 1935, trong một bài toán thường được kể dưới cái tên “định lý hôn nhân”. Cả hai kết quả ra đời trước khi có máy tính và không nhằm mục đích thuật toán nào. Đến năm 1955, Kuhn kết hợp các ý tưởng của König và Egerváry thành một thuật toán cho bài toán phân công — và ông đặt tên nó là “phương pháp Hungary” để ghi công hai nhà toán học Hungary đó[20], một cử chỉ hiếm thấy trong việc đặt tên thuật toán.

Cuối cùng, một liên hệ hiện đại đáng chú ý: đầu những năm 2000, giới thị giác máy tính phát hiện ra rằng bài toán gán nhãn ảnh với ràng buộc trơn có thể giải *chính xác* bằng lát cắt nhỏ nhất, thay vì bằng các phương pháp lặp không có đảm bảo. Điều này biến một thuật toán vận trù học của thập niên 1950 thành công cụ chủ lực cho phân đoạn ảnh và cho ước lượng độ sâu từ ảnh lập thể trong suốt một thập kỷ, cho tới khi học sâu thay đổi bức tranh.

Gốc rễ: ý tưởng này ra đời từ bài toán nào

Mốc	Bài toán thúc ép lúc đó	Điều thay đổi sau đó
1931 — König	Quan hệ giữa ghép đôi và phủ đỉnh trên đồ thị hai phía.	Định lý König; biến bài phủ đỉnh trên đồ thị hai phía thành bài đa thức.
1935 — Hall	Khi nào thì ghép được mọi phần tử của một phía?	Điều kiện Hall; công cụ để chứng minh <i>bất khả thi</i> , không chỉ để tìm lời giải.
1955 — Harris & Ross (RAND)	Năng lực mạng đường sắt Liên Xô, và cần cắt tuyến nào để chặn.	Bài toán luồng cực đại và lát cắt nhỏ nhất được đặt ra cùng lúc, từ hai phía của một tình huống thật.
1955 — Kuhn	Bài toán phân công người vào việc với chi phí.	Phương pháp Hungary, dựa trên kết quả của König và Egerváry.
1956 — Ford & Fulkerson	Cần thuật toán và cần biết khi nào dừng được.	Thuật toán đường tăng và định lý luồng cực đại bằng lát cắt nhỏ nhất — luồng vừa cho lời giải vừa cho chứng cứ tối ưu.
1970–72 — Dinic; Edmonds & Karp	Ford–Fulkerson có thể rất chậm hoặc không dừng.	Cổ định thứ tự chọn đường (ngắn nhất trước) cho chặn độc lập với sức chứa; Dinic thêm đồ thị phân tầng, cho $O(m\sqrt{n})$ trên mạng đơn vị.
Đầu thập niên 2000 — thị giác máy tính	Gán nhãn điểm ảnh với ràng buộc trơn, các phương pháp lặp không có đảm bảo.	Cắt đồ thị cho lời giải <i>chính xác</i> cho một lớp bài năng lượng; luồng trở thành công cụ chủ lực của phân đoạn ảnh và ảnh lập thể suốt một thập kỷ.

6.8 Liên hệ ngang

Liên hệ ngang: ý này còn sống ở đâu nữa

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu
Thị giác máy tính	Cắt đồ thị cho phân đoạn ảnh và ảnh lập thể	Mô hình mẫu 4 ở mục 4: chi phí gán nhãn cộng chi phí khác nhau giữa hai điểm kề. Điểm đáng nhớ: đây là một trong số ít bài năng lượng trong thị giác máy tính có lời giải <i>tối ưu toàn cục</i> trong thời gian đa thức.

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu (tiếp)
Quy hoạch tuyến tính	Đối ngẫu: bài cực đại và bài cực tiểu có cùng giá trị tối ưu	Luồng–lát cắt là một trường hợp riêng đẹp của đối ngẫu tuyến tính. Ý tưởng chung rất mạnh: nghiệm của bài đối ngẫu là <i>chứng cứ</i> cho tính tối ưu của bài gốc.
Kinh tế học	Bài toán vận tải, phân bổ nguồn lực, thị trường ghép cặp	Cùng mô hình; ghép cặp ổn định (bệnh viện–bác sĩ nội trú, trường–học sinh) là họ hàng gần và đã được dùng trong các hệ thống phân bổ thật.
Mạng máy tính	Bảng thông, định tuyến đa đường, độ tin cậy mạng	Bài toán gốc. Số đường đi rời cạnh giữa hai đỉnh bằng luồng cực đại với mọi sức chứa bằng 1 — định lý Menger, một cách phát biểu khác của cùng kết quả.
Thể thao, lập lịch	Đội nào còn cơ hội vô địch	Bài toán loại trừ: dựng mạng phân phối các trận còn lại; nếu luồng không bão hoà thì đội đang xét chắc chắn bị loại. Ví dụ điển hình cho việc bài toán không có “dòng chảy” nào vẫn là bài luồng.
Sản xuất	Lập lịch máy, phân công ca	Ràng buộc ở đỉnh (mỗi máy tối đa k việc) xử lý bằng tách đỉnh; ràng buộc cận dưới (mỗi ca phải có ít nhất 2 người) xử lý bằng luồng có cận dưới.
SLAM và robot	Chọn tập ràng buộc nhất quán, phân công nhiệm vụ cho đội robot	Phân công nhiệm vụ nhiều robot là bài ghép đôi có trọng số — đúng bài toán Hungarian. Việc chọn tập quan sát nhất quán thì gần với các bài chọn tập con hơn (C/15), nhưng khi ràng buộc là “mỗi điểm mốc ghép với đúng một quan sát” thì lại quay về ghép đôi.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì mỗi lát cắt là một *chặn trên* cho mọi luồng — luồng nào cũng phải đi qua nó — nên luồng cực đại không vượt lát cắt nhỏ nhất. Chiều còn lại được chứng minh bằng cách xây dựng: khi thuật toán dừng vì không còn đường tăng, tập các đỉnh còn tới được từ nguồn trong đồ thị thặng dư tạo thành một lát cắt có giá đúng bằng luồng hiện tại. Hai bên kẹp nhau nên bằng nhau. Đây là hiện tượng đối ngẫu, và ý nghĩa thực dụng là luồng cho ta cả lời giải lẫn chứng cứ kiểm tra được rằng nó tối ưu.
- A2. Thuật toán trở thành tham lam không hối tiếc và sẽ cho kết quả dưới mức tối ưu. Đường tăng đầu tiên có thể chiếm một cạnh hẹp mà lẽ ra nên dành cho luồng khác, và không có cách nào sửa. Cạnh ngược cho phép các bước sau rút lại một phần quyết định cũ, biến thuật toán thành có khả năng tự sửa — đúng thứ mà chương C/15 nói là điều kiện để tham lam đúng, chỉ ở đây ta đạt được bằng cách thêm khả năng hối tiếc thay vì bằng chứng minh.
- A3. Vì bài phân công có đúng cấu trúc “mỗi bên dùng một lần”: dựng đồ thị hai phía, nguồn nối tới mọi người với sức chứa 1, mỗi người nối tới các việc họ làm được, mỗi việc nối tới đích với sức chứa 1. Sức chứa 1 trên các cạnh từ nguồn và tới đích chính là cách mã hoá ràng buộc “mỗi người một việc, mỗi việc một người”. Một đơn vị luồng tương ứng với một cặp được ghép — và câu này chính là bài kiểm tra ở khối *Cạm bẫy*.
- A4. Nó cho thấy tính dừng không phải là hệ quả tự nhiên của việc “mỗi bước đều cải thiện”. Với sức chứa nguyên, mỗi bước tăng ít nhất một đơn vị nên có một đại lượng nguyên không âm giảm — và đó là chứng minh dừng. Với sức chứa vô tỉ, các bước tăng có thể nhỏ dần theo cấp số nhân và tổng hội tụ về một giá trị nhỏ hơn tối ưu, nên thuật toán chạy mãi. Đại lượng giảm phải bị chặn dưới bởi một lượng dương cố định, không chỉ đơn thuần là “giảm”.
- A5. Vì trong bài phân đoạn ảnh, *lát cắt chính là lời giải*: nó nói mỗi điểm ảnh thuộc phía nào, tức là được gán nhãn gì. Luồng chỉ là phương tiện để tính ra lát cắt nhỏ nhất, và giá trị luồng chỉ là tổng chi phí. Đây là một chỗ đáng nhớ về mặt tư duy: khi mô hình hoá bằng luồng, hãy luôn hỏi bạn cần *giá trị* hay cần *cấu hình* — và nếu cần cấu hình thì nó thường nằm ở phía lát cắt.

NÓI LẠI THẬT ĐƠN GIẢN

Hãy hình dung một hệ thống ống nước: một bể nguồn, một bể đích, và nhiều đoạn ống nối nhau, mỗi đoạn chịu được một lưu lượng tối đa. Câu hỏi là bơm được nhiều nhất bao nhiêu nước từ nguồn tới đích.

Cách làm rất tự nhiên: tìm một đường ống còn chỗ trống từ nguồn tới đích, bơm thêm nước vào đó, rồi lặp lại. Chỉ có một mẹo phải nhớ, và đó là điểm tinh tế duy nhất của cả chương: mỗi khi đã bơm nước qua một đoạn ống, hãy ghi nhớ rằng ta *có quyền rút lại* phần nước đó nếu về sau thấy nên dùng ống ấy cho luồng khác. Không có quyền rút lại thì lần bơm đầu tiên có thể lỡ chiếm mất một đoạn ống hẹp quan trọng, và ta bị kẹt ở kết quả không tốt nhất.

Điều đẹp nhất của bài toán này là một sự trùng khớp. Nếu bạn hỏi ngược lại — “muốn chặn hoàn toàn dòng nước thì phải cắt những đoạn ống nào cho ít tốn kém nhất” — thì câu trả lời của bài chặn đúng bằng câu trả lời của bài bơm. Trục giác thì rõ: mọi giọt nước đều phải đi qua chỗ hẹp nhất, nên chỗ hẹp nhất quyết định lượng nước. Điều đáng giá là chiều ngược lại cũng đúng, nên khi tìm ra cách chặn rẻ nhất là bạn có luôn bằng chứng rằng không thể bơm nhiều hơn.

Và đây là lý do mô hình này quan trọng hơn hẳn phạm vi ống nước. Bài phân công nhân viên vào công việc chính là bài này với mỗi ống chịu được đúng một đơn vị. Bài tách vật thể khỏi nền trong ảnh cũng là bài này: mỗi điểm ảnh phải chọn một trong hai phía, và “chi phí cắt” chính là chi phí đặt ranh giới. Khi bạn thấy đề bài nói tới sức chứa, hoặc “mỗi thứ dùng một lần”, hoặc “chia hai phía sao cho ít mâu thuẫn nhất” — hãy nghĩ tới ống nước.

Một câu: *Cắt chỗ hẹp nhất và bơm nhiều nhất là hai câu hỏi cho cùng một đáp số.*

Chỗ không rút gọn được: Việc dựng đúng mô hình cho một bài mới vẫn đòi kinh nghiệm; mẫu ở mục 4 chỉ phủ những dạng hay gặp, không phủ hết.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Ý tưởng then chốt của thuật toán	Cạnh ngược trong đồ thị thặng dư: cho phép rút lại luồng đã đẩy, biến tham lam thành thuật toán tự sửa được.
••• Định lý luồng-lát cắt	Luồng cực đại bằng lát cắt nhỏ nhất; lát cắt là chặn trên, và tập đỉnh tới được từ nguồn khi thuật toán dừng cho lát cắt đạt đúng giá trị đó.
••• Dấu hiệu bài luồng trên đề	Có “sức chứa”; “mỗi thứ dùng đúng một lần”; “phân công hai loại đối tượng”; “chia hai phía sao cho ít mâu thuẫn nhất”.
••• Ghép đôi hai phía	Quy về luồng với mọi sức chứa 1; Dinic cho $O(m\sqrt{n})$. Một đơn vị luồng = một cặp được ghép.
••• Định lý König và hệ quả	Trên đồ thị hai phía: ghép đôi lớn nhất = phủ đỉnh nhỏ nhất; tập độc lập lớn nhất = n trừ ghép đôi lớn nhất.
••• Bốn mẫu mô hình hoá	Phân công; tách đỉnh khi ràng buộc ở đỉnh; chọn dự án (bài đóng lớn nhất) qua lát cắt; gán nhãn hai phía với chi phí trơn.
•• Vì sao Ford–Fulkerson có thể không dừng	Với sức chứa vô tỉ, các bước tăng nhỏ dần vô hạn; đại lượng giảm phải bị chặn dưới bởi một lượng dương cố định.
•• Edmonds–Karp	Luôn chọn đường tăng ngắn nhất (duyệt rộng) $\Rightarrow O(nm^2)$, độc lập với sức chứa. Bài học: cố định thứ tự lựa chọn tạo ra đảm bảo.
•• Định lý Hall	Ghép bão hoà phía trái tồn tại khi và chỉ khi mọi tập con S có ít nhất $ S $ láng giềng — dùng để chứng minh <i>bất khả thi</i> .
•• Bài phân công có chi phí	Luồng chi phí nhỏ nhất hoặc Hungarian; với $n \leq 20$ thì quy hoạch động mặt nạ bit thường ngắn hơn ($C/16$).
•• Cạm bẫy lớn nhất	Dựng sai mô hình, và sai im lặng. Kiểm bằng câu “một đơn vị luồng tương ứng với cái gì” và bằng stress test ở $n \leq 5$.
• Định lý Menger	Số đường đi rời cạnh giữa hai đỉnh bằng số cạnh ít nhất phải bỏ để ngắt liên thông — một cách phát biểu khác của định lý luồng-lát cắt.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
• Luồng cực đại tốn bao nhiêu	Dòng tổ hợp dừng ở $O(nm)$ (2013); hướng tối ưu hoá liên tục cộng cấu trúc dữ liệu động cho $m^{1+o(1)}$ (2022). Thực hành vẫn là Dinic, và ghép đôi hai phía vẫn là Hopcroft–Karp.

• • • phải thuộc • • nên nắm • biết là có

I Câu hỏi ôn tập có đáp án (che phần đáp án, tự trả lời trước)

1. Tôi nên cài thuật toán luồng nào cho thi đấu?

| Dinic là một lựa chọn khởi đầu thực dụng. Nó đủ nhanh cho gần như mọi bài có giới hạn hợp lý, đặc biệt mạnh trên mạng ghép đôi hai phía nơi nó cho $O(m\sqrt{n})$. Edmonds–Karp đáng biết về mặt lý thuyết nhưng chậm hơn nhiều trong thực tế. Ford–Fulkerson thuần tuý thì dừng dừng vì chi phí phụ thuộc sức chứa. Hãy có sẵn một bản Dinic đã stress test trong thư viện riêng (A/02).

2. Làm sao xử lý ràng buộc “mỗi đỉnh chỉ đi qua một lần”?

| Tách đỉnh: mỗi đỉnh v thành $v_{\text{vào}}$ và v_{ra} , nối bởi một cạnh sức chứa 1 (hoặc bằng giới hạn của đỉnh). Mọi cạnh vào v nay nối tới $v_{\text{vào}}$, mọi cạnh ra khỏi v xuất phát từ v_{ra} . Kỹ thuật này chuyển ràng buộc từ đỉnh sang cạnh — nơi mô hình luồng biểu diễn được — và nó là một trong hai ba mẹo cần thuộc của chương.

3. Bài của tôi cần mỗi cạnh có cả cận dưới lẫn cận trên. Làm sao?

| Đó là bài luồng có cận dưới, và có một phép biến đổi chuẩn: với mỗi cạnh có cận dưới l , trừ l khỏi sức chứa và bù lại bằng cách thêm luồng cưỡng bức vào hai đầu qua một nguồn và đích phụ. Sau biến đổi, việc tồn tại luồng hợp lệ tương đương với việc mạng phụ bão hoà. Kỹ thuật này không khó nhưng nhiều chi tiết, nên hãy cài một lần cho kỹ và cất vào thư viện.

4. Vì sao bài chọn dự án lại là lát cắt chứ không phải luồng?

| Vì cái ta cần là *tập dự án được chọn*, tức là một cách chia đỉnh thành hai phía — đúng định nghĩa của lát cắt. Cách dựng: nguồn nối tới mỗi dự án với sức chứa bằng lợi nhuận, mỗi thiết bị nối tới đích với sức chứa bằng chi phí, và dự án nối tới thiết bị nó cần với sức chứa vô hạn. Cạnh vô hạn bảo đảm không thể chọn dự án mà không trả tiền thiết bị. Lợi nhuận lớn nhất bằng tổng lợi nhuận trừ giá lát cắt nhỏ nhất.

5. Khi nào bài ghép đôi *không* quy về luồng được?

| Khi đồ thị không phải hai phía. Ghép đôi cực đại trên đồ thị tổng quát cần thuật toán Blossom của Edmonds, phức tạp hơn hẳn và hiếm khi xuất hiện trong thi đấu. Đây cũng là chỗ ranh giới lý thuyết thú vị: nhiều bài dễ trên đồ thị hai phía lại khó hơn hẳn hoặc NP-khó trên đồ thị tổng quát — phủ định là ví dụ rõ nhất (D/23).

6. Tôi dựng mô hình xong nhưng không chắc đúng. Kiểm thế nào?

| Ba bước, theo thứ tự rẻ dần. Một, trả lời bằng lời câu “một đơn vị luồng ứng với gì” và “một lát cắt ứng với gì” — nếu áp ứng thì mô hình sai. Hai, thử tay một ví dụ $n = 2$ hoặc $n = 3$ và so với đáp án tính tay. Ba, stress test với vết cận trên $n \leq 5$. Bước ba tốn mười phút và bắt được gần như mọi lỗi mô hình — rẻ hơn nhiều so với ngồi đọc lại code.

7. Luồng chi phí nhỏ nhất khác luồng cực đại thế nào về mặt tư duy?

| Luồng cực đại hỏi “đẩy được bao nhiêu”; luồng chi phí nhỏ nhất hỏi “đẩy đúng lượng này với tổng chi phí nhỏ nhất”. Thuật toán chuẩn thay việc tìm đường tăng bất kỳ bằng việc tìm đường tăng *rẻ nhất* — tức là chạy Bellman–Ford hoặc Dijkstra với thế năng trên đồ thị thặng dư. Điều cần chú ý: đồ thị thặng dư có cạnh ngược với chi phí âm, nên Dijkstra thuần tuý không dùng được và phải chuyển đổi trọng số (C/17).

I Bài tập (mức: • khởi động • • chuẩn • • • khó)

Bài	Nguồn	Điều cần nhận ra
• Ghép đôi cực đại trên đồ thị hai phía	CSES “School Dance”[1]	Dựng mạng và chạy Dinic; giải thích ý nghĩa của mỗi đơn vị luồng.
•• Luồng cực đại cơ bản	CSES “Download Speed”[1]	Chú ý cạnh song song và cạnh hai chiều.
•• Số đường đi rời cạnh giữa hai đỉnh	CSES “Distinct Routes”[1]	Sức chứa 1; truy vết các đường sau khi có luồng.
•• Phủ đỉnh nhỏ nhất và tập độc lập lớn nhất trên đồ thị hai phía	Bài kinh điển	Dùng định lý König; dựng phủ đỉnh từ ghép đôi bằng cách duyệt xen kẽ.
•• Đặt quân xe trên bàn cờ có ô cấm sao cho không ăn nhau	Bài kinh điển	Hàng và cột là hai phía — một bài mà mô hình hoá là toàn bộ lời giải.
••• Bài chọn dự án có lợi nhuận lớn nhất	Bài kinh điển	Lát cắt nhỏ nhất với cạnh vô hạn; tính lợi nhuận bằng tổng trừ lát cắt.
••• Phân công với chi phí, $n \leq 200$	Bài kinh điển	Luồng chi phí nhỏ nhất hoặc Hungarian; so với quy hoạch động mặt nạ bit ở $n \leq 20$.
••• Đội bóng nào còn cơ hội vô địch	Bài kinh điển	Bài loại trừ thể thao; ví dụ đẹp nhất cho việc bài không có “dòng chảy” nào vẫn là bài luồng.
••• Phân đoạn ảnh nhị phân bằng lát cắt nhỏ nhất	Bài tập ứng dụng	Dựng chi phí gán nhãn và chi phí trơn; chạy trên ảnh nhỏ và xem kết quả bằng mắt.
••• Luồng có cận dưới trên mỗi cạnh	Bài thi đấu kinh điển	Phép biến đổi chuẩn; cài một lần cho kỹ rồi cắt vào thư viện riêng.

Kết chương và đọc tiếp

Luồng là chương mà công sức bỏ ra chủ yếu nằm ở việc *dựng mô hình*, không ở việc cài thuật toán. Ba điều đáng mang theo: cạnh ngược là ý tưởng làm cho thuật toán tự sửa được; định lý luồng–lát cắt cho ta cả lời giải lẫn chứng cứ tối ưu; và bốn mẫu mô hình ở mục 4, đặc biệt mẫu “gán nhãn hai phía với chi phí trơn” vì nó xuất hiện ở những nơi bất ngờ nhất.

Hai chương còn lại của phần C là hai công cụ cắt ngang, dùng được cho mọi chương trước. Chương C/19 nói về một kỹ thuật có tỉ lệ “biến bài khó thành bài dễ” cao nhất trong cả quyển sách: khi bài toán tối ưu khó, hãy đổi nó thành bài toán kiểm tra và tìm nhị phân trên đáp án.

Đọc thêm: công trình gốc (chỉ liệt kê nguồn đã qua phản biện; bản tiền ấn được ghi rõ)

Mức	Công trình và tác giả	Nơi công bố, năm	Vì sao đọc, đọc phần nào
nền tảng	Edmonds & Karp, “Theoretical Improvements in Algorithmic Efficiency for Network Flow Problems”	Journal of the ACM, 1972	Đọc để thấy một thuật toán không có đảm bảo trở nên có đảm bảo chỉ nhờ cố định thứ tự chọn đường tăng.
nền tảng	Hopcroft & Karp, “An $n^{5/2}$ Algorithm for Maximum Matchings in Bipartite Graphs”	SIAM Journal on Computing, 1973	Thuật toán ghép đôi nên dùng trong thực hành; đọc phần lập luận về các pha đường tăng ngắn nhất.
tra khi cần	Orlin, “Max Flows in $O(nm)$ Time, or Better”	STOC, 2013	Điểm dừng của dòng thuật toán tổ hợp; đọc phần mở đầu để có bản đồ lịch sử các cận.
tra khi cần	Chen, Kyng, Liu, Peng, Probst Gutenberg, Sachdeva, “Maximum Flow and Minimum-Cost Flow in Almost-Linear Time”	FOCS, 2022 (Best Paper)	Chỉ cần đọc mục giới thiệu: nó nói rõ vì sao hướng liên tục thắng và cấu trúc dữ liệu động đóng vai trò gì.

Mức	Công trình và tác giả	Nơi công bố, năm	Vì sao đọc (tiếp)
tra khi cần	Ahuja, Magnanti, Orlin, <i>Network Flows</i>	Prentice-Hall, 1993	Sách tra cứu cho các biến thể mô hình hoá: luồng chi phí nhỏ nhất, luồng có cận dưới, bài toán vận tải.

Nguồn

- [1] A. Laaksonen. “CSES Problem Set.” (2026), address: <https://cses.fi/problemset/> (visited on 09/06/2026).
- [2] M. Blum, R. W. Floyd, V. Pratt, et al., “Time Bounds for Selection,” *Journal of Computer and System Sciences*, vol. 7, no. 4, pp. 448–461, 1973.
- [3] V. Strassen, “Gaussian Elimination Is Not Optimal,” *Numerische Mathematik*, vol. 13, pp. 354–356, 1969.
- [4] D. Harvey and J. van der Hoeven, “Integer Multiplication in Time $O(n \log n)$,” *Annals of Mathematics*, vol. 193, no. 2, pp. 563–617, 2021.
- [5] J. Alman and V. Vassilevska Williams, “A Refined Laser Method and Faster Matrix Multiplication,” in *Proceedings of the 32nd ACM-SIAM Symposium on Discrete Algorithms (SODA)*, 2021.
- [6] J. Alman, R. Duan, V. Vassilevska Williams, et al. “Improving the Matrix Multiplication Exponent with Modern Optimization and AlphaEvolve.” bản tiền ấn arXiv:2608.16884, chưa qua phản biện. (2026), address: <https://arxiv.org/abs/2608.16884> (visited on 09/07/2026).
- [7] J. W. Cooley and J. W. Tukey, “An Algorithm for the Machine Calculation of Complex Fourier Series,” *Mathematics of Computation*, vol. 19, no. 90, pp. 297–301, 1965.
- [8] D. R. Musser, “Introspective Sorting and Selection Algorithms,” *Software: Practice and Experience*, vol. 27, no. 8, pp. 983–993, 1997.
- [9] J. L. Bentley and M. D. McIlroy, “Engineering a Sort Function,” *Software: Practice and Experience*, vol. 23, no. 11, pp. 1249–1265, 1993.
- [10] D. E. Knuth, “Optimum Binary Search Trees,” *Acta Informatica*, vol. 1, pp. 14–25, 1971.
- [11] F. F. Yao, “Speed-Up in Dynamic Programming,” *SIAM Journal on Algebraic and Discrete Methods*, vol. 3, no. 4, pp. 532–540, 1982.
- [12] A. Aggarwal, M. M. Klawe, S. Moran, et al., “Geometric Applications of a Matrix-Searching Algorithm,” *Algorithmica*, vol. 2, pp. 195–208, 1987.
- [13] D. S. Hirschberg, “A Linear Space Algorithm for Computing Maximal Common Subsequences,” *Communications of the ACM*, vol. 18, no. 6, pp. 341–343, 1975.
- [14] R. E. Tarjan, “Depth-First Search and Linear Graph Algorithms,” *SIAM Journal on Computing*, vol. 1, no. 2, pp. 146–160, 1972.
- [15] M. L. Fredman and R. E. Tarjan, “Fibonacci Heaps and Their Uses in Improved Network Optimization Algorithms,” *Journal of the ACM*, vol. 34, no. 3, pp. 596–615, 1987.
- [16] M. Thorup, “Undirected Single-Source Shortest Paths with Positive Integer Weights in Linear Time,” *Journal of the ACM*, vol. 46, no. 3, pp. 362–394, 1999.
- [17] R. Duan, J. Mao, X. Mao, et al., “Breaking the Sorting Barrier for Directed Single-Source Shortest Paths,” in *Proceedings of the 57th Annual ACM Symposium on Theory of Computing (STOC)*, Best Paper Award, 2025.
- [18] E. W. Dijkstra, “A Note on Two Problems in Connexion with Graphs,” *Numerische Mathematik*, vol. 1, pp. 269–271, 1959.

- [19] KTH Royal Institute of Technology. “KACTL: KTH Algorithm Competition Template Library.” (2026), address: <https://github.com/kth-competitive-programming/kactl> (visited on 09/06/2026).
- [20] H. W. Kuhn, “The Hungarian Method for the Assignment Problem,” *Naval Research Logistics Quarterly*, vol. 2, no. 1-2, pp. 83–97, 1955.
- [21] J. B. Orlin, “Max Flows in $O(nm)$ Time, or Better,” in *Proceedings of the 45th Annual ACM Symposium on Theory of Computing (STOC)*, 2013, pp. 765–774.
- [22] L. Chen, R. Kyng, Y. P. Liu, et al., “Maximum Flow and Minimum-Cost Flow in Almost-Linear Time,” in *Proceedings of the 63rd IEEE Symposium on Foundations of Computer Science (FOCS)*, Best Paper Award, 2022.
- [23] J. E. Hopcroft and R. M. Karp, “An $n^{5/2}$ Algorithm for Maximum Matchings in Bipartite Graphs,” *SIAM Journal on Computing*, vol. 2, no. 4, pp. 225–231, 1973.
- [24] J. Bentley, *Programming Pearls*, 2nd ed. Addison-Wesley, 2000.
- [25] R. Motwani and P. Raghavan, *Randomized Algorithms*. Cambridge University Press, 1995.

Tìm kiếm trên đáp án (Binary Search on the Answer)

PHỤ THUỘC A/01 cho heuristic “đảo ngược câu hỏi”; A/04 cho bất biến của tìm kiếm nhị phân; B/06 cho các phép quét dùng trong hàm kiểm tra.

TÓM TẮT

Đây là chương có tỉ lệ “biến bài khó thành bài dễ” cao nhất trong quyển sách, và ý tưởng chỉ gồm một câu: thay vì hỏi “đáp án là bao nhiêu”, hãy hỏi “đáp án có đạt được mức x không” — câu thứ hai thường dễ hơn nhiều bậc. Nếu câu trả lời cho câu thứ hai có tính đơn điệu theo x thì tìm nhị phân trên x cho lời giải với chi phí bằng chi phí kiểm tra nhân log. Chương cũng nói về hai họ hàng: tìm kiếm ba phân cho hàm một cực trị, và các phương pháp tìm kiếm cục bộ khi không có cấu trúc đơn điệu nào. Nếu chỉ nhớ một điều: dấu hiệu nhận biết là đề bài có dạng “nhỏ nhất của giá trị lớn nhất” hoặc “lớn nhất của giá trị nhỏ nhất” — gặp cụm từ đó thì gần như chắc chắn nên tìm nhị phân trên đáp án.

KIẾN THỨC CẦN CÓ TRƯỚC

- **vị từ đơn điệu** — hàm $P(x)$ nhận giá trị đúng/sai sao cho nếu $P(x)$ đúng thì $P(y)$ đúng với mọi $y > x$ (hoặc mọi $y < x$). Đây là điều kiện sống còn của cả chương.
- **tìm nhị phân trên đáp án** — nhị phân trên miền giá trị của kết quả, mỗi bước gọi một hàm kiểm tra “có đạt được mức này không”.
- **hàm kiểm tra (predicate)** — hàm trả lời câu hỏi quyết định; nó thường là một phép quét tham lam hoặc một quy hoạch động đơn giản.
- **hàm một cực trị (unimodal)** — hàm tăng rồi giảm (hoặc ngược lại), không có cực trị địa phương nào khác — điều kiện để dùng tìm kiếm ba phân.
- **tìm kiếm ba phân** — thu hẹp khoảng bằng cách so hai điểm bên trong; dùng cho hàm một cực trị, tương tự nhị phân nhưng cho bài tối ưu thay vì bài quyết định.
- **tìm kiếm cục bộ** — lớp phương pháp đi từ một lời giải và cải thiện dần bằng các thay đổi nhỏ; không có bảo đảm tối ưu nhưng dùng được khi không còn cấu trúc nào khác.

CÂU HỎI MỞ ĐẦU

- Q1. Vì sao câu hỏi “đáp án có $\geq x$ không” lại thường dễ hơn hẳn câu hỏi “đáp án là bao nhiêu”?
- Q2. Kỹ thuật này đòi tính đơn điệu. Làm sao kiểm tra tính đơn điệu *trước khi* cài, và nó hỏng thế nào nếu bạn nhầm?
- Q3. Tìm nhị phân trên số thực khác gì trên số nguyên, và vì sao “lặp cho tới khi hai đầu bằng nhau” lại là một lỗi?
- Q4. Vì sao tìm kiếm ba phân cần hàm *một cực trị* chứ không chỉ cần “có cực đại”?
- Q5. Khi bài toán không có tính đơn điệu và cũng không có cấu trúc nào khác, còn lại lựa chọn gì?

Trong danh sách mười hai heuristic ở chương A/01 có một câu: “thay vì hỏi đáp án là bao nhiêu, hãy hỏi với giá trị x cho trước thì đáp án có $\geq x$ không”. Chương này khai thác trọn vẹn câu đó, và lý do nó đáng có một chương riêng là vì hiệu lực bất thường của nó: rất nhiều bài tối ưu trông không có lời giải nào lại có một hàm kiểm tra hiển nhiên chỉ dài năm dòng.

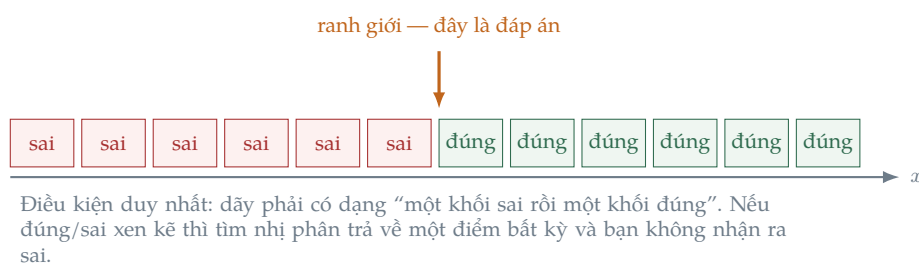
Cơ chế đằng sau khá rõ. Bài toán tối ưu buộc bạn phải *xây dựng* lời giải tốt nhất, tức là vừa phải quyết định cấu hình vừa phải bảo đảm không có cấu hình nào tốt hơn. Bài toán quyết định thì nhẹ hơn nhiều: bạn được cho sẵn ngưỡng, và chỉ cần trả lời có hoặc không — thường bằng một phép quét tham lam, vì khi ngưỡng đã cố định thì mọi lựa chọn trở nên rõ ràng. Đổi bài tối ưu thành bài quyết định là chuyển gánh nặng từ “tìm” sang “kiểm”, và kiểm thì rẻ hơn nhiều.

7.1 Khuôn chuẩn và điều kiện áp dụng

Khuôn gồm ba phần. *Một*: xác định miền giá trị của đáp án — cận dưới và cận trên chắc chắn. *Hai*: viết hàm $P(x)$ trả lời “có phương án nào đạt mức x không”. *Ba*: nhị phân trên x , giữ bất biến “ P sai với mọi giá trị nhỏ hơn lo , đúng với hi ” như ở chương A/04.

Điều kiện sống còn là *tính đơn điệu* của P : nếu đạt được mức x thì cũng đạt được mọi mức dễ hơn. Cách kiểm nhanh và đáng tin: phát biểu P thành lời và hỏi “nếu tôi làm được với ngân sách x , tôi có chắc làm được với ngân sách $x + 1$ không?”. Trong phần lớn bài đúng dạng, câu trả lời là hiển nhiên vì *chính phương án cũ vẫn hợp lệ*. Nếu bạn phải suy nghĩ lâu để trả lời thì rất có thể tính đơn điệu không có, và kỹ thuật này sẽ cho kết quả sai một cách im lặng.

Chi phí là $O(\log(\text{miền giá trị}) \times \text{chi phí kiểm tra})$. Thừa số log hầu như luôn nhỏ — với miền 10^{18} thì chỉ 60 vòng — nên bạn được phép có hàm kiểm tra khá đắt. Đây là điểm đáng nhớ: kỹ thuật này cho phép bạn “mua” một bậc log để đổi lấy một bài toán dễ hơn hẳn.



Hình 7.1. Tìm nhị phân trên đáp án không tìm trên dữ liệu mà tìm trên *miền giá trị của kết quả*. Việc bạn thật sự làm là tìm ranh giới giữa vùng “không đạt được” và vùng “đạt được”. Toàn bộ tính đúng nằm ở giả thiết rằng ranh giới đó tồn tại và chỉ có một.

7.2 Dấu hiệu nhận biết trên đề

Ba dấu hiệu, xếp theo độ tin cậy.

Dấu hiệu thứ nhất, gần như chắc chắn: đề có cụm “nhỏ nhất của giá trị lớn nhất” hoặc “lớn nhất của giá trị nhỏ nhất”. Ví dụ: chia mảng thành k đoạn sao cho tổng lớn nhất của một đoạn là nhỏ nhất; đặt k trạm sao cho khoảng cách xa nhất từ một nhà tới trạm là nhỏ nhất. Dạng “cực tiểu của cực đại” hầu như luôn giải bằng kỹ thuật này, vì hàm kiểm tra “có chia được sao cho mọi đoạn $\leq x$ không” là một phép quét tham lam khi các phần tử không âm và số đoạn được giới hạn từ trên. Với số âm, phải thiết kế lại hàm kiểm tra; từ khoá minimax tự nó không chứng minh rằng kiểm tra là dễ.

Dấu hiệu thứ hai: đề hỏi giá trị tối ưu mà việc *kiểm tra* một giá trị cụ thể lại dễ. Ví dụ: tốc độ ăn nhỏ nhất để kịp thời hạn; kích thước lớn nhất của k miếng cắt bằng nhau.

Dấu hiệu thứ ba: đề hỏi “lớn nhất/nhỏ nhất sao cho một tính chất vẫn giữ được”, và tính chất đó rõ ràng dễ mất đi khi ta đòi hỏi nhiều hơn.

Có một biến thể quan trọng: *nhị phân trên tỉ số*. Với các bài tối ưu tỉ số kiểu “tìm tập con có trung bình lớn nhất”, hãy nhị phân trên giá trị tỉ số x và kiểm tra bằng cách trừ x khỏi mỗi phần tử rồi hỏi có tập con nào có tổng không âm không. Kỹ thuật này biến bài tỉ số — vốn không cộng dồn được — thành bài tổng thông thường, và nó giải cả họ bài “chu trình trung bình nhỏ nhất” hay “đoạn con có trung bình lớn nhất với độ dài tối thiểu”.

7.3 Ba cạm bẫy cài đặt

Cạm bẫy 1 — số thực và điều kiện dừng. Trên số thực, đừng lặp cho tới khi $lo = hi$ vì điều đó có thể không bao giờ xảy ra do sai số biểu diễn. Hai cách đúng: lặp một số vòng cố định — chọn k sao cho $(hi_0 - lo_0)/2^k \leq \varepsilon$ trong số học chính xác — hoặc dừng khi $hi - lo$ nhỏ hơn một ngưỡng đã chọn dựa trên yêu cầu của đề. Số vòng cần thiết phụ thuộc độ rộng ban đầu và sai số yêu cầu. Với dấu phẩy động, còn phải dừng nếu trung điểm bằng một đầu mút do làm tròn; tăng số vòng không vượt qua được độ phân giải của kiểu số.

Cạm bẫy 2 — biên và kiểu số. Miền giá trị phải chứa đáp án: cận dưới thường là 0 hoặc giá trị nhỏ nhất, cận trên phải là một giá trị chắc chắn đạt được. Với dữ liệu lớn, $(lo + hi)/2$ có thể tràn — hãy viết $lo + (hi - lo)/2$. Và kiểm tra kiểu: nếu đáp án tới 10^{18} thì cả biên lẫn giá trị trung gian trong hàm kiểm tra đều phải là 64 bit.

Cạm bẫy 3 — hàm kiểm tra sai một cách tinh vi. Đây là lỗi hay gặp nhất và khó thấy nhất, vì phần nhị phân thì đúng còn phần kiểm tra thì sai ở một trường hợp biên. Cách phòng hiệu quả nhất: viết hàm kiểm tra trước, test riêng nó với vài giá trị x cụ thể mà bạn tính được đáp án bằng tay, rồi mới ghép vào vòng nhị phân.

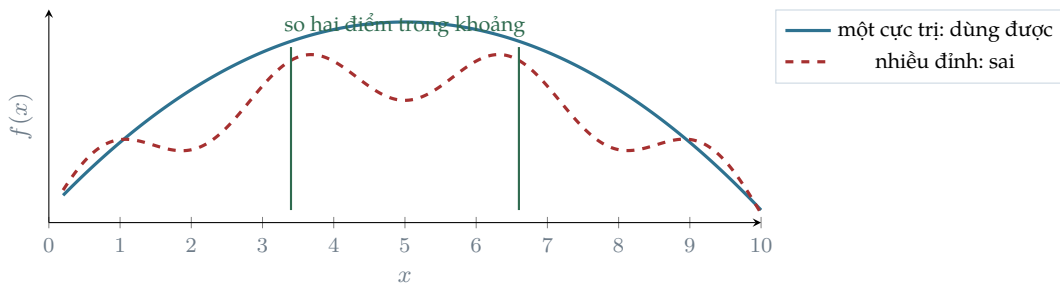
7.4 Tìm kiếm ba phân: khi hàm có một đỉnh

Nếu bài toán không cho một vị từ đơn điệu mà cho một *hàm số* tăng rồi giảm, ta không nhị phân được nhưng vẫn thu hẹp khoảng được. Ý tưởng: lấy hai điểm $m_1 < m_2$ bên trong khoảng; nếu $f(m_1) < f(m_2)$ thì cực đại không nằm bên trái m_1 , nên bỏ đoạn đầu; ngược lại bỏ đoạn cuối.

Điều kiện là hàm phải *một cực trị* — tăng rồi giảm, không có gợn nào khác. Đây là điều kiện mạnh hơn “có cực đại” rất nhiều, và đó là câu trả lời cho câu hỏi mở đầu thứ tư: nếu hàm có nhiều đỉnh địa phương thì phép so sánh hai điểm không còn nói được gì về vị trí cực đại toàn cục, và thuật toán trả về một đỉnh bất kỳ.

Trên miền số nguyên, tìm kiếm ba phân dễ sai ở những bước cuối khi khoảng còn vài phần tử; cách an toàn là thu hẹp tới khi còn dưới ba phần tử rồi thử hết. Trên số thực, dùng số vòng lặp cố định như ở mục trước.

Một biến thể đáng biết cho các bài tối ưu lồi: nếu hàm *lồi* thì có thể nhị phân trên *đạo hàm* rồi rạc $f(i+1) - f(i)$, vốn đơn điệu — và như vậy quay về bài toán nhị phân thông thường. Cách nhìn này nối chương với các tối ưu quy hoạch động ở C/16, nơi tính lồi cũng là điều kiện then chốt.



Hình 7.2. Tìm kiếm ba phân cần hàm *một cực trị*, không chỉ “có cực đại”. Với đường liền, việc so hai điểm bên trong cho biết cực đại nằm về phía nào. Với đường nét đứt nhiều gợn, cùng phép so chỉ nói lên xu hướng *cực bộ*, nên thuật toán hội tụ về một đỉnh bất kỳ.

7.5 Khi không có cấu trúc nào: tìm kiếm cực bộ

Nếu không có tính đơn điệu, không có tính lồi, và không có cấu trúc tổ hợp nào khai thác được, ta rơi vào lớp bài mà không thuật toán chính xác nào chạy nổi. Lúc đó còn một họ phương pháp: bắt đầu từ một lời giải bất kỳ và cải thiện dần bằng các thay đổi nhỏ.

Leo đồi luôn nhận thay đổi làm tốt lên; nó nhanh nhưng kẹt ở cực trị địa phương — đúng thất bại mà chương C/15 đã mô tả cho tham lam. *Mô phỏng luyện kim* thỉnh thoảng chấp nhận cả thay đổi làm tệ đi, với xác suất giảm dần theo thời gian; điều đó cho phép thoát khỏi cực trị địa phương ở giai đoạn đầu và ổn định ở giai đoạn sau. *Khởi động lại nhiều lần* thì đơn giản hơn: chạy leo đồi từ nhiều điểm xuất phát ngẫu nhiên và giữ kết quả tốt nhất.

Cần thẳng thắn về vị trí của nhóm này. Chúng không có bảo đảm chất lượng, kết quả phụ thuộc tham số và may rủi, và trong thi đấu thì hiếm khi là lời giải mong muốn. Nhưng trong công việc thật — lập lịch, bố trí, hiệu chỉnh tham số — chúng thường là thứ duy nhất chạy được trong thời gian cho phép. Nguyên tắc dùng: chỉ chuyển sang nhóm này sau khi đã chứng minh được rằng bài toán không có cấu trúc khai thác được, không phải vì lười tìm.

7.6 Một vết chạy đầy đủ: chia mảng để giảm tổng lớn nhất

Lấy $[7, 2, 5, 10, 8]$, chia thành tối đa hai đoạn không rỗng; mọi phần tử không âm. Miền đáp án là $[10, 32]$. Hàm $P(x)$ quét trái sang phải, chỉ mở đoạn mới khi thêm phần tử kế tiếp làm tổng vượt x . Quét như vậy dùng ít đoạn nhất: mỗi đoạn lấy tiền tố dài nhất còn hợp lệ; không âm bảo đảm phần đuôi không khó hơn khi ta lấy thêm phần tử vào đoạn trước.

Bảng 7.1. Tìm giá trị đúng đầu tiên, với $mid = lo + \lfloor (hi - lo)/2 \rfloor$. Đúng thì $hi = mid$, sai thì $lo = mid + 1$.

lo	hi	mid	Số đoạn	Chia tham lam tại mid
10	32	21	2: đúng	$[7, 2, 5] \mid [10, 8]$
10	21	15	3: sai	$[7, 2, 5] \mid [10] \mid [8]$
16	21	18	2: đúng	$[7, 2, 5] \mid [10, 8]$
16	18	17	3: sai	$[7, 2, 5] \mid [10] \mid [8]$

Bảng 7.1 kết thúc ở $lo = hi = 18$. Hai phép kiểm quan trọng nhất là $P(18)$ đúng và $P(17)$ sai: chúng kiểm ranh giới, không chỉ một điểm ở xa đáp án. Nếu đề yêu cầu đúng k đoạn, điều kiện “tối đa k ” tương đương khi $1 \leq k \leq n$ và các phần tử không âm: có thể tách thêm đoạn mà không tăng tổng lớn nhất.

7.7 Gốc rễ: từ giải phương trình tới ủ kim loại

Tìm kiếm nhị phân là ý tưởng cổ hơn máy tính rất nhiều: nó chính là *phương pháp chia đôi* của giải tích số để tìm nghiệm phương trình, và ý tưởng “giá trị đối dấu giữa hai đầu thì có nghiệm ở giữa” đã được dùng từ lâu trước thế kỷ 20. Nhưng phiên bản rời rạc trên mảng lại có một lịch sử đáng nhớ về sự khó cài: Knuth ghi nhận rằng thuật toán được công bố lần đầu vào năm 1946, nhưng phiên bản đầu tiên xử lý đúng *mọi* kích thước mảng chỉ xuất hiện vào năm 1962 — mười sáu năm cho một thuật toán mà ai cũng nghĩ là hiển nhiên. Cùng với quan sát của Bentley[24] rằng rất nhiều lập trình viên chuyên nghiệp viết sai nó khi phải viết từ đầu, đây là bằng chứng mạnh cho luận điểm ở chương A/04: sự đơn giản của ý tưởng không nói gì về độ khó của việc cài đúng.

Ý tưởng *tìm nhị phân trên đáp án* thì không có tác giả rõ ràng — nó là một kỹ thuật hiển nhiên với người đã quen nhìn bài toán tối ưu như bài toán quyết định. Nhưng cách nhìn đó lại có nguồn gốc lý thuyết rất rõ: trong lý thuyết độ phức tạp, mối quan hệ giữa bài toán tối ưu và bài toán quyết định là quan hệ nền tảng — người ta định nghĩa lớp NP qua bài toán quyết định chính vì lý do đó, và mọi bài tối ưu đều có một phiên bản quyết định đi kèm (D/23). Kỹ thuật của chương này là ứng dụng thực dụng của cùng ý tưởng: nếu bài quyết định dễ hơn thì hãy giải nó và tìm ranh giới.

Mô phỏng luyện kim có một gốc rễ liên ngành đẹp: năm 1983, Kirkpatrick và các cộng sự tại IBM đề xuất nó bằng cách mượn thẳng một hiện tượng vật lý. Khi làm nguội kim loại nóng chảy thật chậm, các nguyên tử có thời gian sắp xếp về cấu hình năng lượng thấp và tinh thể tạo thành đều đẹp; làm nguội nhanh thì kim loại đông cứng ở trạng thái năng lượng cao và có nhiều khuyết tật. Họ nhận ra rằng cùng cơ chế — cho phép hệ tạm thời đi lên mức năng lượng cao hơn, với xác suất giảm dần theo “nhiệt độ” — có thể dùng để thoát khỏi cực trị địa phương trong bài toán tối ưu tổ hợp. Đây là một trong những ví dụ rõ nhất về việc mượn cơ chế từ ngành khác, và nó cũng giải thích tên gọi cùng các tham số của phương pháp.

Gốc rễ: ý tưởng này ra đời từ bài toán nào

Mốc	Bài toán thúc ép lúc đó	Điều thay đổi sau đó
Trước thế kỷ 20 — phương pháp chia đôi 1946–1962 — tìm kiếm nhị phân rời rạc	Tìm nghiệm phương trình khi không có công thức đóng. Tra cứu trong bảng đã sắp.	Ý tưởng nhị phân trên miền liên tục; tổ tiên trực tiếp của cả chương này. Công bố 1946, nhưng bản đúng cho mọi kích thước mảng phải tới 1962 — bằng chứng rằng ý tưởng đơn giản không đồng nghĩa với cài đặt dễ.
Thập niên 1970 — lý thuyết độ phức tạp	Cần một cách chuẩn hoá để so độ khó của các bài toán.	Quan hệ giữa bài tối ưu và bài quyết định trở thành khái niệm nền; kỹ thuật của chương này là ứng dụng thực dụng của nó (D/23).
1983 — Kirkpatrick và cộng sự	Bài tối ưu tổ hợp lớn, leo đồi kẹt ở cực trị địa phương.	Mô phỏng luyện kim: mượn cơ chế làm nguội chậm trong vật lý để cho phép tạm thời đi ngược hướng cải thiện.

7.8 Liên hệ ngang

Liên hệ ngang: ý này còn sống ở đâu nữa

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu
Giải tích số	Phương pháp chia đôi, phương pháp Newton	Cùng ý tưởng thu hẹp khoảng chứa nghiệm. Khác: Newton hội tụ nhanh hơn nhiều nhưng cần đạo hàm và có thể phân kỳ; chia đôi chậm mà chắc — đúng đánh đổi giữa tốc độ và tính bền vững.
Gỡ lỗi phần mềm	git bisect: tìm commit đầu tiên làm hỏng	Đúng thuật toán của chương, áp lên trục thời gian của kho mã. Điều kiện áp dụng cũng giống hệt: tính đơn điệu — trước điểm đó thì tốt, sau điểm đó thì hỏng. Nếu lỗi xuất hiện rồi biến mất rồi lại xuất hiện thì công cụ trả về kết quả vô nghĩa.
Vật lý	Ủ kim loại, hạ nhiệt độ chậm để đạt trạng thái năng lượng thấp	Nguồn gốc trực tiếp của mô phỏng luyện kim, kể cả tên gọi và tham số “nhiệt độ”. Bài học chuyển được: cho phép tạm thời đi ngược hướng cải thiện là cách thoát khỏi cực trị địa phương.
Học máy	Dò tham số, dừng sớm, tìm tốc độ học	Kỹ thuật tìm tốc độ học bằng cách tăng dần và quan sát chính là tìm ranh giới trên một đại lượng gần đơn điệu. Cạm bẫy cũng giống: nếu quan hệ không đơn điệu thì kết luận sai.
Ước lượng robot, SLAM	Chọn ngưỡng loại ngoại lai, hiệu chỉnh tham số	Nhiều tham số ngưỡng có quan hệ đơn điệu với tỉ lệ chấp nhận, nên tìm nhị phân dùng được. Cạm bẫy quen thuộc: quan hệ đó chỉ đơn điệu <i>trung bình</i> , còn trên một tập dữ liệu cụ thể thì có nhiễu — và nhị phân trên hàm nhiễu cho kết quả không ổn định.
Kinh tế học, đấu giá	Tìm giá cân bằng bằng cách dò tăng dần	Cùng cấu trúc: cầu giảm khi giá tăng — một vị từ đơn điệu — nên giá cân bằng tìm được bằng chia đôi. Đây là cơ chế của nhiều thuật toán đấu giá thật.
Tối ưu hoá	Tìm kiếm cục bộ, thuật toán tiến hoá	Cùng họ với mục 5. Nhận xét đáng nhớ: các phương pháp này không có bảo đảm, nên chúng chỉ nên được dùng <i>sau khi</i> đã xác nhận bài toán không có cấu trúc khai thác được.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì bài tối ưu đòi bạn vừa xây dựng cấu hình vừa bảo đảm không có cấu hình nào tốt hơn, còn bài quyết định cho sẵn ngưỡng nên mọi lựa chọn trở nên rõ ràng. Khi đã biết “mỗi đoạn không được vượt x ”, việc chia mảng thành ít đoạn nhất trở thành một phép quét tham lam hiển nhiên: cứ nhồi vào đoạn hiện tại tới khi vượt thì mở đoạn mới. Nói cách khác, ngưỡng cố định đã loại bỏ phần cần cân nhắc, và cái còn lại thường chỉ là một lượt duyệt.
- A2. Kiểm bằng cách phát biểu vị từ thành lời rồi hỏi “nếu làm được với ngân sách x thì có chắc làm được với $x + 1$ không”. Trong các bài đúng dạng, câu trả lời hiển nhiên vì chính phương án cũ vẫn hợp lệ. Nếu nhầm — vị từ không đơn điệu — thì tìm nhị phân vẫn chạy và vẫn trả về một số, nhưng số đó là một điểm chuyển bất kỳ chứ không phải đáp án; đây là lỗi im lặng, và chỉ stress test với vết cạn mới bắt được.
- A3. Trên số thực, phép chia đôi có thể không bao giờ làm hai đầu bằng nhau do sai số biểu diễn, nên vòng lặp treo. Hai cách đúng: lặp một số vòng cố định (100 vòng là quá đủ, vì mỗi vòng giảm khoảng đi một nửa) hoặc dừng khi $hi - lo$ nhỏ hơn một ngưỡng chọn theo yêu cầu độ chính xác của đề. Cách lặp cố định an toàn hơn vì độ chính xác tương đối không phụ thuộc độ lớn của giá trị.
- A4. Vì phép so sánh hai điểm bên trong chỉ cho thông tin về vị trí cực đại nếu hàm không có gợn nào khác. Với hàm nhiều đỉnh, việc $f(m_1) < f(m_2)$ không loại trừ được khả năng cực đại toàn cục nằm bên trái m_1 — nó chỉ nói rằng *quanh đó* hàm đang đi lên. Kết quả là thuật toán hội tụ về một đỉnh địa phương nào đó. “Có cực đại” là điều kiện quá yếu; phải là tăng rồi giảm, không có ngoại lệ.
- A5. Còn nhóm tìm kiếm cục bộ: leo đồi, mô phỏng luyện kim, khởi động lại nhiều lần, các thuật toán tiến hoá. Chúng không có bảo đảm chất lượng và kết quả phụ thuộc tham số, nên chỉ nên dùng sau khi đã xác nhận không còn cấu trúc nào khai thác được. Trong công việc thật — lập

lịch, bố trí, hiệu chỉnh — chúng thường là thứ duy nhất chạy được trong thời gian cho phép, và lúc đó việc cần làm là đo phân phối kết quả thay vì tin vào một lần chạy.

NÓI LẠI THẬT ĐƠN GIẢN

Có một mẹo rất đơn giản mà lại giải được rất nhiều bài khó: thay vì hỏi “đáp án tốt nhất là bao nhiêu”, hãy hỏi “liệu có đạt được mức này không”.

Ví dụ: bạn cần chia một dãy số thành phần liên tiếp sao cho phần nặng nhất càng nhẹ càng tốt. Nghĩ trực tiếp thì rối. Nhưng nếu tôi hỏi “có chia được sao cho mọi phần đều không quá 100 không?” thì bạn trả lời được ngay: cứ nhồi số vào phần hiện tại tới khi sắp vượt 100 thì mở phần mới, cuối cùng đếm xem có quá năm phần không. Chỉ vài dòng.

Bây giờ để ý điều này: nếu chia được với ngưỡng 100 thì chắc chắn cũng chia được với ngưỡng 120 — ngưỡng rộng hơn thì chỉ dễ hơn. Nghĩa là câu trả lời “được / không được” xếp thành một dãy: không, không, không, rồi được, được, được. Đáp án chính là chỗ chuyển từ “không” sang “được”, và tìm chỗ chuyển đó thì đúng là trò đoán số: hỏi ở giữa, rồi thu hẹp một nửa.

Đó là toàn bộ kỹ thuật. Dấu hiệu để nhận ra nó trên đề bài rất dễ nhớ: khi thấy chữ “nhỏ nhất của cái lớn nhất” hoặc “lớn nhất của cái nhỏ nhất”, gần như chắc chắn là nó.

Có đúng một điều phải kiểm tra, và nếu bỏ qua thì chương trình vẫn chạy nhưng ra số sai: dãy trả lời phải thật sự có dạng “một khối không rồi một khối có”. Nếu “được” và “không được” xen kẽ nhau thì trò đoán số vô nghĩa, vì bạn sẽ dừng lại ở một chỗ chuyển ngẫu nhiên nào đó.

Một câu: Đổi câu hỏi “bao nhiêu” thành câu hỏi “có đạt được mức này không”, rồi đoán số trên mức đó.

Chỗ không rút gọn được: Việc chứng minh vị từ đơn điệu vẫn phải làm riêng cho từng bài; không có dấu hiệu bề ngoài nào bảo đảm điều đó.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Ý tưởng cốt lõi	Đổi bài tối ưu thành bài quyết định: “có đạt được mức x không”; ngưỡng cố định làm mọi lựa chọn trở nên rõ ràng, hàm kiểm tra thường chỉ là một lượt quét tham lam.
••• Điều kiện sống còn	Vị từ phải đơn điệu: đạt được x thì đạt được mọi mức dễ hơn. Kiểm bằng câu “phương án cũ có còn hợp lệ với ngân sách rộng hơn không”.
••• Dấu hiệu trên đề	“Nhỏ nhất của giá trị lớn nhất” hoặc “lớn nhất của giá trị nhỏ nhất” — gần như chắc chắn là kỹ thuật này.
••• Chi phí	$O(\log(\text{miền giá trị}) \times \text{chi phí kiểm tra})$; log của 10^{18} chỉ là 60, nên hàm kiểm tra được phép khá đắt.
••• Ba cạm bẫy cài đặt	Số thực: lập số vòng cố định thay vì chờ $lo = hi$. Biên và tràn: dùng $lo + (hi - lo)/2$, kiểu 64 bit. Hàm kiểm tra sai tinh vi: test riêng nó trước khi ghép.
•• Nhị phân trên tỉ số	Bài “trung bình lớn nhất”: nhị phân trên x , trừ x khỏi mỗi phần tử, hỏi có tổng không âm không — biến bài tỉ số thành bài tổng.
•• Tìm kiếm ba phần	Cho hàm một cực trị; so hai điểm trong khoảng để loại một đầu. Nhiều đỉnh địa phương thì sai.
•• Hàm lồi thì nhị phân được	Nhị phân trên đạo hàm rời rạc $f(i+1) - f(i)$, vốn đơn điệu — quay về bài nhị phân thường.
•• Tìm kiếm cục bộ	Leo đồi (kết cực trị địa phương); mô phỏng luyện kim (chấp nhận bước xấu với xác suất giảm dần); khởi động lại nhiều lần.
•• Khi nào dùng tìm kiếm cục bộ	Chỉ sau khi xác nhận bài toán không có cấu trúc khai thác được; không có bảo đảm chất lượng nên phải đo phân phối kết quả.
• Liên hệ với git bisect	Cùng thuật toán trên trục thời gian của kho mã; cùng yêu cầu đơn điệu — lỗi phải xuất hiện từ một điểm và không biến mất.

I Câu hỏi ôn tập có đáp án (che phần đáp án, tự trả lời trước)

1. Nhị phân trên đáp án và nhị phân trên mảng khác nhau thế nào?

| Khác ở *cái được tìm trên*. Nhị phân trên mảng tìm một phần tử trong dữ liệu đã sắp; nhị phân trên đáp án tìm một giá trị trong *miền kết quả*, và dữ liệu có thể hoàn toàn không sắp. Điểm chung là cả hai đều cần một vị từ đơn điệu, và cả hai đều dùng cùng khuôn bất biến ở A/04. Nhận ra chúng là một giúp bạn viết cả hai bằng cùng một mẫu code và ít sai hơn.

2. Tôi nên nhị phân trên số nguyên hay số thực khi đáp án là số thực?

| Nếu đề cho phép sai số, hãy xem có thể nhân lên thành số nguyên không — ví dụ đáp án luôn là bội của 0,001 thì nhân 1000 và làm việc trên số nguyên, tránh hẳn mọi vấn đề sai số. Nếu không, dùng số thực với số vòng lặp cố định. Một mẹo phụ: nhiều bài trông có đáp án thực nhưng đáp án thật sự luôn nằm trong một tập hữu hạn giá trị (một hiệu, một tỉ số của các số nguyên trong dữ liệu) — lúc đó nhị phân trên tập đó cho kết quả chính xác tuyệt đối.

3. Hàm kiểm tra của tôi tốn $O(n \log n)$. Tổng $O(n \log n \log C)$ có quá chậm không?

| Thường là không, và đây là lý do kỹ thuật này mạnh. Với $n = 10^5$ và $C = 10^9$: $10^5 \times 17 \times 30 \approx 5 \cdot 10^7$ — nằm trong ngân sách. Điều cần kiểm là hằng số của hàm kiểm tra: nếu nó cấp phát bộ nhớ hoặc sắp xếp lại từ đầu mỗi lần thì hãy chuyển việc sắp xếp ra *ngoài* vòng nhị phân, vì dữ liệu không đổi giữa các lần kiểm.

4. Làm sao nhận ra tính đơn điệu khi nó không hiển nhiên?

| Ba cách. Một, viết vị từ ra thành lời và tìm một lập luận kiểu “phương án hợp lệ cho x vẫn hợp lệ cho $x + 1$ ”. Hai, tính tay giá trị vị từ cho vài x liên tiếp trên một ví dụ nhỏ và nhìn dãy kết quả. Ba, viết bộ sinh test nhỏ, tính vị từ cho mọi x trong miền, và kiểm bằng máy rằng dãy có dạng “khởi sai rồi khỏi đúng”. Cách ba tốn năm phút và cho câu trả lời chắc chắn — đáng làm khi nghi ngờ.

5. Bài của tôi có hai đại lượng cần tối ưu cùng lúc. Kỹ thuật này còn dùng được không?

| Có, theo hai cách. Nếu hai đại lượng có thứ tự ưu tiên — tối ưu cái thứ nhất trước, cái thứ hai chỉ để phá hoà — thì nhị phân trên cái thứ nhất, rồi trong hàm kiểm tra hãy trả về giá trị tốt nhất của cái thứ hai. Nếu bài yêu cầu tối ưu một tổ hợp tuyến tính có tham số, đó là bài toán tìm kiếm tham số và thường giải bằng nhị phân trên tham số cộng một tính chất lồi — cùng ý tưởng với nhị phân trên tỉ số ở mục 2.

6. Mô phỏng luyện kim có đáng học cho thi đấu không?

| Đáng biết là có, nhưng hiếm khi cần. Nó xuất hiện trong một số kỳ thi cho phép lời giải xấp xỉ và chấm theo điểm chất lượng, và trong các bài tối ưu hình học nhỏ. Trong các kỳ thi chấm đúng/sai thì hầu như không dùng. Ngược lại, trong công việc thật thì nó rất hữu ích — lập lịch, bố trí, hiệu chỉnh tham số — nên nó thuộc loại “học một lần cho biết cách dùng, không cần luyện nhiều”.

7. Có bài nào nhìn giống nhị phân trên đáp án mà thực ra không phải không?

| Có, và dấu hiệu là vị từ không đơn điệu. Ví dụ: “tìm giá trị x sao cho số cách đạt đúng x là lớn nhất” — vị từ ở đây không phải dạng “đạt được hay không” mà là một hàm đếm, và nó có thể lên xuống tùy ý. Một ví dụ khác: các bài mà tăng ngân sách lại làm phương án cũ *không còn hợp lệ* vì có ràng buộc kiểu “phải dùng hết”. Khi thấy chữ “đúng bằng” hoặc “dùng hết” trong đề, hãy dừng lại kiểm tính đơn điệu cẩn thận.

I Bài tập (mức: ● khởi động ●● chuẩn ●●● khó)

Bài	Nguồn	Điều cần nhận ra
● Tìm căn bậc hai nguyên của một số	Bài kinh điển	Nhị phân trên đáp án; chú ý tràn khi bình phương.
● Tốc độ nhỏ nhất để hoàn thành trong h giờ	LeetCode “Koko Eating Bananas”	Bài mẫu nhập môn; viết vị từ ra thành lời trước.

Bài	Nguồn	Điều cần nhận ra (tiếp)
●● Chia mảng thành k đoạn sao cho tổng lớn nhất nhỏ nhất	CSES “Array Division”[1]	Dạng “cực tiểu của cực đại” kinh điển; hàm kiểm tra là một lượt tham lam.
●● Đặt k trạm sao cho khoảng cách xa nhất nhỏ nhất	Bài kinh điển	Cùng dạng; chú ý biên của miền giá trị.
●● Số nhỏ nhất x sao cho có ít nhất k bội số của các số cho trước không vượt x	CSES “Factory Machines”[1]	Nhị phân trên thời gian; cẩn thận tràn khi cộng số lượng.
●● Chuỗi con lặp lại dài nhất	Bài kinh điển	Nhị phân trên độ dài cộng bấm chuỗi ($B/12$); tính đơn điệu ở đây rất rõ.
●●● Đoạn con có trung bình lớn nhất với độ dài tối thiểu k	Bài kinh điển	Nhị phân trên tỉ số: trừ x khỏi mỗi phần tử rồi hỏi có đoạn tổng không âm không.
●●● Chu trình có trung bình trọng số nhỏ nhất	Bài kinh điển	Cùng mẹo tỉ số, kết hợp phát hiện chu trình âm bằng Bellman–Ford ($C/17$).
●●● Tìm điểm cực tiểu của một hàm một cực trị trên số thực	Bài kinh điển	Tìm kiếm ba phân với số vòng cố định; so với cách nhị phân trên đạo hàm rời rạc.
●●● Bài toán tối ưu nhỏ giải bằng mô phỏng luyện kim, so với vét cạn	Bài tập thực nghiệm	Chạy 100 lần và vẽ phân phối kết quả — mục tiêu là thấy tận mắt “không có bảo đảm” nghĩa là gì.
●●● Kiểm tra tính đơn điệu của một vị từ bằng máy trên dữ liệu nhỏ	Bài tập kiểm thử	Viết vòng lặp tính vị từ cho mọi x và khẳng định đây có dạng “sai rồi đúng”; công cụ này nên có sẵn trong thư viện của bạn.

Kết chương và đọc tiếp

Kỹ thuật của chương này rẻ tới mức nên là một trong những thứ đầu tiên bạn thử khi gặp bài tối ưu: đọc đề, tìm cụm “nhỏ nhất của lớn nhất”, và hỏi “nếu tôi cố định ngưỡng thì việc kiểm tra có dễ không”. Hai điều đáng mang theo: tính đơn điệu là điều kiện duy nhất nhưng bắt buộc, và nó phải được kiểm chứ không được giả định; và log là cái giá rất rẻ để đổi lấy một bài toán dễ hơn hẳn.

Chương cuối của phần C là công cụ cắt ngang thứ hai, và nó đã xuất hiện thấp thoáng ở bốn chương trước: bấm phổ dụng, treap, chốt quicksort, ước lượng cây quay lui. Chương [C/20](#) phát biểu nguyên lý chung đằng sau cả bốn — *khi trường hợp xấu phụ thuộc vào dữ liệu, hãy chuyển sự phụ thuộc sang một nguồn ngẫu nhiên mà bạn kiểm soát* — và chỉ ra rằng ngẫu nhiên hoá không phải sự cầu may mà là một công cụ thiết kế có bảo đảm.

Nguồn

- [1] A. Laaksonen. “CSES Problem Set.” (2026), address: <https://cses.fi/problemset/> (visited on 09/06/2026).
- [2] M. Blum, R. W. Floyd, V. Pratt, et al., “Time Bounds for Selection,” *Journal of Computer and System Sciences*, vol. 7, no. 4, pp. 448–461, 1973.
- [3] V. Strassen, “Gaussian Elimination Is Not Optimal,” *Numerische Mathematik*, vol. 13, pp. 354–356, 1969.
- [4] D. Harvey and J. van der Hoeven, “Integer Multiplication in Time $O(n \log n)$,” *Annals of Mathematics*, vol. 193, no. 2, pp. 563–617, 2021.
- [5] J. Alman and V. Vassilevska Williams, “A Refined Laser Method and Faster Matrix Multiplication,” in *Proceedings of the 32nd ACM-SIAM Symposium on Discrete Algorithms (SODA)*, 2021.
- [6] J. Alman, R. Duan, V. Vassilevska Williams, et al. “Improving the Matrix Multiplication Exponent with Modern Optimization and AlphaEvolve.” bản tiền ấn arXiv:2608.16884, chưa qua phản biện. (2026), address: <https://arxiv.org/abs/2608.16884> (visited on 09/07/2026).
- [7] J. W. Cooley and J. W. Tukey, “An Algorithm for the Machine Calculation of Complex Fourier Series,” *Mathematics of Computation*, vol. 19, no. 90, pp. 297–301, 1965.
- [8] D. R. Musser, “Introspective Sorting and Selection Algorithms,” *Software: Practice and Experience*, vol. 27, no. 8, pp. 983–993, 1997.
- [9] J. L. Bentley and M. D. McIlroy, “Engineering a Sort Function,” *Software: Practice and Experience*, vol. 23, no. 11, pp. 1249–1265, 1993.
- [10] D. E. Knuth, “Optimum Binary Search Trees,” *Acta Informatica*, vol. 1, pp. 14–25, 1971.
- [11] F. F. Yao, “Speed-Up in Dynamic Programming,” *SIAM Journal on Algebraic and Discrete Methods*, vol. 3, no. 4, pp. 532–540, 1982.
- [12] A. Aggarwal, M. M. Klawe, S. Moran, et al., “Geometric Applications of a Matrix-Searching Algorithm,” *Algorithmica*, vol. 2, pp. 195–208, 1987.
- [13] D. S. Hirschberg, “A Linear Space Algorithm for Computing Maximal Common Subsequences,” *Communications of the ACM*, vol. 18, no. 6, pp. 341–343, 1975.
- [14] R. E. Tarjan, “Depth-First Search and Linear Graph Algorithms,” *SIAM Journal on Computing*, vol. 1, no. 2, pp. 146–160, 1972.
- [15] M. L. Fredman and R. E. Tarjan, “Fibonacci Heaps and Their Uses in Improved Network Optimization Algorithms,” *Journal of the ACM*, vol. 34, no. 3, pp. 596–615, 1987.
- [16] M. Thorup, “Undirected Single-Source Shortest Paths with Positive Integer Weights in Linear Time,” *Journal of the ACM*, vol. 46, no. 3, pp. 362–394, 1999.
- [17] R. Duan, J. Mao, X. Mao, et al., “Breaking the Sorting Barrier for Directed Single-Source Shortest Paths,” in *Proceedings of the 57th Annual ACM Symposium on Theory of Computing (STOC)*, Best Paper Award, 2025.
- [18] E. W. Dijkstra, “A Note on Two Problems in Connexion with Graphs,” *Numerische Mathematik*, vol. 1, pp. 269–271, 1959.

- [19] KTH Royal Institute of Technology. “KACTL: KTH Algorithm Competition Template Library.” (2026), address: <https://github.com/kth-competitive-programming/kactl> (visited on 09/06/2026).
- [20] H. W. Kuhn, “The Hungarian Method for the Assignment Problem,” *Naval Research Logistics Quarterly*, vol. 2, no. 1-2, pp. 83–97, 1955.
- [21] J. B. Orlin, “Max Flows in $O(nm)$ Time, or Better,” in *Proceedings of the 45th Annual ACM Symposium on Theory of Computing (STOC)*, 2013, pp. 765–774.
- [22] L. Chen, R. Kyng, Y. P. Liu, et al., “Maximum Flow and Minimum-Cost Flow in Almost-Linear Time,” in *Proceedings of the 63rd IEEE Symposium on Foundations of Computer Science (FOCS)*, Best Paper Award, 2022.
- [23] J. E. Hopcroft and R. M. Karp, “An $n^{5/2}$ Algorithm for Maximum Matchings in Bipartite Graphs,” *SIAM Journal on Computing*, vol. 2, no. 4, pp. 225–231, 1973.
- [24] J. Bentley, *Programming Pearls*, 2nd ed. Addison-Wesley, 2000.
- [25] R. Motwani and P. Raghavan, *Randomized Algorithms*. Cambridge University Press, 1995.

Ngẫu nhiên hoá (Randomized Algorithms)

PHỤ THUỘC A/05 cho kỳ vọng tuyến tính và các bất đẳng thức đuôi; B/08 và B/09 cho hai ứng dụng đã gặp; C/14 cho quicksort ngẫu nhiên.

TÓM TẮT

Ngẫu nhiên hoá đã xuất hiện bốn lần trong quyển sách này — băm phổ dụng, treap, chốt quicksort, ước lượng cây quay lui — và chương này phát biểu nguyên lý chung đằng sau cả bốn: *khi trường hợp xấu phụ thuộc vào dữ liệu, hãy chuyển sự phụ thuộc sang một nguồn ngẫu nhiên mà bạn kiểm soát*. Đây không phải sự cầu may: kết quả là các đảm bảo đúng cho mọi dữ liệu vào, kể cả dữ liệu do đối thủ chọn. Chương phân biệt hai loại thuật toán ngẫu nhiên, đi qua các đại diện — kiểm tra nguyên tố, kiểm tra tích ma trận, lát cắt nhỏ nhất của Karger, lấy mẫu dòng dữ liệu — và nói về cách khuếch đại xác suất đúng bằng cách lặp lại. Nếu chỉ nhớ một điều: ngẫu nhiên hoá đôi “luôn đúng, đôi khi rất chậm” lấy “gần như luôn nhanh, và bạn kiểm soát được xác suất sai”.

KIẾN THỨC CẦN CÓ TRƯỚC

- **thuật toán Las Vegas** — luôn cho kết quả đúng; thời gian chạy là biến ngẫu nhiên. Quicksort ngẫu nhiên và treap thuộc loại này.
- **thuật toán Monte Carlo** — luôn chạy trong thời gian định trước; kết quả có thể sai với xác suất nhỏ. Kiểm tra nguyên tố Miller–Rabin thuộc loại này.
- **khuếch đại xác suất** — lặp lại một thuật toán Monte Carlo k lần độc lập; nếu mỗi lần sai với xác suất p thì k lần cùng sai chỉ với xác suất p^k — giảm theo hàm mũ.
- **lấy mẫu hồ chứa** — kỹ thuật chọn k phần tử đều ngẫu nhiên từ một dòng dữ liệu chưa biết độ dài, chỉ dùng $O(k)$ bộ nhớ.
- **khử ngẫu nhiên** — quá trình biến một thuật toán ngẫu nhiên thành tất định, thường bằng cách thay nguồn ngẫu nhiên thật bằng một họ hàm nhỏ.
- **đối thủ thích ứng** — mô hình trong đó dữ liệu được chọn sau khi biết thuật toán — lý do mọi phân tích “trung bình theo dữ liệu” sụp đổ trong thực tế.

CÂU HỎI MỞ ĐẦU

- Q1. Vì sao ngẫu nhiên hoá lại cho đảm bảo *mạnh hơn* phân tích trung bình, dù cả hai đều nói về xác suất?
- Q2. Một thuật toán trả lời sai với xác suất một phần tư nghe rất tệ. Vì sao nó vẫn dùng được trong thực tế?
- Q3. Kiểm tra xem $A \times B = C$ có đúng không mà không nhân ma trận — làm sao có thể nhanh hơn phép nhân?
- Q4. Trong lập trình thi đấu, vì sao dùng hàm sinh số ngẫu nhiên cố định hạt giống lại nguy hiểm?
- Q5. Thuật toán lát cắt nhỏ nhất của Karger có ngẫu nhiên các cạnh và thường cho kết quả sai. Vì sao nó vẫn là một thuật toán tốt?

Ngẫu nhiên hoá là chủ đề dễ bị hiểu sai nhất trong quyển sách này, vì cái tên gợi ý sự may rủi. Thực tế thì ngược lại: nó là công cụ để *loại bỏ* may rủi.

Hãy nhớ lại tình huống lặp đi lặp lại ở phần B. Bảng băm nhanh “trung bình”, nhưng trung bình đó giả định dữ liệu ngẫu nhiên — và dữ liệu thật thì không ngẫu nhiên, thậm chí có thể do người khác chọn để hại bạn. Cây tìm kiếm nhị phân cân bằng “trung bình”, nhưng dữ liệu đã sắp sẵn là chuyện thường ngày. Quicksort nhanh “trung bình”, với cùng lỗ hổng. Trong cả ba trường hợp, phân tích trung bình là một lời hứa dựa trên giả định về dữ liệu, và giả định đó không do bạn kiểm soát.

Ngẫu nhiên hoá sửa đúng chỗ đó: thay vì hy vọng dữ liệu ngẫu nhiên, hãy *tự tạo ra ngẫu nhiên* ở chỗ bạn kiểm soát — chọn hàm băm, chọn độ ưu tiên, chọn chốt. Khi đó hành vi của thuật toán không còn phụ thuộc thứ tự dữ liệu, và đảm bảo trở thành đúng cho mọi dữ liệu vào. Đây là khác biệt then chốt so với phân tích trung bình, và nó là câu trả lời cho câu hỏi mở đầu thứ nhất.

8.1 Hai loại và cách chọn giữa chúng

Phân biệt dưới đây theo khuôn Las Vegas / Monte Carlo của Motwani và Raghavan[25].

Las Vegas luôn đúng, thời gian chạy ngẫu nhiên. Bạn không bao giờ phải lo về tính đúng; bạn chỉ phải chấp nhận rằng thỉnh thoảng nó chạy lâu hơn. Quicksort với chốt ngẫu nhiên, treap, và thuật toán chọn phần tử thứ k đều thuộc loại này.

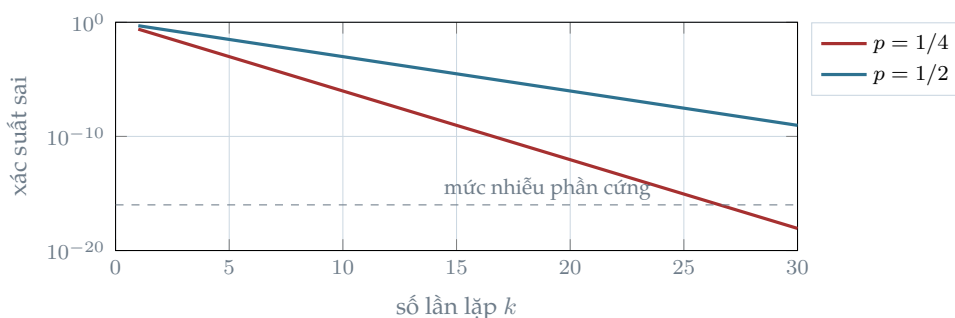
Monte Carlo luôn nhanh, kết quả có thể sai. Bạn kiểm soát được xác suất sai và làm nó nhỏ tùy ý bằng cách lặp lại. Kiểm tra nguyên tố, kiểm tra tích ma trận, băm chuỗi đều thuộc loại này.

Có một quy tắc chuyển đổi hữu ích: nếu kết quả *kiểm chứng được nhanh*, thì mọi thuật toán Monte Carlo biến thành Las Vegas bằng cách lặp lại tới khi kiểm chứng thành công. Ví dụ: một thuật toán ngẫu nhiên đề xuất một cách ghép cặp; kiểm tra cách ghép có hợp lệ hay không thì rẻ, nên cứ thử lại tới khi được. Ngược lại, khi kết quả không kiểm chứng được — “số này có phải nguyên tố không” — thì bạn buộc phải sống với xác suất sai.

8.2 Khuếch đại xác suất: vì sao “sai một phần tư” vẫn dùng được

Đây là kỹ thuật làm cho toàn bộ hướng tiếp cận trở nên khả thi, và nó dựa trên một tính toán rất đơn giản.

Giả sử thuật toán trả lời sai với xác suất tối đa $1/4$, và các lần chạy độc lập. Chạy k lần và lấy kết quả xuất hiện nhiều nhất; xác suất tất cả k lần cùng sai là $(1/4)^k$. Với $k = 30$, con số đó nhỏ hơn 10^{-18} — tức là nhỏ hơn xác suất phần cứng của bạn bị lỗi bit trong lúc chạy.



Hình 8.1. Khuếch đại xác suất. Xác suất sai giảm theo *hàm mũ* của số lần lặp, nên vài chục lần là đủ để đưa nó xuống dưới mức mà các nguồn sai khác — kể cả lỗi phần cứng — đã chi phối. Điều kiện bắt buộc: các lần chạy phải độc lập.

Điều đáng nhấn mạnh về mặt kỹ thuật: đây là dạng đơn giản nhất của các bất đẳng thức đuôi ở chương A/05, và nó cho thấy vì sao Chernoff quan trọng — xác suất sai giảm theo *hàm mũ* của số lần lặp, chứ không theo tỉ lệ nghịch. Chính điều đó cho phép ta trả một chi phí nhỏ (vài chục lần lặp) để đổi lấy độ tin cậy thực tế tuyệt đối.

Một điều kiện dễ bị quên: các lần chạy phải *độc lập*. Nếu bạn lặp lại thuật toán với cùng hạt giống, hoặc với các lựa chọn tương quan nhau, thì phép nhân xác suất không còn đúng và bạn chỉ đang lãng phí thời gian.

CẦN NHỚ

Trong lập trình thi đấu, luôn khởi tạo bộ sinh ngẫu nhiên bằng một nguồn không đoán trước — thời gian đồng hồ hệ thống ở độ phân giải cao, hoặc một giá trị phụ thuộc địa chỉ bộ nhớ. Dùng hạt giống cố định biến thuật toán ngẫu nhiên của bạn thành tất định, và khi đó người ra đề (hoặc người viết test phá) có thể dựng dữ liệu chạm đúng trường hợp xấu — đúng lỗ hổng mà ngẫu nhiên hoá lẽ ra đã bịt.

8.3 Bốn đại diện đáng biết

Kiểm tra nguyên tố. Muốn biết n có phải số nguyên tố, thử chia tới $\sqrt{n}$ là $O(\sqrt{n})$ — quá chậm với số 60 bit. Kiểm tra Miller–Rabin chọn ngẫu nhiên vài “nhân chứng” và dùng một tính chất số học để phát hiện hợp số; mỗi nhân chứng phát hiện hợp số với xác suất ít nhất $3/4$, nên vài chục nhân chứng cho độ tin cậy tuyệt đối trong thực tế. Đây là thuật toán chạy trong mọi thư viện mật mã, và là ví dụ chuẩn cho việc ngẫu nhiên hoá không phải giải pháp tạm mà là công cụ sản xuất.

Kiểm tra tích ma trận (Freivalds). Cho ba ma trận A, B, C cỡ $n \times n$, hỏi $AB = C$ có đúng không. Tính AB tốn $O(n^3)$. Mẹo: chọn một vector ngẫu nhiên r gồm các bit, và so $A(Br)$ với Cr — cả hai về chỉ cần các phép nhân ma trận–vector, tốn $O(n^2)$. Nếu $AB \neq C$ thì phép so sai với xác suất tối đa $1/2$, và lặp vài chục lần là đủ. Đây là câu trả lời cho câu hỏi mở đầu thứ ba, và nó minh hoạ một ý sâu hơn: *kiểm tra một đẳng thức rẻ hơn tính ra nó*, vì ta chỉ cần kiểm nó trên một hướng ngẫu nhiên thay vì trên mọi hướng.

Lát cắt nhỏ nhất của Karger. Lặp lại việc co một cạnh ngẫu nhiên cho tới khi còn hai đỉnh; các cạnh còn lại là một lát cắt. Xác suất ra đúng lát cắt nhỏ nhất chỉ khoảng $2/n^2$ — rất thấp — nhưng vì đó là xác suất chứ không phải sự bế tắc, ta chỉ cần lặp $O(n^2 \log n)$ lần để có độ tin cậy cao. Điều làm thuật toán này nổi tiếng không phải hiệu năng mà là sự đơn giản: nó không dùng luồng, không dùng lý thuyết nào ngoài việc co cạnh, và nó chứng minh rằng một cách tiếp cận “ngây thơ có ngẫu nhiên” vẫn có thể cho kết quả sâu sắc.

Lấy mẫu hồ chứa. Chọn đều ngẫu nhiên k phần tử từ một dòng dữ liệu mà bạn không biết trước độ dài và không lưu được. Giữ k phần tử đầu; với phần tử thứ $i > k$, giữ nó với xác suất k/i và nếu giữ thì thay thế một phần tử ngẫu nhiên trong hồ. Chứng minh tính đúng bằng quy nạp là một bài tập đẹp, và kỹ thuật này là công cụ chuẩn trong xử lý dòng dữ liệu (D/24).

8.4 Ngẫu nhiên hoá như một mẫu thiết kế

Nhìn lại toàn bộ quyển sách, ngẫu nhiên hoá xuất hiện dưới bốn vai trò khác nhau. Nhận ra bốn vai trò này giúp bạn biết khi nào nên nghĩ tới nó.

Vai trò 1 — phá vỡ trường hợp xấu do dữ liệu. Băm với hạt giống ngẫu nhiên (B/08), treap (B/09), chốt quicksort (C/14). Dấu hiệu: thuật toán của bạn nhanh trừ khi dữ liệu có cấu trúc đặc biệt.

Vai trò 2 — kiểm tra rẻ hơn tính toán. Freivalds, băm chuỗi (B/12), so sánh hai đa thức bằng cách thay một giá trị ngẫu nhiên. Dấu hiệu: bạn cần kiểm một đẳng thức chứ không cần biết giá trị.

Vai trò 3 — lấy mẫu thay cho vét cạn. Ước lượng kích thước cây quay lui (C/13), lấy mẫu hồ chứa, các phương pháp Monte Carlo cho tích phân. Dấu hiệu: không gian quá lớn để duyệt nhưng bạn chỉ cần một ước lượng.

Vai trò 4 — thoát khỏi cấu trúc xấu. Karger, khởi động lại nhiều lần trong tìm kiếm cục bộ (C/19), RANSAC trong thị giác máy tính. Dấu hiệu: có nhiều lời giải khả dĩ và bạn chỉ cần tìm được một cái tốt, chứ không cần chứng minh nó tốt nhất.

CẠM BẮY

Ngẫu nhiên hoá không sửa được một thuật toán *sai*. Nếu ý tưởng của bạn có phần ví dụ về mặt logic thì việc thêm ngẫu nhiên chỉ làm nó sai một cách khó dự đoán hơn. Ngẫu nhiên hoá chỉ giải quyết vấn đề *phân phối dữ liệu*, không giải quyết vấn đề tính đúng. Trước khi thêm ngẫu nhiên, hãy chắc chắn bạn đang ở đúng loại vấn đề.

8.5 Gốc rễ: một ván bài trong bệnh viện và bài toán số nguyên tố

Phương pháp Monte Carlo có một nguồn gốc được kể lại khá thống nhất. Năm 1946, Stanislaw Ulam đang nằm dưỡng bệnh và chơi bài một mình. Ông tự hỏi xác suất để một ván bài kiểu solitaire thắng là bao nhiêu, và nhận ra rằng việc tính chính xác bằng tổ hợp là gần như bất khả thi — trong khi việc *chơi thử một trăm ván và đếm* thì rất dễ. Ý tưởng đó, mang sang cho các bài toán khuếch tán neutron mà ông và von Neumann đang làm tại Los Alamos, trở thành phương pháp Monte Carlo. Cái tên được đặt theo sông bạc ở Monaco, nơi một người họ hàng của Ulam hay lui tới.

Bài học phương pháp ở đây rất đáng nhớ và nó áp dụng cho cả việc giải thuật toán: khi một đại lượng khó tính chính xác nhưng dễ *lấy mẫu*, hãy đổi bài toán tính sang bài toán ước lượng. Đây chính là vai trò thứ ba ở mục 5.

Nhánh thuật toán ngẫu nhiên trong khoa học máy tính bắt đầu nghiêm túc vào giữa thập niên 1970 với bài toán kiểm tra nguyên tố. Solovay và Strassen, rồi Rabin, đưa ra các thuật toán xác suất nhanh cho một bài toán mà khi đó chưa ai biết có lời giải đa thức tất định hay không. Điều này đặt ra một câu hỏi lý thuyết quan trọng: ngẫu nhiên có thực sự cho thêm sức mạnh tính toán, hay mọi thuật toán ngẫu nhiên đều khử ngẫu nhiên được? Câu chuyện có một kết cục thú vị: năm 2002, ba nhà nghiên cứu Ấn Độ công bố một thuật toán *tất định* thời gian đa thức cho kiểm tra nguyên tố, giải quyết đúng bài toán mở đầu của cả nhánh — dù trong thực tế thì các thuật toán ngẫu nhiên vẫn nhanh hơn nhiều và vẫn là thứ được dùng.

Câu hỏi tổng quát vẫn còn mở nhưng nghiêng về một phía: phần lớn giới nghiên cứu tin rằng ngẫu nhiên *không* cho thêm sức mạnh về mặt lớp độ phức tạp, tức là mọi thuật toán ngẫu nhiên đa thức đều có phiên bản tất định đa thức. Điều đó không làm ngẫu nhiên hoá mất giá trị — các phiên bản tất định thường phức tạp và chậm hơn nhiều bậc trong thực tế — nhưng nó nói rằng sức mạnh thật sự của ngẫu nhiên nằm ở *sự đơn giản*, không ở khả năng tính toán.

Gốc rễ: ý tưởng này ra đời từ bài toán nào

Mốc	Bài toán thúc ép lúc đó	Điều thay đổi sau đó
1946 — Ulam, von Neumann	Tính xác suất thắng của một ván bài; rồi bài toán khuếch tán neutron ở Los Alamos.	Phương pháp Monte Carlo: khi khó tính chính xác nhưng dễ lấy mẫu, hãy đổi sang ước lượng.
1975–76 — Solovay, Strassen, Rabin	Kiểm tra số lớn có nguyên tố không — chưa ai biết lời giải đa thức tất định.	Kiểm tra nguyên tố xác suất; khai sinh nhánh thuật toán ngẫu nhiên trong khoa học máy tính.
1977 — Freivalds	Kiểm tra $AB = C$ mà không muốn tốn $O(n^3)$.	Kiểm bằng một vector ngẫu nhiên trong $O(n^2)$; ý tưởng “kiểm rẻ hơn tính”.
1979 — Carter & Wegman	Bảng băm nhanh trung bình nhưng không có đảm bảo với dữ liệu thù địch.	Băm phổ dụng: ngẫu nhiên nằm ở lựa chọn hàm, không ở dữ liệu (B/08).
1993 — Karger	Tìm lát cắt nhỏ nhất mà không dùng luồng.	Co cạnh ngẫu nhiên; xác suất thành công thấp nhưng lặp lại được — minh hoạ rằng đơn giản cộng ngẫu nhiên vẫn cho kết quả sâu.
2002 — Agrawal, Kayal, Saxena	Kiểm tra nguyên tố có lời giải đa thức tất định không?	Có — nhưng chậm hơn bản ngẫu nhiên nhiều bậc trong thực tế. Sức mạnh của ngẫu nhiên nằm ở sự đơn giản, không ở lớp độ phức tạp.

8.6 Liên hệ ngang

Liên hệ ngang: ý này còn sống ở đâu nữa

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu
Thị giác máy tính, SLAM	RANSAC: lấy mẫu ngẫu nhiên tập con nhỏ để ước lượng mô hình	Vai trò 4 ở mục 5, và là ứng dụng ngẫu nhiên hoá quan trọng nhất trong ngành này. Cùng lập luận khuếch đại: xác suất một lần lấy mẫu “sạch” thì thấp, nhưng lặp đủ nhiều thì gần như chắc chắn có ít nhất một lần sạch.
Ước lượng trạng thái	Bộ lọc hạt: biểu diễn phân phối bằng một tập mẫu	Monte Carlo áp cho bài toán ước lượng liên tục. Cùng tinh thần với mục 4: khi phân phối khó mô tả bằng công thức, hãy mô tả nó bằng mẫu. Cạm bẫy đặc thù là suy thoái mẫu, cần lấy mẫu lại.
Thống kê	Bootstrap, MCMC, kiểm định hoán vị	Cùng ý tưởng gốc của Ulam: thay tính toán giải tích bằng mô phỏng lặp lại. Khác: ở đó mục tiêu là ước lượng độ bất định, ở đây là ước lượng một đại lượng.
Mật mã	Sinh khoá, kiểm tra nguyên tố, giao thức xác suất	Kiểm tra nguyên tố ngẫu nhiên chạy mỗi lần bạn tạo một khoá. Điểm khác biệt quan trọng: ở đó chất lượng nguồn ngẫu nhiên là vấn đề an ninh, không chỉ là vấn đề hiệu năng.
Học máy	Khởi tạo ngẫu nhiên, bỏ ngẫu nhiên nơ-ron, trộn dữ liệu, rừng ngẫu nhiên	Ngẫu nhiên ở đây phục vụ tính tổng quát hoá chứ không phục vụ tốc độ: nó phá vỡ sự tương quan và ngăn mô hình bám vào cấu trúc giả. Cùng nguyên lý phá vỡ cấu trúc xấu, khác mục tiêu.
Xử lý dòng dữ liệu	Lấy mẫu hồ chứa, sơ đồ đếm xấp xỉ, HyperLogLog	Ngẫu nhiên là điều kiện để làm việc với bộ nhớ nhỏ hơn dữ liệu nhiều bậc (D/24). Cùng đánh đổi: chấp nhận sai số nhỏ để đổi lấy khả năng chạy được.
Trò chơi và AI	Tìm kiếm cây Monte Carlo	Thay vì duyệt cây trò chơi đầy đủ, lấy mẫu các ván chơi tới cuối và thống kê. Cùng vai trò 3; đây là kỹ thuật đứng sau bước nhảy vọt của các chương trình chơi cờ vây.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì hai loại phân tích nói về hai nguồn ngẫu nhiên khác nhau. Phân tích trung bình giả định dữ liệu được rút từ một phân phối nào đó — một giả định về thể giới, và nó sai khi dữ liệu có cấu trúc hoặc do đối thủ chọn. Ngẫu nhiên hoá đặt nguồn ngẫu nhiên vào lựa chọn của thuật toán, thứ mà bạn kiểm soát và đối thủ không biết. Kết quả là một đảm bảo đúng cho mọi dữ liệu vào, không kèm giả định nào về phân phối dữ liệu.
- A2. Vì xác suất sai giảm theo hàm mũ khi lặp lại. Chạy k lần độc lập, xác suất tất cả cùng sai là $(1/4)^k$; với $k = 30$ thì nhỏ hơn 10^{-18} , tức là nhỏ hơn xác suất máy tính bị lỗi phần cứng trong lúc chạy. Điều kiện là các lần chạy phải thật sự độc lập — dùng lại cùng hạt giống hoặc các lựa chọn tương quan sẽ phá vỡ phép nhân xác suất và bạn chỉ đang lặp lại cùng một sai lầm.
- A3. Vì để kiểm một đẳng thức, ta không cần kiểm nó theo mọi hướng — chỉ cần theo một hướng ngẫu nhiên là đủ để phát hiện sai lệch với xác suất cao. Cụ thể, thay vì tính AB rồi so với C , ta chọn vector ngẫu nhiên r và so $A(Br)$ với Cr ; cả hai về chỉ cần nhân ma trận với vector, tốn $O(n^2)$. Nếu hai ma trận khác nhau thì chúng khác nhau trên phần lớn các vector, nên xác suất bỏ sót là nhỏ và giảm theo hàm mũ khi lặp lại.
- A4. Vì hạt giống cố định làm thuật toán trở thành tất định, và mọi thứ tất định đều dựng được test phá. Người viết test chỉ cần chạy đúng chương trình của bạn để biết nó sẽ chọn chốt nào, chọn hàm băm nào, rồi dựng dữ liệu chạm đúng trường hợp xấu. Toàn bộ giá trị của ngẫu nhiên hoá nằm ở chỗ đối thủ không đoán được, nên hạt giống phải lấy từ nguồn không đoán trước như đồng hồ độ phân giải cao.
- A5. Vì thất bại của nó là thất bại có xác suất, không phải bế tắc, nên lặp lại giải quyết được. Xác suất một lần chạy ra đúng lát cắt nhỏ nhất là khoảng $2/n^2$; lặp $O(n^2 \log n)$ lần cho xác suất bỏ sót nhỏ tùy ý. Giá trị của nó không nằm ở hiệu năng — các thuật toán dựa trên luồng nhanh hơn — mà ở chỗ nó cho thấy một thủ tục cực kỳ đơn giản, không dùng lý thuyết nào ngoài việc co cạnh, vẫn giải được bài toán mà trước đó chỉ có lời giải phức tạp.

NÓI LẠI THẬT ĐƠN GIẢN

Có một hiểu lầm phổ biến: nghĩ rằng dùng số ngẫu nhiên là phó mặc cho may rủi. Thực ra thì ngược lại — người ta dùng ngẫu nhiên để *khỏi phụ thuộc vào may rủi*.

Hãy nhớ lại chuyện cái tủ hồ sơ ở chương về băm. Nếu quy tắc xếp hồ sơ của bạn là cố định thì người khác có thể mang tới toàn những hồ sơ rơi vào cùng một ngăn, và tủ của bạn thành vô dụng. Nhưng nếu mỗi sáng bạn trộn thêm một con số bí mật vào quy tắc thì họ không đoán được nữa. Bạn không hy vọng gặp may; bạn đã *loại bỏ* khả năng bị hại.

Cùng logic đó áp cho việc sắp xếp: chọn phần tử làm mốc một cách ngẫu nhiên thì không có dây đầu vào nào là “dây xấu” nữa, vì cái xấu bây giờ phụ thuộc vào đồng xu của bạn chứ không phụ thuộc vào dữ liệu.

Loại thứ hai của ngẫu nhiên hoá thì khác: chấp nhận trả lời sai đôi khi, để đổi lấy tốc độ. Nghe đáng lo, cho tới khi bạn làm phép tính: nếu mỗi lần trả lời sai với xác suất một phần tư, thì ba mươi lần liên tiếp cùng sai có xác suất nhỏ hơn xác suất máy tính của bạn hỏng giữa chừng. Ở mức đó, “có thể sai” trở thành một khái niệm không còn ý nghĩa thực tế.

Loại thứ ba là lấy mẫu, và nó đến từ một ý rất đời: nếu bạn muốn biết trong nồi canh đủ mặn chưa, bạn không uống cả nồi — bạn ném một thìa. Với điều kiện là đã khuấy đều. Toàn bộ nghệ thuật của các phương pháp lấy mẫu nằm ở đúng chữ “khuấy đều” đó.

Một câu: Đừng hy vọng dữ liệu ngẫu nhiên; hãy tự tạo ngẫu nhiên ở chỗ bạn kiểm soát.

Chỗ không rút gọn được: Chất lượng của nguồn ngẫu nhiên là chuyện thật, không phải chi tiết: một bộ sinh kém hoặc một hạt giống đoán được sẽ phá huỷ mọi đảm bảo ở trên.

Thuật ngữ / câu hỏi	Trả lời
●●● Nguyên lý cốt lõi	Khi trường hợp xấu phụ thuộc dữ liệu, chuyển sự phụ thuộc sang nguồn ngẫu nhiên bạn kiểm soát; đảm bảo trở thành đúng cho <i>mọi</i> dữ liệu vào.
●●● Ngẫu nhiên hoá khác phân tích trung bình	Trung bình giả định về phân phối <i>dữ liệu</i> (không kiểm soát được); ngẫu nhiên hoá đặt ngẫu nhiên vào <i>lựa chọn của thuật toán</i> .
●●● Las Vegas so với Monte Carlo	Las Vegas: luôn đúng, thời gian ngẫu nhiên. Monte Carlo: luôn nhanh, có thể sai. Kết quả kiểm chứng được nhanh thì chuyển Monte Carlo thành Las Vegas bằng cách lặp.
●●● Khuếch đại xác suất	Sai với xác suất p mỗi lần, lặp k lần độc lập thì p^k ; $k = 30$ với $p = 1/4$ cho dưới 10^{-18} . Bắt buộc: các lần chạy độc lập.
●●● Hạt giống trong thi đấu	Phải lấy từ nguồn không đoán trước (đồng hồ độ phân giải cao); hạt giống cố định biến thuật toán thành tất định và dựng test phá được.
●● Bốn vai trò của ngẫu nhiên hoá	Phá trường hợp xấu do dữ liệu; kiểm rẻ hơn tính; lấy mẫu thay vết cặn; thoát khỏi cấu trúc xấu.
●● Freivalds	Kiểm $AB = C$ bằng vector ngẫu nhiên trong $O(n^2)$ thay vì $O(n^3)$; ý tưởng “kiểm một đẳng thức chỉ cần một hướng ngẫu nhiên”.
●● Miller–Rabin	Chọn nhân chứng ngẫu nhiên; mỗi nhân chứng phát hiện hợp số với xác suất $\geq 3/4$; là thuật toán dừng trong mọi thư viện mật mã.
●● Karger	Co cạnh ngẫu nhiên tới khi còn hai đỉnh; xác suất đúng $\approx 2/n^2$, lặp $O(n^2 \log n)$ lần. Giá trị nằm ở sự đơn giản, không ở tốc độ.
●● Lấy mẫu hồ chứa	Giữ k phần tử đầu; phần tử thứ $i > k$ được giữ với xác suất k/i , thay thế ngẫu nhiên một phần tử trong hồ. Bộ nhớ $O(k)$, không cần biết độ dài dòng.
●● Ngẫu nhiên không sửa được cái gì	Không sửa được thuật toán <i>sai</i> về logic; nó chỉ giải quyết vấn đề phân phối dữ liệu.
● Ngẫu nhiên và lớp độ phức tạp	Phần lớn giới nghiên cứu tin rằng ngẫu nhiên không thêm sức mạnh về lớp; giá trị thật của nó là sự đơn giản và hằng số nhỏ.

●●● phải thuộc ●● nên nắm ● biết là có

I Câu hỏi ôn tập có đáp án (che phần đáp án, tự trả lời trước)

1. Tôi nên dùng bộ sinh ngẫu nhiên nào trong C++?

| Dùng `mt19937` (hoặc `mt19937_64` cho số 64 bit) với hạt giống lấy từ đồng hồ độ phân giải cao. Tránh `rand()`: chất lượng kém, khoảng giá trị nhỏ trên một số nền tảng, và cách lấy ngẫu nhiên trong một khoảng bằng phép chia dư gây lệch phân phối. Để lấy số ngẫu nhiên trong $[a, b]$, dùng `uniform_int_distribution` thay vì phép chia dư.

2. Trộn ngẫu nhiên mảng trước khi sắp có thực sự cần không?

| Cần, trong hai tình huống. Thứ nhất, khi bạn tự cài quicksort mà không chọn chốt ngẫu nhiên — lúc đó trộn trước là cách rẻ nhất để loại bỏ trường hợp xấu. Thứ hai, khi dùng hàm sắp xếp thư viện trong một kỳ thi mà bạn biết có test phá nhắm vào cài đặt cụ thể của thư viện đó. Chi phí là một dòng và $O(n)$, còn lợi ích là loại hẳn một lớp thất bại — một đánh đổi rất đáng.

3. Làm sao chứng minh một thuật toán ngẫu nhiên là đúng?

| Chia làm hai phần và đừng lẫn chúng. Phần tính đúng: với Las Vegas thì chứng minh như thuật toán tất định — bất biến, quy nạp ($A/\emptyset$); ngẫu nhiên chỉ ảnh hưởng thời gian. Với Monte Carlo thì phải chặn xác suất sai, thường bằng cách chỉ ra “một lựa chọn ngẫu nhiên là xấu chỉ khi rơi vào một tập nhỏ”. Phần chi phí: dùng kỳ vọng tuyến tính và các bất đẳng thức đuôi ở $A/\emptyset$, và nhớ nói rõ kỳ vọng lấy theo cái gì.

4. Nếu ngẫu nhiên hoá tốt như vậy, vì sao không dùng nó ở mọi nơi?

| Vì nó có ba cái giá. Một, kết quả không tái lập được giữa các lần chạy, gây khó cho gỡ lỗi — cách chữa là ghi lại hạt giống của mỗi lần chạy để tái hiện. Hai, với Monte Carlo, bạn phải chấp nhận xác suất sai khác không; trong các hệ thống quan trọng, điều này cần được nêu rõ chứ không được giấu. Ba, một số bài đơn giản là không có cấu trúc để ngẫu nhiên hoá khai thác. Ngẫu nhiên là công cụ, không phải liều thuốc chung.

5. Trong hệ thống thật, tôi phải cẩn thận gì với ngẫu nhiên?

| Ba điều. Một, ghi lại hạt giống để tái lập được sự cố — đây là điều khác biệt lớn nhất giữa hệ thống có kỷ luật và hệ thống không. Hai, phân biệt bộ sinh ngẫu nhiên thông thường với bộ sinh an toàn mật mã: dùng loại thứ nhất cho khoá hay mã phiên là một lỗ hổng bảo mật thật sự. Ba, trong hệ nhiều luồng, chia sẻ một bộ sinh giữa các luồng vừa gây tranh chấp vừa có thể phá tính độc lập — mỗi luồng nên có bộ sinh riêng với hạt giống khác nhau.

6. RANSAC có phải thuật toán ngẫu nhiên theo nghĩa của chương này không?

| Đúng, và nó thuộc vai trò thứ tư. Bài toán là ước lượng một mô hình từ dữ liệu có nhiều điểm ngoại lai; vết cận mọi tập con là bất khả thi. RANSAC lấy ngẫu nhiên một tập con nhỏ vừa đủ để xác định mô hình, đếm số điểm ủng hộ, và lặp lại. Lập luận về số lần lặp cần thiết chính là lập luận khuếch đại ở mục 3: nếu xác suất một lần lấy mẫu “sạch” là p thì sau k lần, xác suất chưa từng sạch là $(1 - p)^k$, và ta chọn k để con số đó đủ nhỏ.

7. Có cách nào biến thuật toán ngẫu nhiên thành tất định không?

| Có, và nhánh nghiên cứu đó gọi là khử ngẫu nhiên. Kỹ thuật thường dùng là thay nguồn ngẫu nhiên thật bằng một họ hàm nhỏ có tính chất tương tự — ví dụ dùng một họ hàm băm độc lập đôi một thay vì hàm ngẫu nhiên hoàn toàn, khi phân tích chỉ cần tính độc lập đôi một. Trong thực hành thì đầu, khử ngẫu nhiên hiếm khi đáng làm; giá trị của nó nằm ở lý thuyết và ở các tình huống bắt buộc phải tái lập được kết quả.

Bài tập (mức: ● khởi động ●● chuẩn ●●● khó)

Bài	Nguồn	Điều cần nhận ra
● Trộn ngẫu nhiên một mảng đúng cách	Bài tập cài đặt	Thuật toán Fisher–Yates; kiểm bằng thống kê rằng mọi hoán vị xuất hiện đều nhau trên $n = 4$.
● Cài quicksort với chốt ngẫu nhiên và đo trên mảng đã sắp	Bài tập thực nghiệm	So với chốt cố định trên cùng dữ liệu — khác biệt sẽ rất rõ.
●● Kiểm tra nguyên tố Miller–Rabin cho số tới 10^{18}	Bài kinh điển	Chú ý phép nhân modulo tránh tràn; so thời gian với thử chia tới $\sqrt{n}$.
●● Kiểm tra $AB = C$ bằng thuật toán Freivalds	Bài kinh điển	Đo thời gian so với nhân ma trận thật; lặp 30 lần và ước lượng xác suất sai.
●● Lấy mẫu hồ chứa trên một dòng dữ liệu dài	Bài tập cài đặt	Chứng minh tính đều bằng quy nạp trước khi cài; kiểm bằng thống kê sau khi cài.
●● Tìm phần tử xuất hiện quá nửa bằng phương pháp ngẫu nhiên	Bài kinh điển	Chọn ngẫu nhiên một phần tử và kiểm; xác suất trúng $> 1/2$ nên vài chục lần là đủ. So với thuật toán bỏ phiếu tất định.
●●● Cài lát cắt nhỏ nhất của Karger	Bài kinh điển	Dùng DSU cho phép co cạnh ($B/11$); đo tỉ lệ ra đúng theo số lần lặp.
●●● Băm chuỗi với cơ số ngẫu nhiên, so với cơ số cố định trên test phá	Bài tập an ninh	Tự dựng test phá cho cơ số cố định — bài tập cho thấy rõ nhất vì sao phải ngẫu nhiên hoá.
●●● Ước lượng số nút cây quay lui bằng phương pháp Knuth	Chương C/13	Vai trò 3 của ngẫu nhiên hoá; so ước lượng với số thật trên bài n quân hậu.
●●● RANSAC ước lượng một đường thẳng từ dữ liệu có 50% điểm ngoại lai	Bài tập ứng dụng	Tính số lần lặp cần thiết trước khi chạy, rồi kiểm chứng bằng thực nghiệm.

Bài	Nguồn	Điều cần nhận ra (tiếp)
●●● Tìm kiếm cây Monte Carlo cho một trò chơi nhỏ	Bài tập cài đặt	So với duyệt đầy đủ có cắt tia (C/13); quan sát chất lượng theo số lần mô phỏng.

Kết chương và đọc tiếp

Phần C khép lại ở đây. Chương này gom lại một sợi chỉ đã chạy suốt từ phần B: ngẫu nhiên hoá không phải sự cầu may mà là cách chuyển sự phụ thuộc từ dữ liệu — thứ bạn không kiểm soát — sang các lựa chọn của chính thuật toán — thứ bạn kiểm soát. Hai điều đáng mang theo: bốn vai trò ở mục 5 để biết khi nào nghĩ tới nó, và phép tính khuếch đại cho thấy vì sao “có thể sai” lại trở thành khái niệm không có ý nghĩa thực tế sau vài chục lần lặp.

Tám chương của phần C đều trả lời câu hỏi “dùng khuôn tư duy nào để sinh ra lời giải”. Phần D đổi câu hỏi sang một hướng khác và ít dễ chịu hơn: *khi nào thì nên ngừng tìm lời giải nhanh*. Nó bắt đầu bằng hai miền có bấy số học riêng — số học và hình học — rồi tới chương quan trọng nhất của phần: cách nhận ra một bài toán là NP-khó, và bốn cách sống chung với sự thật đó (D/23).

Đọc thêm: công trình gốc (chỉ liệt kê nguồn đã qua phản biện; bản tiền ấn được ghi rõ)

Mức	Công trình và tác giả	Nơi công bố, năm	Vì sao đọc, đọc phần nào
nền tảng	Karger & Stein, “A New Approach to the Minimum Cut Problem”	Journal of the ACM, 1996	Thuật toán co cạnh ngẫu nhiên kèm mẹo đệ quy làm xác suất thành công tăng vọt. Đọc phần phân tích xác suất — ngắn, đẹp, và là mẫu mực cho việc khuếch đại.
nền tảng	Motwani & Raghavan, <i>Randomized Algorithms</i>	Cambridge University Press, 1995	Sách chuẩn của lĩnh vực; đọc chương về thuật toán Las Vegas và Monte Carlo cùng chương về bất đẳng thức tập trung.
tra khi cần	Karp & Rabin, “Efficient Randomized Pattern-Matching Algorithms”	IBM Journal of Research and Development, 1987	Băm chuỗi như một thuật toán ngẫu nhiên Monte Carlo; đọc kèm B/08.
tra khi cần	Alon, Matias, Szegedy, “The Space Complexity of Approximating the Frequency Moments”	Journal of Computer and System Sciences, 1999	Bài mở đường cho thuật toán luồng dữ liệu; đọc khi cần ước lượng thống kê với bộ nhớ nhỏ (D/24).
tra khi cần	Mitzenmacher & Upfal, <i>Probability and Computing</i>	Cambridge University Press, 2005	Dễ vào hơn Motwani–Raghavan và dạy luôn phần xác suất cần thiết.

Nguồn

- [1] A. Laaksonen. “CSES Problem Set.” (2026), address: <https://cses.fi/problemset/> (visited on 09/06/2026).
- [2] M. Blum, R. W. Floyd, V. Pratt, et al., “Time Bounds for Selection,” *Journal of Computer and System Sciences*, vol. 7, no. 4, pp. 448–461, 1973.
- [3] V. Strassen, “Gaussian Elimination Is Not Optimal,” *Numerische Mathematik*, vol. 13, pp. 354–356, 1969.
- [4] D. Harvey and J. van der Hoeven, “Integer Multiplication in Time $O(n \log n)$,” *Annals of Mathematics*, vol. 193, no. 2, pp. 563–617, 2021.
- [5] J. Alman and V. Vassilevska Williams, “A Refined Laser Method and Faster Matrix Multiplication,” in *Proceedings of the 32nd ACM-SIAM Symposium on Discrete Algorithms (SODA)*, 2021.
- [6] J. Alman, R. Duan, V. Vassilevska Williams, et al. “Improving the Matrix Multiplication Exponent with Modern Optimization and AlphaEvolve.” bản tiền ấn arXiv:2608.16884, chưa qua phản biện. (2026), address: <https://arxiv.org/abs/2608.16884> (visited on 09/07/2026).
- [7] J. W. Cooley and J. W. Tukey, “An Algorithm for the Machine Calculation of Complex Fourier Series,” *Mathematics of Computation*, vol. 19, no. 90, pp. 297–301, 1965.
- [8] D. R. Musser, “Introspective Sorting and Selection Algorithms,” *Software: Practice and Experience*, vol. 27, no. 8, pp. 983–993, 1997.
- [9] J. L. Bentley and M. D. McIlroy, “Engineering a Sort Function,” *Software: Practice and Experience*, vol. 23, no. 11, pp. 1249–1265, 1993.
- [10] D. E. Knuth, “Optimum Binary Search Trees,” *Acta Informatica*, vol. 1, pp. 14–25, 1971.
- [11] F. F. Yao, “Speed-Up in Dynamic Programming,” *SIAM Journal on Algebraic and Discrete Methods*, vol. 3, no. 4, pp. 532–540, 1982.
- [12] A. Aggarwal, M. M. Klawe, S. Moran, et al., “Geometric Applications of a Matrix-Searching Algorithm,” *Algorithmica*, vol. 2, pp. 195–208, 1987.
- [13] D. S. Hirschberg, “A Linear Space Algorithm for Computing Maximal Common Subsequences,” *Communications of the ACM*, vol. 18, no. 6, pp. 341–343, 1975.
- [14] R. E. Tarjan, “Depth-First Search and Linear Graph Algorithms,” *SIAM Journal on Computing*, vol. 1, no. 2, pp. 146–160, 1972.
- [15] M. L. Fredman and R. E. Tarjan, “Fibonacci Heaps and Their Uses in Improved Network Optimization Algorithms,” *Journal of the ACM*, vol. 34, no. 3, pp. 596–615, 1987.
- [16] M. Thorup, “Undirected Single-Source Shortest Paths with Positive Integer Weights in Linear Time,” *Journal of the ACM*, vol. 46, no. 3, pp. 362–394, 1999.
- [17] R. Duan, J. Mao, X. Mao, et al., “Breaking the Sorting Barrier for Directed Single-Source Shortest Paths,” in *Proceedings of the 57th Annual ACM Symposium on Theory of Computing (STOC)*, Best Paper Award, 2025.
- [18] E. W. Dijkstra, “A Note on Two Problems in Connexion with Graphs,” *Numerische Mathematik*, vol. 1, pp. 269–271, 1959.

- [19] KTH Royal Institute of Technology. “KACTL: KTH Algorithm Competition Template Library.” (2026), address: <https://github.com/kth-competitive-programming/kactl> (visited on 09/06/2026).
- [20] H. W. Kuhn, “The Hungarian Method for the Assignment Problem,” *Naval Research Logistics Quarterly*, vol. 2, no. 1-2, pp. 83–97, 1955.
- [21] J. B. Orlin, “Max Flows in $O(nm)$ Time, or Better,” in *Proceedings of the 45th Annual ACM Symposium on Theory of Computing (STOC)*, 2013, pp. 765–774.
- [22] L. Chen, R. Kyng, Y. P. Liu, et al., “Maximum Flow and Minimum-Cost Flow in Almost-Linear Time,” in *Proceedings of the 63rd IEEE Symposium on Foundations of Computer Science (FOCS)*, Best Paper Award, 2022.
- [23] J. E. Hopcroft and R. M. Karp, “An $n^{5/2}$ Algorithm for Maximum Matchings in Bipartite Graphs,” *SIAM Journal on Computing*, vol. 2, no. 4, pp. 225–231, 1973.
- [24] J. Bentley, *Programming Pearls*, 2nd ed. Addison-Wesley, 2000.
- [25] R. Motwani and P. Raghavan, *Randomized Algorithms*. Cambridge University Press, 1995.

Nguồn

- [1] A. Laaksonen. “CSES Problem Set.” (2026), address: <https://cses.fi/problemset/> (visited on 09/06/2026).
- [2] M. Blum, R. W. Floyd, V. Pratt, et al., “Time Bounds for Selection,” *Journal of Computer and System Sciences*, vol. 7, no. 4, pp. 448–461, 1973.
- [3] V. Strassen, “Gaussian Elimination Is Not Optimal,” *Numerische Mathematik*, vol. 13, pp. 354–356, 1969.
- [4] D. Harvey and J. van der Hoeven, “Integer Multiplication in Time $O(n \log n)$,” *Annals of Mathematics*, vol. 193, no. 2, pp. 563–617, 2021.
- [5] J. Alman and V. Vassilevska Williams, “A Refined Laser Method and Faster Matrix Multiplication,” in *Proceedings of the 32nd ACM-SIAM Symposium on Discrete Algorithms (SODA)*, 2021.
- [6] J. Alman, R. Duan, V. Vassilevska Williams, et al. “Improving the Matrix Multiplication Exponent with Modern Optimization and AlphaEvolve.” bản tiền ấn arXiv:2608.16884, chưa qua phản biện. (2026), address: <https://arxiv.org/abs/2608.16884> (visited on 09/07/2026).
- [7] J. W. Cooley and J. W. Tukey, “An Algorithm for the Machine Calculation of Complex Fourier Series,” *Mathematics of Computation*, vol. 19, no. 90, pp. 297–301, 1965.
- [8] D. R. Musser, “Introspective Sorting and Selection Algorithms,” *Software: Practice and Experience*, vol. 27, no. 8, pp. 983–993, 1997.
- [9] J. L. Bentley and M. D. McIlroy, “Engineering a Sort Function,” *Software: Practice and Experience*, vol. 23, no. 11, pp. 1249–1265, 1993.
- [10] D. E. Knuth, “Optimum Binary Search Trees,” *Acta Informatica*, vol. 1, pp. 14–25, 1971.
- [11] F. F. Yao, “Speed-Up in Dynamic Programming,” *SIAM Journal on Algebraic and Discrete Methods*, vol. 3, no. 4, pp. 532–540, 1982.
- [12] A. Aggarwal, M. M. Klawe, S. Moran, et al., “Geometric Applications of a Matrix-Searching Algorithm,” *Algorithmica*, vol. 2, pp. 195–208, 1987.
- [13] D. S. Hirschberg, “A Linear Space Algorithm for Computing Maximal Common Subsequences,” *Communications of the ACM*, vol. 18, no. 6, pp. 341–343, 1975.
- [14] R. E. Tarjan, “Depth-First Search and Linear Graph Algorithms,” *SIAM Journal on Computing*, vol. 1, no. 2, pp. 146–160, 1972.
- [15] M. L. Fredman and R. E. Tarjan, “Fibonacci Heaps and Their Uses in Improved Network Optimization Algorithms,” *Journal of the ACM*, vol. 34, no. 3, pp. 596–615, 1987.
- [16] M. Thorup, “Undirected Single-Source Shortest Paths with Positive Integer Weights in Linear Time,” *Journal of the ACM*, vol. 46, no. 3, pp. 362–394, 1999.
- [17] R. Duan, J. Mao, X. Mao, et al., “Breaking the Sorting Barrier for Directed Single-Source Shortest Paths,” in *Proceedings of the 57th Annual ACM Symposium on Theory of Computing (STOC)*, Best Paper Award, 2025.
- [18] E. W. Dijkstra, “A Note on Two Problems in Connexion with Graphs,” *Numerische Mathematik*, vol. 1, pp. 269–271, 1959.

- [19] KTH Royal Institute of Technology. “KACTL: KTH Algorithm Competition Template Library.” (2026), address: <https://github.com/kth-competitive-programming/kactl> (visited on 09/06/2026).
- [20] H. W. Kuhn, “The Hungarian Method for the Assignment Problem,” *Naval Research Logistics Quarterly*, vol. 2, no. 1-2, pp. 83–97, 1955.
- [21] J. B. Orlin, “Max Flows in $O(nm)$ Time, or Better,” in *Proceedings of the 45th Annual ACM Symposium on Theory of Computing (STOC)*, 2013, pp. 765–774.
- [22] L. Chen, R. Kyng, Y. P. Liu, et al., “Maximum Flow and Minimum-Cost Flow in Almost-Linear Time,” in *Proceedings of the 63rd IEEE Symposium on Foundations of Computer Science (FOCS)*, Best Paper Award, 2022.
- [23] J. E. Hopcroft and R. M. Karp, “An $n^{5/2}$ Algorithm for Maximum Matchings in Bipartite Graphs,” *SIAM Journal on Computing*, vol. 2, no. 4, pp. 225–231, 1973.
- [24] J. Bentley, *Programming Pearls*, 2nd ed. Addison-Wesley, 2000.
- [25] R. Motwani and P. Raghavan, *Randomized Algorithms*. Cambridge University Press, 1995.